澳大利亚国立大学

计算机学院

# JHQ-GPU

## 面向表示的 GPU 近似最近邻搜索与工作负载级精排预算校准

Rui Luo

导师：Dr Mengxuan Zhang

中文阅读对照稿

章节、公式与图表编号对应英文稿。图内保留英文标签，便于与原始实验及英文稿核对。

COMP8800 — Advanced Computing Research Project

2026 年 10 月

## 摘要

降低近似最近邻（ANN）查询时间需要高效筛选与选择性精排。本文提出 JHQ-GPU，通过面向表示的执行、GPU 索引构建与工作负载级预算校准实现 JHQ。Cartesian 码本因子化将每张 256 项距离表替换为两张 16 项表，保持实数运算下的表示距离，并将表存储缩小八倍。候选并行扫描、打包访问与 lane 协作精排配合该设计。预算通过采样 top-$k$ 与较大参考预算的输出一致性校准，不需要真实标签或学习预测器。

RTX 5090 上的实验覆盖六个数据集。两组表在 22/24 配置领先；最大索引搜索 17.78 百万向量，常驻显存 20.44 GiB。全池校准在 12/12 单元满足相对最佳被测预算的 $10^{-3}$ 留出 recall 损失目标，最大损失为 0.0006；较小样本暴露工作负载相关质量风险。recall 0.97、batch 1,000 时，JHQ 在四个数据集中的三个领先，在 arXiv 上与 IVF-RaBitQ 接近平手。OpenAI 运行使用 TF32；同一 TF32 训练缓存上的 FP32 检查仍保留有利比较方向。相同 recall、batch 8 时，四个数据集均由 CAGRA fp32 领先。这些结果界定 GPU JHQ 在执行、质量与容量之间的取舍。



## 致谢

感谢 Mengxuan Zhang 博士指导本项目，并感谢本文引用的 JHQ 作者及开源 ANN/GPU 软件贡献者。


**研究与 AI 协助**。本文扩展与 Mengxuan Zhang 合作的 JHQ-GPU 工作 [21]，并以 JHQ 为基础 [16]。OpenAI Codex 协助实质性起草、修改、LaTeX、分析及代码 / 日志核查 [30]；ChatGPT 提供框架和审阅 [29]；Anthropic Claude 协助实现、实验及审阅 [4]。部分提示词、产物对应与记录局限见 `docs/ai-use-record.md`。论文作者对提交主张负责。

# 目录

**摘要** i

**致谢** i

**Notation and Abbreviations** vi

**1 引言** 1
- 1.1 背景与实际目标 . . . 1
- 1.2 问题定义与范围 . . . 1
- 1.3 三个相互关联的技术挑战 . . . 1
- 1.4 研究问题与可反驳主张 . . . 2
- 1.5 研究贡献 . . . 2

**2 背景与相关工作** 4
- 2.1 ANN 目标与评价指标 . . . 4
- 2.2 IVF、PQ、ADC 与残差精排 . . . 4
- 2.3 继承的 JHQ：旋转、Cartesian 编码与假设 . . . 5
- 2.4 本文使用的 GPU 执行概念 . . . 6
- 2.5 GPU 图搜索：SONG、CAGRA 与 BANG . . . 6
- 2.6 量化 ANN、快速查表与其他方案 . . . 7
- 2.7 批判性综合与研究缺口 . . . 8

**3 研究设计与测量** 9
- 3.1 将研究问题转化为受控比较 . . . 9
- 3.2 数据集、查询、真值与划分 . . . 9
- 3.3 测量边界、波动与显存核算 . . . 10
- 3.4 基线、调参网格与前沿构造 . . . 11
- 3.5 带版本的主张与证据登记 . . . 12
- 3.6 证据强度与有效性控制 . . . 13

**4 GPU 索引设计与构建** 14
- 4.1 端到端建库与搜索边界 . . . 14
- 4.2 训练、旋转与码本生成 . . . 15
- 4.3 IVF 分配、ID 与物理内存布局 . . . 17
- 4.4 常驻存储、工作区与索引生命周期 . . . 18
- 4.5 构建成本与已测试规模 . . . 19
- 4.6 索引表示与安全契约 . . . 21

**5 面向表示的 GPU 搜索** 23
- 5.1 查询路径与四种不同的排序对象 . . . 23
- 5.2 精确的 Cartesian LUT 分解 . . . 24
- 5.3 打包编码与候选并行的内存访问 . . . 26
- 5.4 通过多 block 扫描实现精确全局主码 Top-c . . . 27
- 5.5 共享完整 Key 阈值与执行调度 . . . 29
- 5.6 Lane 协作的残差评分与最终 Top-k . . . 32
- 5.7 作为独立原语的 IVF 探针选择 . . . 34

**6 工作负载级预算校准** 36
- 6.1 固定精排预算为何可能浪费 . . . 36
- 6.2 无需 GT 的采样输出一致性 . . . 37
- 6.3 预算搜索、嵌套性与并列安全 . . . 38
- 6.4 预算基线及其回答的问题 . . . 40
- 6.5 留出风险、校准成本与回本 . . . 40

6.6 策略失效模式与适用范围 . . . . . 41

**7 实验评估** **42**
7.1 实验协议 . . . . . 42
7.2 现代 GPU ANN 的召回率—QPS 前沿 . . . . . 42
7.3 笛卡尔查找表、打包加载与 lane 协作精排 . . . . . 46
7.4 全局 top-c、共享界与探针选择的正确性及消融 . . . . . 48
7.5 留出查询上的预算质量、吞吐量与回本 . . . . . 49
7.6 批大小、扫描组调度与工作负载偏斜 . . . . . 54
7.7 冷构建、常驻显存与已验证规模 . . . . . 56
7.8 数值稳健性、异常与召回率归因 . . . . . 59

**8 讨论与未来工作** **60**
8.1 三项跨层认识 . . . . . 60
8.2 两项深入的负结果案例 . . . . . 60
8.3 内部与外部有效性威胁 . . . . . 61
8.4 可检验的后续研究方向 . . . . . 62
8.5 有条件的部署建议 . . . . . 62

**9 结论** **64**
9.1 四个研究问题的答案 . . . . . 64
9.2 系统原则与展望 . . . . . 64

**参考文献** **65**

**A Reproducibility and Experiment Manifest** **69**
A.1 Source, Environment and Trained States . . . . . 69
A.2 Measurement Contracts . . . . . 69
A.3 Budget Harness Contract . . . . . 70
A.4 Supplementary Evidence . . . . . 70
A.5 Strict Selection, Small Batches and Artifact Access . . . . . 71

**B 补充结果与正确性检查** **72**
B.1 完整参数记录 . . . . . 72
B.2 正确性与数值证据 . . . . . 73
B.3 次要结果与证据状态 . . . . . 73
B.4 构建容量与精度锚点 . . . . . 74
B.5 材料获取 . . . . . 74

# 插图目录

1.1 问题、机制与证据 . . . . . 2
2.1 候选与精度的层级流水线 . . . . . 5
3.1 主机与设备计时边界 . . . . . 11
4.1 构建与查询架构 . . . . . 14
4.2 按列表排列的物理索引布局 . . . . . 17
5.1 查询阶段与选择边界 . . . . . 23
5.2 打包后的候选并行访问 . . . . . 26
5.3 局部覆盖与截断式两两合并 . . . . . 28
5.4 共享完整键过滤 . . . . . 31
5.5 十六 lane 残差精排 . . . . . 33
6.1 精排预算影响的工作范围 . . . . . 36
6.2 工作负载级校准与评测 . . . . . 37
6.3 嵌套与非嵌套候选池 . . . . . 39
7.1 E1 召回率与吞吐量前沿 . . . . . 43
7.2 TF32 与 FP32 精度前沿 . . . . . 45
7.3 小批次竞争区间 . . . . . 45
7.4 笛卡尔查找表分组 . . . . . 47
7.5 打包访存与精排对照 . . . . . 47
7.6 不同样本大小的留出风险 . . . . . 51
7.7 有限池风险预测与实测 . . . . . 53
7.8 校准后复用的直接累计时间 . . . . . 54
7.9 批大小与执行区间 . . . . . 55
7.10 已记录的构建成本 . . . . . 56
7.11 常驻显存分解 . . . . . 57
7.12 大数据集搜索前沿 . . . . . 58
8.1 Mailman 与面向排序编码 . . . . . 61

# 表格目录

1.1 技术贡献与证据边界 . . . . . 3
2.1 设计选择与研究缺口 . . . . . 8
3.1 研究问题与实验对照 . . . . . 9
3.2 数据集与训练配置 . . . . . 10
3.3 E1 主要调参维度 . . . . . 12
4.1 单向量存储模型 . . . . . 18
4.2 各构建阶段的行数组显存模型 . . . . . 20
4.3 原始输入 FP64 重编码审计 . . . . . 21
4.4 表示契约与审计覆盖 . . . . . 22
5.1 查询阶段的选择契约 . . . . . 24
5.2 笛卡尔查找表的资源权衡 . . . . . 25
6.1 预算对照及其作用 . . . . . 40
7.1 召回率 0.97 的跨系统比较 . . . . . 43
7.2 匹配配置下的 FP32 敏感性 . . . . . 44
7.3 小批次同召回率吞吐比 . . . . . 46
7.4 128 探针下的精排收益 . . . . . 48
7.5 严格计算分数选择检查 . . . . . 48
7.6 探针选择吞吐量比 . . . . . 49
7.7 全池预算校准 . . . . . 50
7.8 采样预算的留出质量 . . . . . 50
7.9 采样校准的经济性 . . . . . 52
7.10 500 个复用批次的直接成本 . . . . . 53
7.11 集成执行与调度收益 . . . . . 55
7.12 冷构建与常驻显存 . . . . . 56
7.13 E1 显存分配范围 . . . . . 58
8.1 有效性威胁 . . . . . 62
A.1 Component and run provenance . . . . . 69
B.1 E1 精确配置数量 . . . . . 72
B.2 校验覆盖范围 . . . . . 73
B.3 独立构建缓冲容量 . . . . . 74
B.4 同会话精度锚点 . . . . . 74

# Notation and Abbreviations

| Symbol | Meaning |
|---|---|
| $N, d$ | Number of database vectors and original dimension |
| $M, D_s$ | Number of primary subspaces and dimensions per subspace |
| $a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$ | Configured add, staged input, and independently allocated assignment rows |
| $n_b, S_{\text{stage}}$ | Pipeline buffer count and byte budget per input/rotated array |
| $\Pi$ | Square orthogonal transform; $q' = \Pi q$ |
| $B, B_r$ | Primary bits per subspace and residual bits per coordinate |
| $\rho(p), o_\ell$ | Input ID at physical position $p$, and start of IVF list $\ell$ |
| $k$ | Number of returned neighbours |
| $n_c, c$ | Routed candidates and primary survivors receiving refinement |
| $\alpha$ | Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$ |
| $S, \epsilon$ | Calibration sample size and allowed disagreeing result slots |
| $C_\alpha, \mathcal{R}_\alpha$ | Valid primary survivor set and returned ID set at budget $\alpha$ |
| $\alpha_{\text{ref}}, B_{\text{reuse}}$ | Reference refinement budget and number of subsequent reuse batches |
| $g_{\text{LUT}}$ | Number of groups in a factorised lookup table |
| $g_{\text{scan}}$ | Number of scan groups per query; zero labels the one-block path |
| $C_{\text{buf}}, T_{\text{blk}}$ | Scan buffer capacity and threads per block |
| $n_{\text{LUT}}(g_{\text{LUT}})$ | Number of LUT entries per primary subspace |
| $\kappa(D, p)$ | Packed complete-key encoding for bound publication |
| $\ell_{\text{ref}}$ | Number of cooperating lanes per refined candidate |
| $D_{\text{raw}}$ | Squared distance between original vectors |
| $D_{\text{P}}$ | Distance under the primary reconstruction |
| $D_{\text{H}}$ | Refined ranking score: reconstructed squared distance plus a query-only constant |

| Abbreviation | Meaning |
|---|---|
| ANN | Approximate nearest neighbour |
| ADC | Asymmetric distance computation (distinct from the ADC conference) |
| IVF | Inverted file |
| JHQ | Johnson–Lindenstrauss enhanced hierarchical quantisation |
| LUT | Lookup table |
| PQ | Product quantisation |
| QPS | Queries per second |
| GT | Ground truth |
| FP32 | IEEE single-precision storage and arithmetic, with multiplication mode stated separately |
| TF32 | Tensor-core multiplication format with reduced input precision and FP32 accumulation |
| CV | Coefficient of variation of candidate counts |

# 第 1 章引言

## 1.1 背景与实际目标

近似最近邻（ANN）通过避免穷举、全精度距离计算降低查询时间。CAGRA 等图方法通过导航限制访问，压缩 IVF 将选择性列表访问与低成本评分结合 [18, 28, 33]。被测嵌入为 768–3,072 维，最多 17.78 百万向量；Stella 原始 FP32 数组本身需要 67.81 GiB（第7.7节）。紧凑编码解决容量问题，但建表、查表、选择与启动开销决定其查询时间收益。

JHQ 将筛选与精排分开：紧凑主码评分候选，残差精排短名单 [16]。由此产生两个决策：如何执行广泛筛选，以及精排多少候选。复用训练索引时可分别改变执行或预算，隔离其效果。本文以 Recall@10、主机可见吞吐 / 延迟、常驻显存及构建成本评估这些决策，同时纳入竞争方法或未修改设计更快的配置。

## 1.2 问题定义与范围

数据库 $\mathcal{X} \subset \mathbb{R}^d$ 和查询 $q$ 的目标是原始平方 L2 top-$k$。实现先用 IVF 路由，再按 primary 重建分数 $D_{\mathrm{P}}$ 选最多 $c = \max\{k, \lceil \alpha k \rceil\}$ 个候选，最后按残差精排分数 $D_{\mathrm{H}}$ 返回。两种分数都不同于原始距离，精排值还带 query 常数（第2.3节）。选择过程的精确性针对计算分数与并列契约，最终 ANN 质量仍要对 GT 实测。

表示继承 JHQ 的平方正交旋转、Cartesian 主码和相对 primary 重建的残差。本文研究 GPU 训练 / 编码、物理放置、查询执行及单个工作负载级精排倍率。倍率以无标签查询输出和大预算参考的一致性选出，预期在稳定工作负载上复用，不是逐 query 学习控制器或绝对 recall 证书。

主跨系统比较在一张 RTX 5090、平方 L2 top-10、batch 1,000 和四个数据集上进行。六数据集支持组件与预算研究，大型索引另测。T5 在内部调度消融之外加入跨系统 batch 8/32 对比（第7.2.5节）。压缩索引常驻单卡，CPU 模型准备和主机查询传输仍属于实际架构。多 GPU、在线更新、主机残差分页及十亿规模部署不在已测系统范围。

## 1.3 三个相互关联的技术挑战

图 1.1把三个挑战连接到机制与证据。

首先，直接 GPU 移植可能忽略 primary 码本的数学结构。每 8-bit 子空间索引 256 个距离，其 Cartesian 坐标允许精确 16+16 分解，但进一步拆分会增加 lookup 和算术。挑战是利用可分性，同时不假定最小表最快。候选并行 word 访问与残差 lane 合作也需要不同映射。

第二，primary 优胜者可能集中在路由全集任何位置。并行组不能安全地各保留全局需求的一小部分。精确局部保留、全局合并与共享完整分数界提供可靠路线，却增加缓冲、staging 和同步。大 batch 可能已有足够 block，不均衡工作负载仍可能受益于分组，因而正确性与执行决策耦合。IVF 列表选择又是独立于向量保留的原语。

第三，固定倍率可能反复精排无关向量，小倍率则可能过早删除真近邻。大预算参考一致性无需 GT，却不能修复参考自身错误或保证未来查询 recall。样本与预算选择也耗时，必须联合评估质量、校准成本及复用。这些问题组织迭代项目记录，不把观察收益包装成预注册预测。

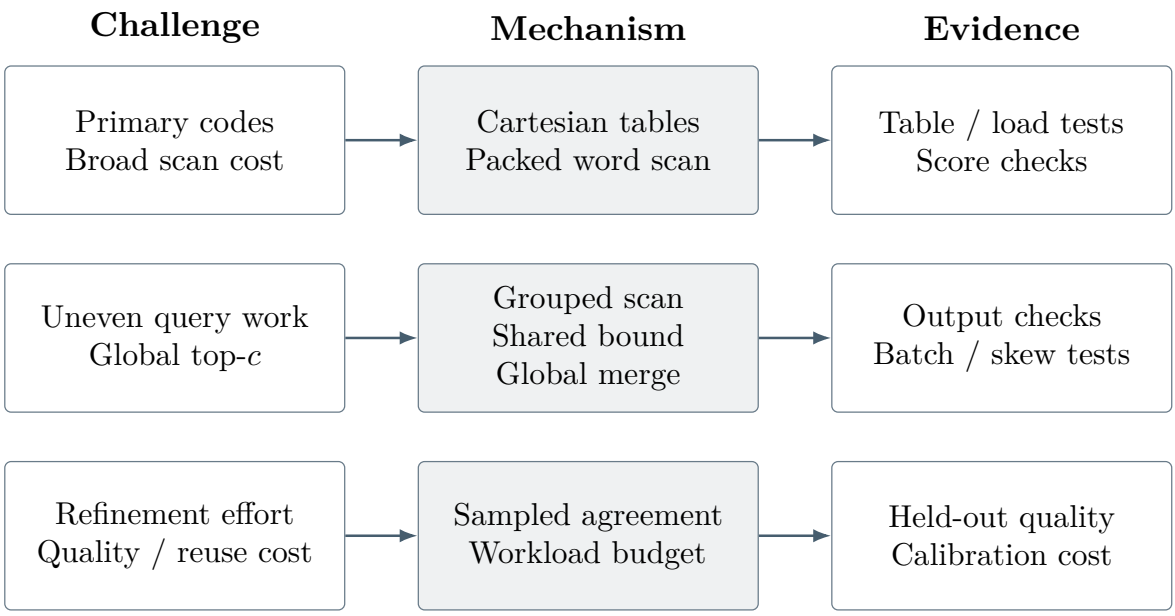



**图 1.1.** 问题、机制与证据。箭头组织研究问题，不意味着独立测量的提速比可以相乘，也不意味着无标签一致性保证 GT recall。

## 1.4 研究问题与可反驳主张

**RQ1：构建与容量。** 如何把 JHQ 层次训练、编码并组织成可搜索的 GPU 常驻索引，实际构建成本和容量是多少？证据为物理布局、train/add 记录和常驻核算（第7.7节）。构建失败或分配超过设备预算会限制容量主张，常驻样本不代表构建峰值。

**RQ2：正确高效的执行。** 哪些面向表示的评分、保留和调度选择，在满足分数与并列契约时改善 GPU 搜索？LUT、packed load、精排和扫描配对实验连接参考检查与 batch 敏感性（sections 7.3、7.4与7.6）。非并列选择错误挑战正确性，持续减速挑战执行选择。

**RQ3：预算质量与总成本。** 无标签一致性能否选出有用预算，什么抽样与复用条件下校准值得？互斥查询评价固定预算、抽样 / FULL、枚举及 GT oracle（第7.5节）。越阈挑战可靠性，包含校准后没有正节省则挑战净收益。oracle 是评估对照，不是可部署的无标签决策。

**RQ4：竞争区间。** 完整系统在共同主机到主机边界下，哪些实测 recall 区间具有竞争力？依据独立调优 E1 前沿和明确显存定义（第7.2节）。int8 图对手、高 recall 反转和未覆盖区间都是答案的一部分，不追求跨硬件、显存预算和请求大小的普遍排名。

第 3 章将这些问题转成受控比较，分别回答建成了什么、执行是否正确、工作负载需要多少工作，以及完整系统有没有有用竞争区间。

## 1.5 研究贡献

表 1.1列出三项关联项目贡献及继承基础。核心是具体表示–执行–预算组合的适配与评价。JHQ、Cartesian 量化、IVF、bitonic / radix、merge 与 CUDA Graph 仍属于前人工作。

**表 1.1.** 技术贡献与证据边界。

| | 继承基础 | 项目贡献 | 证据 |
|---|---|---|---|
| C1 | JHQ 旋转、码本与层次 | GPU 训练 / 编码、有界 batch 构建及列表有序 packed 表示 | 17.78M 向量搜索、常驻 20.44 GiB、重复 train+add 10.54–10.65 s、采样构建峰值 22.58 GiB |
| C2 | Cartesian 可加性与既有 GPU 原语 | 融合拆表评分、完整键分组保留 / 界、lane 精排与调度 | 两组 LUT 在 22/24 单元领先；batch 8 调度 2.727×；明确正确性范围 |
| C3 | 候选扩张和输出一致性思路 | 工作负载预算规则、有条件选择分析、留出对照与成本 / 风险评估 | FULL 的 12/12 单元相对损失达标；小样本揭示风险；四个单元直接测量校准复用 |

性能依赖批大小与质量目标；第七、八章报告不利区间和数值边界。

# 第 2 章背景与相关工作

ANN 系统结合候选访问、数据表示与选择过程。三者成本互相影响：紧凑编码可能需要昂贵评分器，准确评分也无法恢复此前被排除的向量。本章建立技术词汇，明确继承自 JHQ 的部分，并比较推动 GPU 执行与预算问题的替代方案。

## 2.1 ANN 目标与评价指标

设带稳定 ID 的数据库 $\mathcal{X} = \{x_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$，对查询 $q$，精确任务选择原始向量距离最小的 $k$ 项：

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

并列需要指定顺序，或允许多个等价答案。本文使用存储 GT 的 top-10 集合及其固定并列选择。ANN 在允许遗漏该集合成员的同时降低访问与评分成本。Recall@10（式 (3.1)）是经验集合恢复指标，较高实测 recall 不构成最坏情况的乘性距离保证。

本文关注四个成本轴：查询吞吐是在指定 batch 和边界下每秒完成的查询数；batch 延迟是一次调用耗时，吞吐本身不给并发服务尾延迟保证；构建成本包括明确计时的训练和编码；存储区分每向量持久字节与模型、工作区、context、临时分配。实际边界由第 3 章定义。一个方法可以改善某轴而损害另一个轴，同 recall、同显存与同构建投入是不同的比较。

调参产生有限运行点。Pareto 前沿保留 recall 和吞吐不能同时被其他实测配置改进的点。指定 recall 下，最快合格点是实际可用配置，插值则估计邻近点之间的值。未覆盖区间既不支持优势，也不证明不可达。比较系统使用不同码精度与索引大小时，这些区别尤其关键；估计器和端点容差见第3.4节。

## 2.2 IVF、PQ、ADC 与残差精排

IVF 围绕粗质心分区，列表保存归属于各质心的向量。查询选 $n_{\text{probe}}$ 个列表，得到含 $n_c$ 个候选的路由全集 $U(q)$。少访问列表可以省工，却可能在精细评分前排除真近邻。质心用于路由，其距离选择与选择数据库向量是两个不同问题。

PQ 把向量分成 $M$ 个坐标子空间，每个子空间用小型学习码本的标签表示 [17]。若 $\widehat{x}_m = \mathcal{C}_m[z_m]$，ADC 保持查询不量化，计算

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

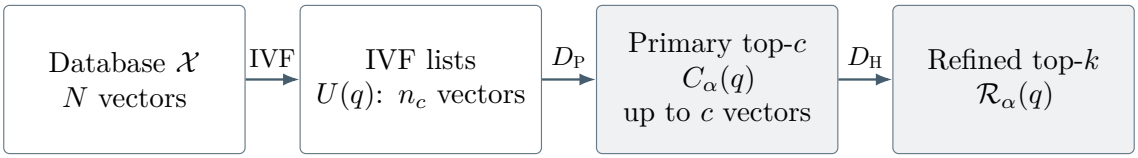
查询表预计算子空间距离，扫描执行 code load、查表与累加。乘积存在于子空间之间，学习子码本内部的质心未必还有坐标乘积。这决定第 5 章的精确内部拆表是否成立。

这里的“残差”有两种含义。IVF-PQ 常编码 $x - \mu_{\ell(x)}$，粗质心参与重建；JHQ 用 IVF 路由，却相对旋转后的 primary 量化重建定义残差。两者依赖不同；给 JHQ 残差重建再加 IVF 质心，会加入编码器未减去的量。原 PQ/ADC 框架及 GPU 实现也支持扩大候选后进行更精确的 refinement [10, 18]。

在至少有 $k$ 个有效路由候选的分析范围内，JHQ 保留有效 primary 集合 $C_\alpha(q)$，容量上限 $c(\alpha) = \max\{k, \lceil \alpha k \rceil\}$，精排返回 $\mathcal{R}_\alpha(q)$：

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

与 GT 求交得到各阶段覆盖对最终 recall 的上界，却不能定量归因各阶段损失；后者需要固定候选全集的反事实评分。精排可以改善保留向量之间的次序，无法恢复被排除 ID。




Query $q' = \Pi q$ stays FP32.
Primary codes score the broad scan.

Residual $r = \Pi x - \widehat{p}$.
Survivors decode $\widehat{p} + \widehat{r}$.

**图 2.1.** 候选与精度级联。IVF 选列表，primary 选向量，只有 survivor 读残差。层次结构是继承的；图定义逻辑集合与评分精度，不表示实测 recall 损失或 GPU 内核边界。

忽略路由与选择，广泛查表评分需要 $O(n_c M)$ 次 lookup，残差精排需要 $O(cd)$ 坐标工作。这一不对称推动选择性精度，却不能预测延迟，还必须考虑内存、算术、并行及候选选择。增加 probe 改变 $U$，增加 $\alpha$ 改变该全集内的 survivor；把二者都称搜索深度会掩盖差异。

## 2.3 继承的 JHQ：旋转、Cartesian 编码与假设

### 2.3.1 继承部分

JHQ 结合正交变换、解析 Cartesian 主码和较高精度残差层 [16]。本文变换为平方矩阵：$\Pi \in \mathbb{R}^{d\times d}$，$\Pi^\top \Pi = I$，$y = \Pi x$、$q' = \Pi q$，因此

$$\|q - x\|^2 = \|\Pi(q - x)\|^2. \tag{2.4}$$

原工作称其为 orthogonal JL 变换，但此配置不降维；实数距离精确保留，后续量化引入近似。实际 FP32 正交残差与 GEMM 误差需要独立检查。

把 $y$ 分成 $M$ 个 $D_s = d/M$ 维子空间。每子空间 $B$ bit，共 $K = 2^B$ 标签，由 $L = 2^{B/D_s}$ 个标量电平组合；实现要求 Cartesian 参数使 $L$ 为整数。令 $\Phi$ 为标准高斯 CDF，构造器使用

$$a_i = \sigma \Phi^{-1}\left(\frac{i - 1/2}{L}\right), \quad i = 1, \ldots, L, \qquad \mathcal{C}_m = \{a_1, \ldots, a_L\}^{D_s}. \tag{2.5}$$

所测 $B = D_s = 8$ 时 $L = 2$，$\mathcal{C}_m = \{-a, +a\}^8$，$a \simeq 0.67449\sigma$，每字节记录 8 个坐标符号。实现从向量能量估计 $\sigma$；单位归一化输入对应 $1/\sqrt{d}$ 尺度。该表示来自前作，不是 GPU 执行层的发明。

设 $\widehat{p}$ 拼接 primary 码字，残差 $r = y - \widehat{p}$，量化后为 $\widehat{r}$，层次重建 $\widehat{y} = \widehat{p} + \widehat{r}$。所测残差层每坐标 8 bit，因此主码 / 残差分别占每向量 $d/8$ 与 $d$ 字节，元数据另计。选择性精排改变读多少残差行，不改变存多少行；第 4–5 章把继承的层次搜索实现为常驻 GPU 流水线。

### 2.3.2 模型假设与分数语义

随机旋转可以分散坐标能量，为共享高斯启发标量模型提供动机，却不能强制一份实际数据库具有独立零均值高斯坐标。固定旋转下，经验均值 $\mu$ 变为 $\Pi\mu$，且 $\|\Pi\mu\| = \|\mu\|$。对

随机旋转取期望不等于把固定语料居中。即使球面均匀点，各坐标也受固定范数约束而相关。因此等距性是代数保证，分布近似是模型假设；第 8 章的居中编码 pilot 研究其实践后果。

标量最优性的术语也需精确。实现 primary 电平是高斯中分位点；恰好高斯、零阈值的一比特量化器，平方误差最优重建电平应为条件均值 $\pm\sigma\sqrt{2/\pi} \simeq \pm 0.79788\sigma$。因此继承的分位点构造本身不证明 MSE 最优。固定符号及正常数尺度时，$a$ 不改变 primary 排序：

$$D_{\mathrm{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1, +1\}^d. \tag{2.6}$$

它仍影响残差重建。本文依赖可分码本与实测质量，不依赖普遍高斯或最优量化器主张。

最后区分三种距离：$D_{\mathrm{raw}}$ 使用原始向量，$D_{\mathrm{P}} = \|q' - \widehat{p}\|^2$ 使用 primary 重建；实现的精排分数满足

$$D_{\mathrm{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.7}$$

query 常数不改变排序，但改变返回数值。对计算分数精确选择仍保留表示与路由误差，不等于原始向量精确重排。

## 2.4 本文使用的 GPU 执行概念

CUDA SIMT 把线程组织成 block 与 32 线程 warp，寄存器、block 共享内存及 global memory 服务不同复用范围 [25]。Coalescing 把邻 lane 访问组织成合适事务，不要求逻辑码位更少。primary 扫描把 lane 分给相邻向量，精排则让多个 lane 合作处理一个 survivor 的坐标；两者利用不同并行度，一个映射不必同时适合两阶段。

寄存器、共享状态和 block 几何限制 occupancy。更多驻留线程能隐藏延迟，更多单查询 block 也会重复表与局部缓冲。数据复用足以抵消加载与同步时，共享内存才有价值。这些概念推动第 5 章拆表、packed 访问与 batch 调度，吞吐消融测其组合效果。带宽饱和或具体 cache 瓶颈还需计数器及一致的物理字节模型。

选择本身也是重要计算。Faiss GPU 工作把高效距离与 warp 选择结合 [18]。bitonic、radix 和 merge 都是已有构件，对 $k$、长度和状态成本有不同取舍。AIR top-$k$ 和 GridSelect 进一步展示全局选择如何取决于工作负载及分数是否流式产生 [38]。本文把这些思路适配到 JHQ 的 probe 与 survivor 保留，不把原语称为新发明。

CUDA Graph 回放降低重复 launch 设置；capture、工作区分配与图形状变化属于生命周期，不是数学打分规则。共享阈值发布同样需要排序和内存访问契约；标量分数正确并不足以证明并发准入安全。第 5 章把正确性条件与性能测量分开，第 3 章区分设备 profile 和主机可见搜索区间。

## 2.5 GPU 图搜索：SONG、CAGRA 与 BANG

图 ANN 通过有希望的邻域导航替代列表扫描，以避免评分大多数向量。GPU 难点在于“发现邻居”与“选择扩展哪个邻域”之间的数据依赖。SONG 把搜索拆成阶段来暴露距离并行，并控制动态内存开销 [39]。这是替代访问策略，不是 JHQ 固定路由全集的另一种实现。

CAGRA 针对 GPU 并行设计建图与搜索，覆盖小 batch 和大 batch[28]。图连通性与 search width 形成不同于 IVF probe 的调参面。边与向量的存储形成容量取舍；量化向量

CAGRA 同样重要，所以只测 fp32 图索引不足以评价压缩索引。E1 包含二者，在共同边界下报告不同工作区间，不拿跨论文提速比相除。

BANG 针对另一容量区间：GPU 保存压缩向量，CPU/GPU 合作访问主机图 [37]。拆分工作结构获得容量，同时引入传输与协调成本。其十亿规模说明完整常驻之外的替代方案，不构成本论文的容量证据。本文 JHQ 压缩搜索索引保留在单卡，没有相同的主机图执行路径。

因此比较取决于访问规则性、表示与驻留方式，而不是图和量化的普遍优劣。少量查询时图导航可能有吸引力，列表扫描则暴露规则编码访问与 batch 并行。二者都可能受选择或移动限制；目标 recall 下谁更快，必须指定设备、显存和工作负载实测。E1 只测试 batch 1,000，JHQ 内部小 batch 收益不能回答相对图基线的问题。

## 2.6 量化 ANN、快速查表与其他方案

### 2.6.1 学习表示与利用表示结构

OPQ 通过学习空间分解和码本降低失真 [13]，其旋转为数据优化；JHQ 随机平方旋转支持廉价解析 primary 模型。取舍涉及拟合成本、重建保真和评分结构，更好的通用学习码本未必保留 16+16 精确分解所需的内部 Cartesian 乘积。

ScaNN 的各向异性量化根据最大内积检索的重要性加权误差，说明重建 MSE 不是唯一有用目标 [15]。其目标与实现环境不同于本文平方 L2 层次流水线。可比的是损失选择：面向排序的表示仍须检验实际近邻保留及执行成本。第 8 章停止的排序研究在 JHQ 内部提供了例子。

### 2.6.2 小查找表已有充分先行工作

PQ Fast Scan 把依赖 cache 的表改造成适于 CPU SIMD 寄存器的表 [1]；Quicker ADC 推广寄存器查找，引入不规则量化器与 split tables[2]。这些工作说明表形状要与执行指令共同设计。本文的范围是继承 Cartesian 坐标结构所支持的精确分解及其 GPU 实测；CPU SIMD 结果提供概念先例，不是本 GPU 的吞吐。FP32 结合顺序和 cutoff 并列仍需独立数值检查。

RaBitQ 也使用随机化量化与紧凑符号信息，具有特定距离估计器及理论误差分析 [11]；多 bit 扩展以存储换估计精度 [12]。其保证属于相应构造和假设，不能因双方都用旋转与二值组件就转移给 JHQ。JHQ 主符号选 survivor，显式存储的残差提供下一精度层；质量同时依赖截断与重建。

GPU IVF-RaBitQ 把该族整合成 GPU 建库与搜索，包括 GPU 距离计算和融合搜索 [33]，因此是直接相关的现代基线，不是“缺少 GPU 量化器”的背景证据。其融合执行说明紧凑编码还需要有竞争力的访问和选择流水线。E1 纳入多个 bit 预算，其高 recall 结果限制 JHQ 的宽泛优势主张。

### 2.6.3 精排工作量与自适应停止

候选扩张和重排已经存在于 Faiss 的 `k_factor` 及 cuVS refinement 控制 [7, 10]。Faiss autotuning 可用 GT 标签搜索 recall–时间取舍 [9]；学习式自适应终止预测足够搜索量 [19]，DARTH 围绕声明式 recall 目标组织早停 [6]。这些都是均匀工作量之外的相关替代。第 6 章的具体策略用大预算参考的无标签一致性，为整个工作负载选择一个 survivor 倍率，省去预测器训练，却继承参考的质量局限。这是策略范围，不表示自适应搜索或精排预算是新概念。

## 2.7 批判性综合与研究缺口

表 2.1按所解决约束比较替代方案。已有文献提供 GPU 图搜索、压缩距离、寄存器查找、快速选择和自适应工作量。因此贡献必须明确表示契约、集成决策及证据，仅说 “GPU 版本” 不足以定位贡献。

**表 2.1.** 设计替代及本文较窄的研究缺口。表比较机制，实测排名见第 7 章。

| 替代 | 可利用结构 | 主要取舍 | JHQ-GPU 问题 |
|---|---|---|---|
| IVF-PQ / OPQ | 学习子空间编码与 ADC | 失真、训练及子码本坐标耦合 | 内部 Cartesian 主码支持什么 GPU 评分? |
| Fast Scan / Quicker ADC | 小表与 SIMD | 表精度、形状、ISA 约束 | 精确拆分能否抵消额外 GPU 查找? |
| SONG / CAGRA | 选择性图导航 | 依赖访问、边存储与建图 | 常驻层次扫描在哪些区间竞争? |
| BANG | GPU 压缩评分与主机图 | 容量与通信 | 完整压缩常驻能达到什么范围? |
| IVF-RaBitQ | GPU 编码与融合搜索 | bit 预算、估计器与 probe | 哪些 recall 区间值得残差层次? |
| 自适应工作量 | 精排倍率、autotuning、学习 / 声明式早停 | 标签、参考质量、预测或校准成本 | 无标签工作负载一致性能否以可接受风险和总成本选工作量? |

第一项缺口是具体层次的执行契约：可分表与 packed 访问如何配合精确 primary 保留、并发界发布及残差评分。第二项是跨 batch 和偏斜的执行模式选择，因为拆分工作可能加的比省的更多。第三项是工作负载精排选择的质量与成本核算，明确区分参考一致性与 GT recall、选中臂吞吐与净节省。

这些问题推动固定继承表示、改变执行与工作量。更强学习编码和图导航仍是有意义的替代，但同时改动它们会掩盖组件问题。本文以数学契约、配对实验和竞争前沿评估该受约束设计，并保留分布、并列和校准假设，使结论具体而可检验。

# 第 3 章研究设计与测量

评估需要把系统干预与可观测的成本或质量变化联系起来。本文结合配对执行消融、留出查询上的预算选择，以及各系统独立调优后的前沿比较。这些实验回答不同问题：更快的内核、有效的校准策略和有竞争力的搜索系统，各自需要不同的对照。本章定义这些对照，以及解释结果所需的证据；第 4–6 章介绍机制，第 7 章报告实测结果。

## 3.1 将研究问题转化为受控比较

表 3.1明确比较单位，以及哪些观察会挑战各项研究主张。一个*实验单元*（*cell*）固定数据集、索引配置和工作负载设置；各实验臂是在该单元内替换的实现或预算。配对执行测量固定索引状态、查询、probe 数、精排预算和打分表示，除非这些正是指定的干预变量。配对比值为 $Q_{\text{treatment}}/Q_{\text{control}}$；几何平均对指定单元的乘性比值作汇总，各单元等权。

**表 3.1.** 把研究问题转化为可检验的比较。容量观察与配对执行干预需要不同对照。

| 问题 | 干预 / 比较 | 固定或单独核算的条件 | 指标与反证 |
|---|---|---|---|
| RQ1：构建与容量 | GPU 构建；缓存与冷构建生命周期；压缩与原始存储 | $N, d$、表示、路由、构建计时边界 | train/add 时间与常驻字节；构建失败或模型与分配差额限定可行性 |
| RQ2：执行 | LUT 分组、word 访问、精排 lane、分组扫描、共享界与 probe 选择 | 配对状态、工作负载、分数及参考输出语义 | 墙钟 QPS、阶段时间和输出检查；变慢或非并列差异挑战机制 |
| RQ3：预算 | 抽样 / 全池规则，对照固定 64、100、200、枚举与 GT oracle | 同一索引与路由；校准和留出查询位置互斥 | 留出 recall 损失、选中臂 QPS 与校准成本；越阈挑战可靠性 |
| RQ4：工作区间 | 独立调优的 JHQ、CAGRA、IVF-PQ 和 IVF-RaBitQ | 导出向量、GT、$k$、设备和主机到主机边界；显存另报 | 同 recall QPS 与覆盖；其他 recall 处更强的竞争者挑战宽泛优势 |

研究设计区分机制的实现效果与其原因解释。配对 word 访问消融可以确定该访问路径在所测流水线内的吞吐变化；把变化归因于 cache miss 或 occupancy，还需要硬件计数器。跨系统前沿则比较表示、算法与实现的整体选择，无法单独识别某个内核的贡献。第7.6节提供集成调度比较，避免把不同基线上的组件提速比直接相乘。

这些研究问题组织的是迭代开发记录，并非预注册方案：E1 发现 recall 覆盖不足后扩展了网格，工作负载精排需求的解释也来自预算扫描。低 recall 失利、高 recall 反转和失败方案同样进入评估。与量化参考的输出一致性、相对原始向量 GT 的 recall，按定义就是不同的量；预算规则使用前者，留出评估检验后者。

## 3.2 数据集、查询、真值与划分

表 3.2列出新版预算、LUT 与规模实验使用的六份数据导出；海报 E1 比较使用前四行。Vogue 是时尚图片嵌入，arXiv 是摘要嵌入；两个 OpenAI 数据集来自 Qdrant 的 DBpedia 导出 [5, 31, 32, 34]。BGE 使用意大利语 Wikipedia 配置，Stella 使用 TREC24 BioGen 语料导出 [35, 36]。规模来自准备后的实测文件，而不是当前上游数据卡。

**表 3.2.** 数据与 JHQ 训练清单。$n_q$ 为实际查询数；$n_t = 39n_{\text{list}}$ 是请求的粗聚类训练样本数。E1 使用前四行；全部使用平方 L2 及 $k = 10$。

| 数据集 | $N$ | $d$ | $n_q$ | $n_{\text{list}}$ | $n_t$ |
|---|---|---|---|---|---|
| Vogue | 932,328 | 768 | 1,000 | 4,096 | 159,744 |
| arXiv | 2,253,000 | 768 | 1,000 | 8,192 | 319,488 |
| OpenAI-1536 | 999,000 | 1,536 | 1,000 | 8,192 | 319,488 |
| OpenAI-3072 | 999,000 | 3,072 | 1,000 | 8,192 | 319,488 |
| BGE-M3 | 10,091,524 | 1,024 | 1,000 | 32,768 | 1,277,952 |
| Stella | 17,776,615 | 1,024 | 999 | 32,768 | 1,277,952 |

准备源码规定 L2 归一化，以及带种子的查询抽取与数据库移除。流式 OpenAI/BGE/Stella 转换器先删除非有限行，对范数大于 $10^{-12}$ 的向量归一化，并用 seed 42 无放回选择查询位置。Stella 查询来自 corpus 抽样，没有使用单独的 65 条官方 benchmark 查询；较短的实际导出得到 999 条查询。这些实验评估向量近邻，不评估带相关性标签的检索。两个 OpenAI 导出采用相同位置抽样规则；补证向量核对支持全部查询及 10,299 行 base 样本的按行对应，但未归档原始 source-ID 连接（sections 7.2与7.2.3）。

同一单元的所有方法读取同一查询和 GT 文件。GT 通过对导出 FP32 向量进行穷举平方 L2 搜索得到；流式转换器在数据库分块之间维护 top-$k$。评估沿用存储的 GT 顺序及其浮点并列选择。设 $\mathcal{T}_k(q)$ 为 GT 前 $k$ 个 ID，$\mathcal{R}_k(q)$ 为返回 ID 集合：

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

Vogue/arXiv 的 GT 行宽为 100，其余四份为 20，评分只比较前 10 项。返回重复 ID 不会重复得分；越界、重复和跨调用变化分别记录。

JHQ 采用 $M = d/8$、$B = B_r = 8$、8 次粗聚类迭代，并请求 100,000 条残差训练样本。搜索扫描复用 `arms6_cache` 训练状态，重新编码数据库；冷构建从没有该状态开始。源码从数据库抽取训练样本，与校准查询分开。预算实验以偶数查询位置构成 500 条校准池，奇数位置构成 500 条留出池，Stella 留出 499 条。每个单元在 $S = 32, 64, 128$ 下各抽 64 次，FULL 使用全部 500 条。GT 只参与离线 oracle 和留出质量评估，可部署的一致性规则使用搜索输出（sections A.3与6.2）。

准备源码和形状日志支持上述协议，但历史向量文件与训练状态的完整哈希缺失。证据登记区分已检查的准备流程与每次运行数据的不可变身份。

## 3.3 测量边界、波动与显存核算

主比较从主机内存中的 FP32 查询开始，到流完成后 top-10 ID 与分数回到主机为止（图 3.1）。区间包含 pinned staging、H2D、查询预处理、搜索和 D2H。CAGRA int8 查询量化计入其中，量化器训练和索引构建在区间外。资源句柄与输出缓冲复用。JHQ 回传到 pageable 数组，基线回传到 pinned 数组；CAGRA/PQ 经 Python API，JHQ/RaBitQ 经 C++。解释微小差距时保留这些接口差异。

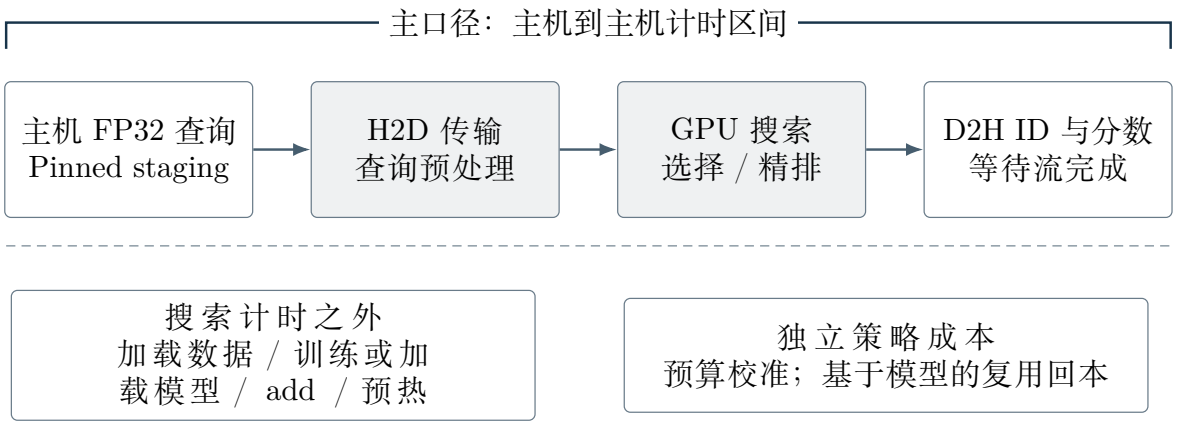



**图 3.1.** 计时边界。主机区间包含传输和查询相关计算；设备阶段 profile 解释其组成。单独 profile 与同步可能改变执行，因此阶段时间之和不能替代端到端延迟。

第 $r$ 轮有 $m_r$ 次计时调用，每次回答 $b$ 条查询：

$$Q_r = \frac{m_r b}{\sum_{i=1}^{m_r} t_{ri}}, \qquad Q = \operatorname{median}_r Q_r, \tag{3.2}$$

其中 $t$ 以秒计。E1 采用 $b = 1{,}000$ 和 5 轮测量。JHQ 每个变体每轮先做 1 次不计时调用，再通常计时 24 次；基线先做 3 次初始预热，每轮通常计时 24 次，对判定较慢的点减少次数，并保留实际次数。预热使 JIT 和 graph capture 不进入稳态计时。batch 调用的中位 / p95 延迟描述重复 batch 调用，不是逐查询或并发请求的延迟分布。

各轮吞吐的中位数与单次调用延迟中位数的倒数不同。在可得时同时保留二者、各轮最小 / 最大值及基线逐调用轨迹。有效慢调用保留在主估计中。各轮共享训练索引，预算抽样共享查询池：其次数衡量计时波动或抽样敏感性，不等于独立训练重复数。微小配对差异结合离散度和一致性解释，不对扫描单元附加依赖独立性假设的显著性主张。

主环境为 RTX 5090，可报告显存 32,607 MiB，驱动 595.71.05，CUDA toolkit 13.0 和 cuVS 26.08.01。运行获取协作 GPU 锁并保存环境快照；25 核容器配额内使用 16 个 OpenMP 线程。E1 记录功率上限 575 W，快照并不证明锁频或恒定宿主负载。JHQ Release 的 compute_80 PTX 由驱动 JIT[27]，库基线使用安装的构建。辅助设备常驻计时与阶段 profile 分别标注，因为 JHQ 主 API 接受主机查询。

显存核算区分持久编码数组、模型 / IVF 元数据、查询工作区和运行时 / context。进程 used 样本包含索引以外的分配；构建前后差值不包含之后创建的内存池。E1 中减 context 后的 JHQ 总量与 CAGRA 索引差值保留各自定义。组件模型解释常驻总量并显示未归因余量（sections 4.4与7.7），不能代替等显存性能比较；补证以 20 ms 设备采样另测构建峰值。冷 train/add 计时同样不含输入加载、此前的对象构造与主机模型设置。

**补证的测量边界。** 补证实验新增的检查见第A.4节。硬件型号及报告驱动 / toolkit 与归档一致，但计时属于另一会话。算术模式及缓存模型下的精度干预见sections 4.2与7.2.3；历史基线比较保留会话区别。

## 3.4 基线、调参网格与前沿构造

竞争方法涵盖图遍历（CAGRA fp32/int8）、乘积量化 IVF 与 GPU IVF-RaBitQ [17, 28, 33]，FP32 GPU 暴力搜索提供穷举参考。各方法扫描自己的实现参数（表 3.3），索引配置的

存储量和重建精度可以不同。因此主前沿衡量这些网格内指定质量下的可达速度，另行报告显存。

**表 3.3.** E1 主要调参维度。逐点参数和覆盖扩展保留在原始记录中；各方法使用不同的有限网格。

| 方法 | 记录的扫描维度 |
|---|---|
| JHQ | probe 与 $\alpha$；初始 probe 为 64/128/256，预算为工作负载基准的 0.5/1/2 倍；覆盖扩展最高至 probe 1,024，并增加更大 / 更小预算 |
| CAGRA fp32/int8 | 图度 32/64；search width 为 1/2/4/8；中间 top-$k$ 容量 32–512 |
| IVF-PQ | 8-bit 子码；$d = 768$ 时码长 192/384/768 字节，更高维度时 384/768/1,536；fp16/fp32 查表设置；probe 16–512 |
| IVF-RaBitQ | 每维 4/8/9 bit、支持的搜索模式、probe 16–1,024；OpenAI 另有 4,096-list 索引 |
| GPU 暴力搜索 | 穷举 FP32 平方 L2，采用相同主机计时边界 |

前沿保留 recall 与 QPS 上不被支配的点：若 $R_a \geq R_b$ 且 $Q_a \geq Q_b$，至少一项严格更好，则 $a$ 支配 $b$。包络可以由不同索引配置组成。对目标 recall $r$，实测估计为

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

没有合格样点时无定义。插值估计使用相邻前沿点 $(R_a, Q_a)$ 与 $(R_b, Q_b)$：

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

归档 parser 先把与目标相差不超过 $5 \times 10^{-4}$ 的前沿点视为直接匹配，包括接近端点的情形；否则只在实测范围内部插值，不对线段外推。差距接近时报告两种估计：前者选实际达到阈值的配置，后者降低网格间距不等的影响；二者都不构成等存储比较。

parser 把失败与非法输出诊断保留在有效点前沿之外。仅有跨调用 ID 变化不会导致删点，原始记录区分集合变化与顺序变化。隔离后的大型基线重跑替代受污染构建，Stella 因 batch 1,000 被跳过的运行不贡献搜索点，决策见第A.2节。论文从全部明确列出的基线族重新构造前沿，包含 CAGRA int8 及不利工作区间（图 7.1）。历史 ADC 与海报图保留各自版本及测量边界。

T5 还保留目标低于某方法前沿最小 recall 时已经达标的最快实测端点；插值未定义不表示更高 recall 的配置不符合目标。

## 3.5 带版本的主张与证据登记

证据登记 `data/ch3/audit.json` 标识配置、测量边界和分析输入，各实验族保留独立版本及对照。哈希标识已读取文件，历史可执行文件和训练状态身份仍不完整。附录 A 列出可获得的复现输入，仓库补充材料给出登记字段。[1]

`scripts/build_ch7.py` 重新生成主要图表，包括 图 7.1、7.4与7.11；`scripts/audit_ch3.py` 核查统一协议。这些是离线重算，重复 GPU 测量还需所列二进制程序、数据和训练状态。

[1] `docs/supplementary-evidence.md`。

## 3.6 证据强度与有效性控制

精确性声明指明计算分数与排序前提，原始向量 recall 还受路由和重建影响。独立参考的覆盖有限，配对计时保留工作负载对照，字节 / 回本模型保留核算前提。总体与排序条件见sections 6.2与6.3；第8.3节汇总共享索引、有限查询池、调参网格、接口及单设备带来的限制。

# 第 4 章GPU 索引设计与构建

GPU 构建训练并编码 JHQ 表示，以有界临时存储生成列表顺序数组 [16]。查询内核与构建实测见第5章与第7.7节。

## 4.1 端到端建库与搜索边界

索引有三个生命周期阶段：训练生成可复用模型状态，添加编码并排列数据库行，搜索使用完成后的数组。实验必须区分这些阶段。训练状态缓存避免重新学习模型，但仍需要数据库编码与列表构建。冷训练加添加，与复用状态后的查询，测量的是不同操作。

主机持有输入向量并提供训练样本，构造方形变换和解析主码码本。GPU 矩阵乘法应用变换；设备训练路径拟合粗粒度 IVF 中心及标量残差码本。添加阶段传输有界批次，编码两级表示，确定 IVF 成员关系，再写成统一的列表顺序表示。因此，主机编排与设备计算共同组成 GPU 常驻搜索系统。

模型状态包含变换、主码码本、残差码本和 IVF 中心。数据库状态进一步包含列表偏移、原始行 ID、主码、残差码及修正项。搜索工作区的逻辑寿命更短：旋转查询、中心分数、探针偏移、LUT 和保留候选缓冲区在下一批被覆盖，但分配的内存保留复用。捕获的 CUDA graph 属于执行状态，引用这些现有数组。

已核对的 API 通过一次成功的 `add` 构造固定快照，并拒绝后续 add。搜索该快照时物理位置保持固定。数据变更通过重建替代快照处理，形成简单的批量重建生命周期；可变插入 / 删除及快照的事务发布还需要额外索引管理。

搜索接收主机查询并返回主机 ID 及计算分数。分数中的查询常数及其对范围阈值的影响见式 (5.20)，实测主机到主机调用边界见第3.3节。

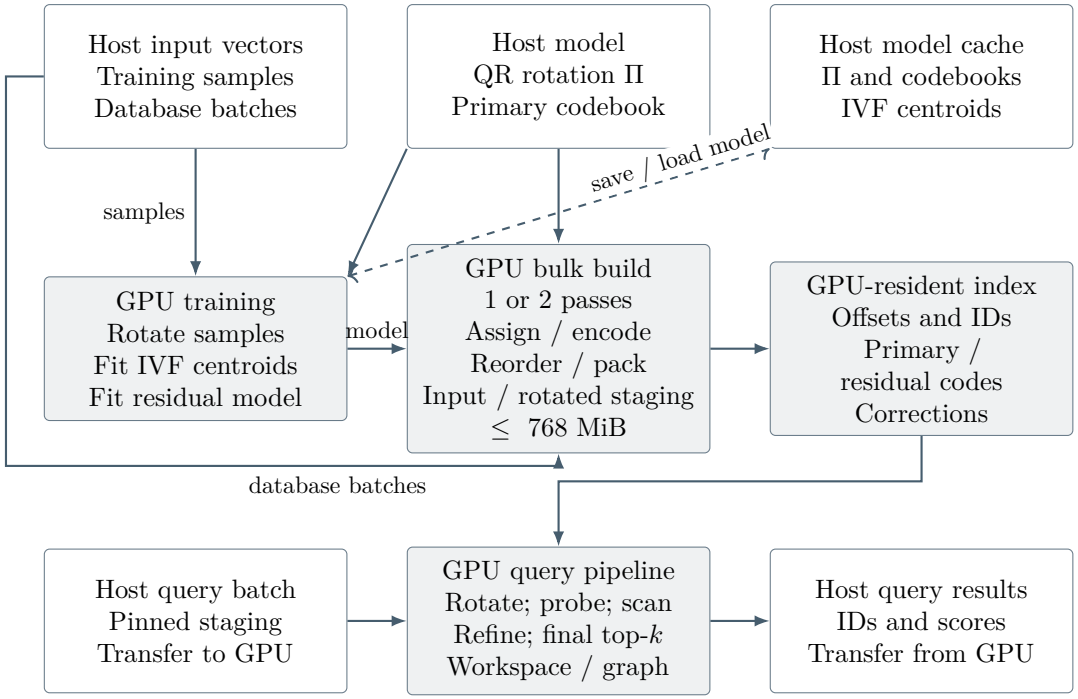



**图 4.1.** 已实现的建库与查询边界。主机负责采样、正交矩阵构造及解析主码码本生成；评测配置在 GPU 上拟合 IVF 与残差模型。缓存保存训练状态，完成后的数据库表示常驻设备。虚线表示训练状态复用。暂存标注仅约束源码默认的原始 / 旋转浮点缓冲区，其他流水线 scratch 另计。

该设计从主机内存分批传输构建输入，同时将压缩数据库保留在 GPU。它没有提供依赖磁盘的搜索路径，也不意味着任意大的压缩索引都能放入单个设备。

## 4.2 训练、旋转与码本生成

### 4.2.1 变换与采样

实现在主机生成带 seed 的高斯矩阵，通过 LAPACK QR 构造方形正交矩阵 $\Pi$。对行优先训练数据 $X_t$，设备矩阵乘法计算

$$Y_t = X_t\Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

输出保留维数 $d$。这是正交坐标变换；QR 构造及设备乘法存在有限精度误差，其测量属于第4.6节的 C0。

测试程序按固定步长选择数据库行。主码 / IVF 样本为 Vogue 159,744 行，arXiv 及两个 OpenAI 各 319,488 行，BGE 与 Stella 各 1,277,952 行，均为 $39n_{\text{list}}$。这与 FAISS 默认的每质心 39 训练点警告阈值一致 [8]，是采样规则，不是八轮 IVF 拟合的统计保证。残差训练另用 100,000 行等距样本，成本不随较大粗训练样本增长。

历史受控扫描固定主模型和 IVF 中心，仅改变 Vogue 与 OpenAI-3072 的残差训练样本量，覆盖 $B_r \in \{4, 8\}$ 和 32/128/256 探针。12 个配置中，100K 相对全量的召回率差值最大为 0.0018：11 项较低、1 项相等。增加到中间样本量没有带来单调提升，残差码本身也发生较大变化。这支持有界训练成本的取舍，同时保留可测量的质量损失。该扫描使用 25 轮残差 Lloyd，不能替代之后六个数据集的评测。历史诊断及输入哈希见 `data/ch4/review-audit.json`。

### 4.2.2 为何保留解析主码模型

主码构造器由样本估计共同尺度：

$$\sigma^2 = \frac{1}{n_t d}\sum_{i=0}^{n_t-1}\|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

在 $B = D_s = 8$ 时，笛卡尔码本使用 $\{-a, +a\}$ 两级，$a = 0.674\sigma$；每字节表示八个符号位。这保留 JHQ 的解析构造 [16]，支持逐坐标 $O(d)$ 编码，以及第5.2节的小 LUT 精确因子化。对每子空间 256 个码字自由执行 Lloyd 优化，通常会破坏这种可分离性。

固定符号重建 $\widehat{p} = as$，$s_j \in \{-1, +1\}$ 时，

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

当 $a > 0$，这些固定符号码的实数算术主码排序与尺度无关。这一代数性质无需高斯坐标假设。但尺度仍改变残差 $r = \Pi x - \widehat{p}$、拟合残差码本和精排顺序；$a = 0$ 是退化情况，浮点并列仍须处理。

早期前沿诊断说明了取舍。在 Vogue、$M = 96$、$B_r = 4$、128 探针下，早期构建与解析主码构建的 primary-only recall 分别为 0.6618 和 0.6342，而完整系统为 0.9452 和 0.9409。残差精排缩小了观测差距。这是包含残差训练变化的历史跨版本比较，不能作为仅替换主码码本的单因素因果估计。本设计选择的是 GPU 执行成本较低的结构化表示；第七章评价其完整质量与成本前沿。

### 4.2.3 粗分配的过滤与精算

IVF 中心从变换样本的等距行初始化，执行配置的八轮更新。分配按 $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$ 排序。源码提供 FP32 与加速分配路径；满足通常的维数约束时，默认用 int8 乘积过滤，再用 FP32 重算有歧义的选择。

**引理 4.1** (具有认证分数区间的分配)**.** 令 $\widetilde{y}, \widetilde{\mu}_\ell$ 为反量化近似，$e = y - \widetilde{y}$，$f_\ell = \mu_\ell - \widetilde{\mu}_\ell$。保持中心范数精确时，近似分数 $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ 的实数算术误差满足

$$E_\ell = 2\big(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|\big). \tag{4.4}$$

若每个目标分数均处于有效区间 $[L_\ell, U_\ell]$，仅丢弃 $L_\ell > \min_j U_j$ 的中心，并精确处理剩余分数，则保留最小值及约定的并列规则。

证明. 将内积误差展开为 $\langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle$，应用 Cauchy–Schwarz。被丢弃分数大于某个保留分数的上界，因此不能取得最小值；保留区间相等情形则不会丢失可能的并列者。 □

实现用保守范数界与 FP32 余量常数实现这一过滤与精算模式。引理说明有效界为何允许精确排除；若要求与每种 FP32 累加顺序逐位一致，还需要覆盖相应算术误差的界。已归档诊断对比 FP16 路径，在 Vogue、BGE、Stella 尾批次分别记录 0/14,838、6/31,748、3/16,359 个分配差异。这是有限实现证据，不是完整 FP32 参考认证；冷建日志也未追踪实际执行的分配精度。

### 4.2.4 残差拟合与迭代预算

IVF 拟合先于残差拟合。后者用固定主模型编码样本，形成 $r = \Pi x - \widehat{p}(x)$，独立于分配的 IVF 中心。每个子空间拟合一个含 $K_r = 2^{B_r}$ 项的有序标量码本，由其 $D_s$ 个坐标共享。GPU 估计器排序残差值，再执行一维 Lloyd 更新；评测配置的上限为 2,000 轮。

历史 100K 样本诊断支持质量收益递减下的固定预算选择：2,000 相对 25 轮提升 0.0063–0.0076 recall。后续变化可能超过 BGE 与 Stella 已报告的两次运行波动，因此该诊断未确立统计上严格的平台。诊断还观察到稀疏尾部 bin 持续变化；这是对所实现浮点更新的观察，不是精确 Lloyd 下降定理失效。包括 Stella 在内的冷日志达到上限，因此将训练描述为有预算的迭代拟合。早期协议在上限 20,000 时，Stella 全量与 100K 样本的结果分别为 0.9921 和 0.9920。

### 4.2.5 构建工作量与观测不均衡

稠密 QR 一次成本为 $O(d^3)$，旋转为 $O(n_t d^2)$；直接粗分配每轮为 $O(n_t n_{\text{list}} d)$。解析主码构造写入 $O(MKD_s)$ 个值，残差提取访问 $O(n_r d)$ 个坐标，之后排序并进行标量更新。这些表示工作量，并非延迟预测。同步阶段诊断将 BGE / Stella 约 83% 训练时间归于 IVF k-means（约 2.97 s）。它使用一个添加缓冲区并强制同步，独立于第7.7节中的正常重叠计时。

列表不均衡进入第5.5节的扫描规则；八轮 IVF、等距种子和有限采样各自对该不均衡的贡献尚未单独隔离。

**被测实现的算术精度。** 评测实现 v69 使用 FP32 矩阵存储，但在 $d \geq 1{,}536$ 时启用 cuBLAS TF32 tensor-core 算术：编译期 faithful 分支将覆盖值置空，随后落入按维度选择的默认分支。这影响两个 OpenAI 数据集在训练、add、查询时的旋转及质心 GEMM，环境变量无法

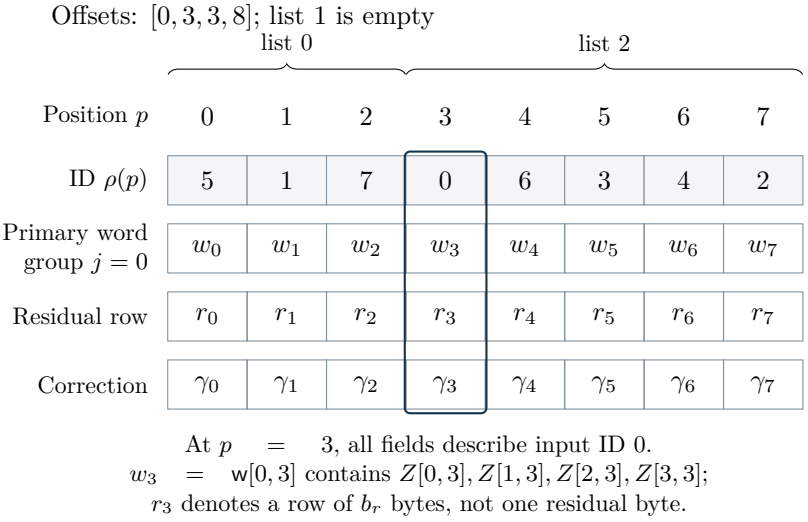



**图 4.2.** 同一物理位置连接 ID、打包主码 word、残差行与修正项。示意的八行索引偏移为 $[0, 3, 3, 8]$，其中有一个空列表。主码 word 打包同一向量的四个子空间。数值与列表大小均为示意。

覆盖该编译分支。因此 FP32 存储不代表完整 FP32 乘法。FP32 变体 v72 修正了这一开关；敏感性实验复用同一 TF32 训练缓存，只改变 add / 查询算术，没有以 FP32 重新训练模型(第7.2.3节)。

## 4.3 IVF 分配、ID 与物理内存布局

### 4.3.1 从数据库 ID 到列表顺序位置

输入行 ID 为 $i \in \{0, \ldots, N-1\}$。粗分配根据计算所得中心分数，将变换后的向量放入某个列表。数学分配目标是

$$a(i) = \arg\min_{0 \le \ell < n_{\text{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

具体实现使用选定的数值分配路径。主码与残差表示编码不减去 $\mu_{a(i)}$；列表成员关系独立于重建，限制之后的搜索集合。

构建器将分配结果与输入 ID 序列共同排序。令 $\rho(p)$ 为排序后物理位置 $p$ 对应的输入 ID，$o_\ell$ 为列表 $\ell$ 的起点。完成后的数组满足

$$0 = o_0 \le o_1 \le \cdots \le o_{n_{\text{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

偏移通过已排序分配值上的 lower-bound 查找获得。相邻偏移相等表示空列表。按分配结果排序不保证不同独立构建中，同一列表内的行具有稳定顺序。位置在完成后的快照中稳定，其含义是全部关联数组共同描述的行 $\rho(p)$。

常驻 ID 数组实现 $p \mapsto \rho(p)$。逆映射 $\rho^{-1}(i)$ 在数学上存在，但搜索无需再常驻一份逆数组。两遍构建器临时构造逆映射，将输入批次散写到最终位置。查询结果把物理位置翻译回输入 ID；第五章在主码并列时使用位置，在最终并列时使用数据库 ID。

**表 4.1.** 每向量字节数。JHQ 含编码、ID 与修正项；SQ8 仅计向量编码；CAGRA int8 模型计向量编码及 32 个四字节邻居。均排除偏移、模型参数、工作区及其他运行时分配，不代表完整进程总量。

| $d$ | 主码 | 残差 | ID + 修正 | JHQ | SQ8 | CAGRA int8 | FP32 |
|---|---|---|---|---|---|---|---|
| 768 | 96 | 768 | 8 | 872 | 768 | 896 | 3,072 |
| 1,024 | 128 | 1,024 | 8 | 1,160 | 1,024 | 1,152 | 4,096 |
| 1,536 | 192 | 1,536 | 8 | 1,736 | 1,536 | 1,664 | 6,144 |
| 3,072 | 384 | 3,072 | 8 | 3,464 | 3,072 | 3,200 | 12,288 |

### 4.3.2 精确地址与表示宽度

令 $Z[m,p]$ 表示物理位置 $p$ 在子空间 $m$ 的主码字节。中间的 subspace-major 副本使用字节偏移 $mN+p$。最终打包为

$$\mathsf{w}[j,p]=\sum_{t=0}^{3}2^{8t}Z[4j+t,p],\qquad \mathrm{addr}_{\mathrm{byte}}(\mathsf{w}[j,p])=4(jN+p). \tag{4.7}$$

这里 $j=0,\dots,M/4-1$。word 布局要求 $M \bmod 4=0$，启用该布局的构建对其他值抛出异常；另行编译的字节布局保留原表示。每份打包数组占 $NM$ 字节，但打包期间新旧副本同时存活，占 $2NM$。一遍构建器临时 gather 成 vector-major 的 $[N,M]$ 主码数组，再转置和打包，每份旧副本用完后释放。两遍构建器直接 scatter 到 subspace-major 数组，然后执行相同的最终打包。搜索时的 lane 映射见第5.3节。

残差码保持 row-major，每向量占 $b_r=\lceil dB_r/8\rceil$ 字节。八比特配置中，行 $p$ 的坐标 $t$ 位于字节偏移 $pb_r+t$。四比特路径中，它位于 $pb_r+\lfloor t/2\rfloor$，偶数坐标在低 nibble，奇数坐标在高 nibble。每物理行另存一个四字节修正项：

$$\gamma[p]=2\langle\widehat{p}(x_{\rho(p)}),\widehat{r}(x_{\rho(p)})\rangle. \tag{4.8}$$

gather 或 scatter 时，其值必须与对应行的编码共同移动。因此，即使两个编码数组采用不同布局，残差访问仍重建主码筛选识别的同一对象。

这一分离适应工作负载的两种访问模式：主码在列表顺序中广泛扫描，完整残差行仅在选定位置读取。设备保留原始 FP32 向量可提供精确重排，但要额外占用 $4Nd$ 字节，并改变所报告的搜索设计。这里精排压缩重建，质量影响由第七章评估。

## 4.4 常驻存储、工作区与索引生命周期

### 4.4.1 存储字节与分配字节

八比特残差配置具有 int8 量级的常驻表示。主码每子空间一字节时，行载荷为

$$S_{\mathrm{rows}}=N\left(M+\left\lceil\frac{dB_r}{8}\right\rceil+8\right),\qquad S_{\mathrm{offsets}}=4(n_{\mathrm{list}}+1). \tag{4.9}$$

额外八字节保存四字节 ID 与 FP32 修正项。在 $M=d/8$、$B_r=8$ 时，编码占 $9d/8$ 字节，比 SQ8 向量的 $d$ 字节多 12.5%。因此，无图层次结构是一种设计选择，不能据此声称独有的 int8 压缩优势。

CAGRA 比较使用图度数 $d_g=32$ 时的 $d+4d_g$ 字节，与测试程序的 32-bit 邻居 ID 一致。在 Stella 上，JHQ 行载荷为 19.20 GiB，成功的度数 32 CAGRA int8 记录报告索引分配

19,656 MiB（19.20 GiB）。尽管计量边界不同，两者接近仍是必要背景。第七章同时报告 int8、fp32 分配与搜索质量及成本。JHQ 的特点是无图构建和查询可调的残差访问预算；降低 $\alpha$ 不释放常驻残差码。

基本 FP32 模型数组中，变换占 $4d^2$ 字节，物化主码码本占 $4MKD_s$，残差码本占 $4MK_r$，IVF 中心与范数占 $4n_{\text{list}}(d+1)$。选定路径还分配派生中心副本和编码级值。查询工作区包括两个 $[b_{\text{cap}}, d]$ FP32 查询矩阵、$[b_{\text{cap}}, n_{\text{list}}]$ 中心分数、LUT、探针及候选缓冲区。可选残差 LUT 另占 $4b_{\text{cap}}dK_r$ 字节；默认精排直接计算残差分数。分组工作区遵循式 (5.13)，其容量可大于当前扫描所需：已记录的 128 探针样本即使走 query-centric 路径，也保留约 93.8 MiB。

原始输入由调用方持有。构建暂存占用 $4n_b a_{\text{in}}d$ pinned-host 字节，每个排序 / 逆映射数组占用 $4N$ 设备字节。分配乘积在 gather 前释放；equations (4.10)与(4.13)分别给出独立容量。完整对象生命周期表保留在仓库补充材料中。

### 4.4.2 复用、恢复与失效

缓存保存旋转、码本和中心。key 混合模型参数及训练样本的等距校验，但不含代码版本，也不覆盖全部数据库字节。因此，复现需要模型来源，并在实现或训练数据变化时显式使缓存失效。搜索实验复用训练状态，冷建记录则构造独立实例；实例间 recall 接近不能证明模型逐字节一致。

缓存命中仍先执行构造器 QR 与库预热，之后训练调用才检查缓存；再加载 / 上传模型、编码数据并构建列表。Stella 的 6.67 s add 记录只是该恢复路径的一个组件，不是实测进程重启时间。add 调用包括初始工作区分配；调度校准与搜索预热在这一孤立 add 数字之外。

固定 $k$ 时，改变 $\alpha$ 会改变候选预算，无需重新训练或添加向量。搜索在捕获前扩展不足的工作区，并在实际批大小、候选数、探针数、精排 lane 数、扫描组数或共享界设置变化时重新捕获。校准尝试预算时可绕过捕获。释放索引会销毁 graph 和设备分配；缓冲区由索引持有，在本文评测的顺序调用之间复用。

## 4.5 构建成本与已测试规模

### 4.5.1 有界输入批次与两遍构建

一次旋转全部 $N$ 个原始向量需要两个 FP32 设备数组，共 $8Nd$ 字节。构建器改用 pinned 主机内存和设备原始 / 旋转缓冲区分批暂存。若有 $n_b$ 个流水缓冲区、配置 add 行数 $a_{\text{add}}$、每数组字节预算 $S_{\text{stage}}$，源码暂存行数为

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

默认三个缓冲区、128 MiB 预算，使输入 / 旋转设备浮点数组至多占 768 MiB。分配乘积和编码 scratch 是独立分配。

一遍路径先编码输入顺序的主码 / 残差数组，再 gather 成列表顺序，临时与最终压缩副本共存。容量规则预测内存不足时，两遍构建先分配列表、排序 ID、构建偏移，再次读取输入，旋转、编码并直接 scatter 到最终位置。它用额外一次输入扫描和旋转换取较少的同时存活压缩副本。两遍路径的默认分配可在原始输入与反变换中心之间比较；正交性给出相应的实数算术恒等式。

**表 4.2.** 行数组的解析占用（GiB），$d = 1{,}024$、$M = 128$、$B_r = 8$。BGE 使用一遍，Stella 使用两遍；Stella 一遍列为反事实容量分析。这些阶段值不含流水线、模型、偏移和运行时成本。

| 数据集 | 最终行载荷 | 一遍 gather | 两遍 repack |
|---|---|---|---|
| BGE | 10.90 | 23.05 | 12.11 |
| Stella | 19.20 | 40.59 | 21.32 |

### 4.5.2 阶段占用与进程峰值

源码的一遍路径分支估计为

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes.} \tag{4.11}$$

其中依赖 $N$ 的项，是 gather / transpose 时同时存活的十个数组的精确载荷：输入顺序的主码、残差和修正项；分配结果及排序顺序；最终 ID、残差和修正项；vector-major 与 subspace-major 主码副本。共计 $3NM + 2Nb_r + 20N$ 字节。额外 1 GiB 是容量启发式的预留，不是其他全部分配的精确核算。

两遍构建在最终 repack 时，同时保留 ID、残差、修正项和新旧主码数组：

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

这一表达式对该阶段的这些数组精确成立。更早阶段还保留流水线 scratch，scatter 时另有临时逆置换。因此，repack 表达式本身不是两遍构建的整体峰值。取整个生命周期最大值时，还需计入模型、排序 / 分配器 scratch、运行时上下文及之后的工作区分配。

表 4.2量化了压缩副本的节省。Stella 冷日志选择两遍，报告式 (4.11) 为 41.6 GiB，尾部可用为 30.6 GiB。另行采样的常驻进程总量包括上下文及查询工作区，BGE 为 11.93 GiB，Stella 为 20.44 GiB。容量模型见第4.4节。补证以 20 ms 间隔采样，BGE 在 add 开头观测 26.08 GiB，Stella 在 repack 阶段观测 22.58 GiB。这是含上下文的设备采样最大值，可能漏掉更短暂分配。

分配乘积容量使用独立配置行数 $a_{\text{assign}}$，由 `JHQ_ASSIGN_BATCH` 或默认值选择：列表数至少 8,192 时为 32,768，否则为 8,192。

$$S_{\text{dots}} = 4n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

分配容量不必随 $a_{\text{in}}$ 或某次调用的实际处理行数缩小。表 B.3核对四种默认配置：1,536 / 3,072 维虽暂存行数不同，分配乘积均占 3 GiB。BGE / Stella 在 add 入口附近仍分配三个各 4 GiB 数组，共 12 GiB。23.05 GiB 的 gather 载荷属于后续阶段，不能直接再加 12 GiB 作为真实峰值。峰值模型应对各阶段实际存活数组（含流水 scratch）取最大值。Stella 两遍 repack 的较小载荷加模型 / 上下文，解释了其数据库更大却观测峰值更低。

### 4.5.3 构建测量

第7.7节通过重复空缓存构建、较宽启动区间和采样设备峰值评价上述阶段模型，并报告基线构建与大数据集搜索。计时边界沿用第3.3节；上述构建算法和分配生命周期决定哪些成本可以重叠。

## 4.6 索引表示与安全契约

历史上，32-bit 的 $n_t d$ 在 $2^{31}$ 处溢出，导致主码尺度归零；后续进程又因 cache key 不含代码版本而加载了错误模型。分块计算范数并以 double 累积修复了溢出。这一故障要求分别核查模型几何、编码行与放置关系，即使粗路由成功也不例外。

### 4.6.1 已完成的表示证据

10 月 13 日的残差下界 pilot 从 arms6 训练缓存导出 II、码本、ID、偏移、修正项及逻辑解码后的打包编码，覆盖完整的 Vogue（932,328 行）、arXiv（2,253,000 行）和 OpenAI-3072（999,000 行）数据库。

码本检查发现可分离性不匹配、标量级值不匹配、残差码本无序对均为零。独立解码导出的主码 / 残差后，FP64 循环遍历每一行，核对 $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$。三个数据库的最大修正项绝对误差为 $1.28 \times 10^{-7}$–$2.77 \times 10^{-7}$；采样 GPU 返回 top-10 分数的最大相对误差为 $2.38 \times 10^{-7}$–$4.85 \times 10^{-5}$，定义为 $|s_{\mathrm{GPU}} - s_{64}|/s_{64}$，参考含查询范数常数。这些是导出表示的检查，不是从原始输入向量重新编码；逐数据集诊断保留在仓库补充材料。

在七个工作负载、896 个查询实例中，pilot 对 72,800,590 个路由候选实例检查所提残差下界与 FP64 重建距离，未报告违反；同一集合也支持 FP64 组合分数恒等式检查。GPU 返回分数检查仅覆盖 top-10 槽位，并跳过路由映射中不存在的 ID，因此尚未完整验证输出成员关系。

规范化命名空间并去掉导出方法后，编码、主码 / 残差训练与构建源码同评测实现一致（`scripts/audit_ch4_review.py`）。该对应覆盖共同路径；后续模型身份与大数据集两遍构建仍需单独核查。

### 4.6.2 独立输入与放置审计

补证导出器在 Vogue、arXiv、OpenAI-3072 与 Stella 上使用共同构建路径。四者全行偏移均单调、端点为 0 和 $N$，ID 均构成数据库行的置换。对存储旋转矩阵做 FP64 乘法，$\|\Pi\Pi^{\mathsf{T}} - I\|$ 最大逐项误差为 3.96–4.34 $\times 10^{-7}$。采样范数检查使用 FP64 旋转而非 GPU GEMM，因此验证的是存储矩阵的几何性质。表 4.3汇总采样重编码。

**表 4.3.** 原始输入 FP64 重编码审计。差异数按坐标决策计数，最后一列为残差不一致坐标离决策边界的最大距离。结构检查覆盖全行，重编码采用采样。

| 数据集 | 重编码行数 | 正交误差 | 主码差异 | 残差差异 | 最大边界距离 |
|---|---|---|---|---|---|
| Vogue | 200,000 | 4.11e-07 | 14 | 5,409 | 1.12e-07 |
| arXiv | 100,000 | 4.11e-07 | 1 | 2,515 | 7.31e-08 |
| OpenAI-3072 | 20,000 | 3.96e-07 | 5,745 | 1,646,806 | 2.67e-05 |
| Stella | 100,000 | 4.34e-07 | 16 | 3,064 | 1.03e-07 |

768 / 1,024 维上的差异均在决策边界 $1.12 \times 10^{-7}$ 内。OpenAI-3072 则明显不同：残差不一致占所查坐标的 2.68%，最远离边界 $2.67 \times 10^{-5}$，与 TF32 旋转一致，因此不能将所有数据集称为 FP32 等价。残差比较以已存主码位为条件，避免重复计入主码翻转。

三个较小数据库在同一模型下强制一遍 / 两遍构建，按 ID 关联的主码和残差字节完全相同，correction 差至多 $8.94 \times 10^{-8}$；列表分配分别有 1、1、40 行不同。Stella 只验证自然两遍路径。导出器不含 IVF 质心，因此列表成员决策本身仍未独立验证，结构正确不等于已证明分配决策正确。

**表 4.4.** 纳入 pilot 及补证输入审计后的表示契约，区分存储几何、采样算术与行放置的覆盖。

| 边界 | 所需性质 | 证据 / 剩余检查 |
|---|---|---|
| C0：几何 | 共同变换，浮点等距误差有界 | 四数据集已查存储矩阵正交性；区分 TF32 GPU 算术 |
| C1：码本 / 修正 | 笛卡尔级值、有序残差码本，修正项匹配解码编码 | 三个数据库 pilot 已完成；全行修正误差见第4.6节 |
| C1：输入编码 | 存储编码对应预期原始输入行 | 四份原始输入采样已做 FP64 重编码；量化边界与 TF32 差异已量化 |
| C1：放置 | 偏移覆盖行，ID 为置换，字段遵循同一置换 | 全行偏移 / ID 核对及三份按 ID 关联的路径比较；质心成员决策未验证 |
| 执行 | 索引、同步和分数顺序契约有效 | 已有查询与 sanitizer 检查，有限覆盖见第7.4节 |

### 4.6.3 契约覆盖与剩余检查

构建要求 $M \mid d$、$D_s \mid B$、$M \bmod 4 = 0$，LUT 验证使用实际码字切片。输入须有限、计数可表示并预留哨兵位置。ID / 位置用 signed 32-bit；较大计数与地址乘积用更宽算术。

审计使用导出模型而非按 seed 重生成。T4 检查计算分数选择，与输入重编码分开(第7.4.1节)；质心成员关系、空列表与对抗尾批次仍待验证。

# 第 5 章面向表示的 GPU 搜索

变换与主码 / 残差层次继承自 JHQ[16]。本章发展 GPU 评分及计算分数选择；测量与预算选择见Chapters 6与7。

## 5.1 查询路径与四种不同的排序对象

对固定索引，查询先变换为 $q' = \Pi q$，再路由到选中的倒排列表，使用主码筛选，仅对保留候选做精排。最终选择器返回 $k$ 个数据库 ID 及计算所得分数。图 5.1区分运行时的三个选择决策；ground-truth 排序属于外部评价对象。

需要分清四种量：

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

其中 $\mu_i$ 是变换坐标中的 IVF 中心，$\widehat{p}(x)$ 为主码重建，$\widehat{r}(x)$ 为解码后的残差。运行时精排分数包含一个仅与查询有关的常数；第5.6节依据实际修正项推导这一点。对固定查询，它保持重建距离的排序，但精排分数和主码分数都不等于原始向量距离。

路由定义含 $n_c$ 个候选实例的已访问集合 $U$，主码筛选保留至多 $c = \max\{k, \lceil \alpha k \rceil\}$ 个候选进行残差评分。表 5.1列出各阶段的输入、读取对象与成本，sections 5.2 to 5.7展开实现。

表 5.1的顺序与精确性列将各声明对应到评分和并列规则。第 7 章分别检查保留集合、有序输出及原始空间召回率。

表 5.1给出阶段契约。E1 / 预算程序显式设置 16 个精排 lane 与扫描策略 $-1$（调度选择 0/2/4/8），库默认则是标量精排及策略 0。共享界开启，仅在 $g_{\mathrm{scan}} > 1$ 时使用；程序用 512 个数据库伪查询校准调度，这一步并非库自动执行。组件消融保留各自开关，表 A.1对应机制及归档实现。

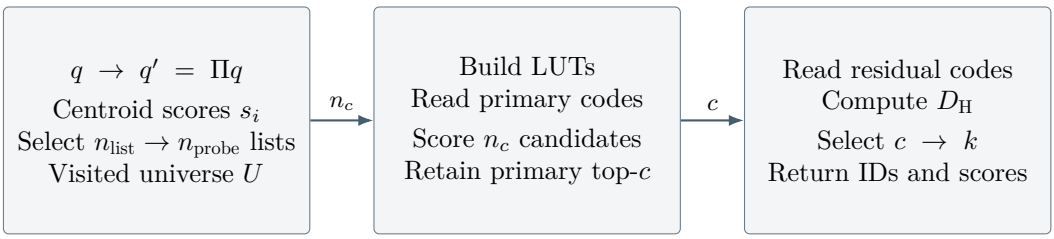



**图 5.1.** 查询执行及其独立选择边界。路由选择列表，主码筛选选择向量候选，最终选择对精排后的保留候选排序。原空间 ground truth 用于评价，不作为这些搜索决策的输入。

**表 5.1.** 逐阶段契约。$c_v = \min(c, n_c)$ 为有效主码保留数。“精确”针对约定的计算分数；路由与量化重建决定原空间近似性。

| 阶段 | 输入 → 输出 | 读取数据 | 主要工作 | 排序 / 精确性 |
|---|---|---|---|---|
| 变换 | $q \to q'$ | 查询；变换 | 应用 $\Pi$ | 变换（精度见第7.2.3节） |
| 探针 | $n_{\text{list}} \to n_{\text{probe}}$ 个列表 | FP32 中心；列表长度 | 评分；radix；名次计数 | $(s_i, i)$；精确探针选择 |
| LUT | $q' \to$ 表 | 主码本 | $MD_s 2^{8/g_{\text{LUT}}}$ 坐标项 | 实数分解精确；存在 FP32 舍入 |
| 主码，$g_{\text{scan}} = 0$ | $n_c \to c_v$ | 打包主码；LUT | $O(n_c M)$ 评分；压缩 | 仅按距离；分数选择精确，截断并列任取 |
| 主码，$g_{\text{scan}} \geq 1$ | $n_c \to c_v$ | 相同主码表示 | 评分；压缩；合并 | $(D_{\text{P}}, p)$；在明确前提下精确 |
| 精排 | $c_v \to c_v$ 个分数 | 残差码；修正项 | $O(c_v d)$ | 计算所得 $D_{\text{H}}$；FP32 结合顺序依 lane 而变 |
| 最终选择 | $c_v \to k$ | 精排分数；ID | $O(kc)$ 槽位扫描 | $(D_{\text{H}}, \text{ID})$；有 $k$ 个有限有效候选时精确 |

## 5.2 精确的 Cartesian LUT 分解

广泛扫描通过非对称距离计算评估主码重建：查询保持 FP32，存储编码用于选择重建坐标 [16, 17]。对变换后的查询 $q'$，主码子空间 $m$ 的普通查找表为

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

八比特码要求每个子空间存储 256 项。这里的优化机会来自码本内部的 Cartesian 结构，而不只是向量被划分成 PQ 子空间。

当 $K = 256$、$D_s = 8$ 时，解析码本构造为每维两个标量水平：

$$\begin{aligned} a &= \sigma \Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\text{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

其中 $\Phi$ 为标准正态分布 CDF，$s_j(x) \in \{-1, +1\}$。每字节编码八个坐标的符号，高位对应靠前的坐标。当 $a > 0$ 时，主码排序等价于查询与符号向量的内积排序。逐维二水平 Cartesian 积也解释了四组和八组分解在代数上仍然精确。这描述的是解析构造；若替换成非 Cartesian 的学习码本，需要重新检查兼容性。

### 5.2.1 在不改变量化器的情况下分解

对被测 $D_s = 8$ Cartesian 码本，高半字节 $h$ 选择前四个重建坐标，低半字节 $l$ 选择后四个。因此 $\mathcal{C}_m[16h + l] = (A_m[h], B_m[l])$，两个坐标组之间不存在交叉依赖。定义

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

平方 L2 对互不相交坐标可加，得到

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

只需存储 32 项，而不是 256 项。重建表示和实数运算下的分数保持不变，无需重训练编码、增加量化或改变候选集合。

**表 5.2.** 八比特、八坐标主码子空间的 Cartesian 表权衡。合并加法为组合查表结果所需的代数最小次数，不含子空间之间的累加，也不是实测指令数。

| $g_{\mathrm{LUT}}$ | 表项 | FP32 字节 | 查表次数 | 合并加法 | 建表坐标项 |
|---|---|---|---|---|---|
| 1 | 256 | 1,024 | 1 | 0 | 2,048 |
| 2 | 32 | 128 | 2 | 1 | 128 |
| 4 | 16 | 64 | 4 | 3 | 32 |
| 8 | 16 | 64 | 8 | 7 | 16 |

建表程序从现有码本读取代表码字 $16h$ 和 $l$，只计算相应坐标片段；另一片段的零编码位与此计算无关。这要求所有共享同一半字节的码字也共享对应坐标片段。`verify_split_lut` 将码字片段与代表码字比较，拒绝不兼容码本。任意学习得到的 256 中心 PQ 子码本不一定满足该条件；仅将编码标签拆成半字节，不会自动分解中心坐标。

第7.3节的同候选 FP32 审计测量各表形式的数值与排名差异，包括主码保留集合相同时仍出现的变化。

### 5.2.2 分组、建表工作量与资源成本

表分组消融将分解推广到 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ 个连续坐标组，每组位字段与 Cartesian 数字对应。当编码为八比特、$D_s = 8$ 时，每组有 $2^{8/g_{\mathrm{LUT}}}$ 项；总表项和每查询 FP32 存储为

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

独立构造表项时，每个子空间计算 $D_s 2^{8/g_{\mathrm{LUT}}}$ 个坐标贡献。相比独立计算完整表的每一项，两组设计将表存储减少八倍、坐标工作量减少十六倍。这是表级成本比，不是应用性能加速预测。

表在全局工作区构造；当完整共享内存占用不超过查询所得的每 block opt-in 上限 $S_{\max}$ 时，由 block 合作搬入共享内存。候选缓冲区容量为 $C_{\mathrm{buf}}$ 时，普通精确扫描与分组扫描分别需要

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

这些公式适用于被测扫描，不包含可选融合精排或提前终止的额外工作区。代码恰在对应占用不超过 $S_{\max}$ 时选择共享内存查表。

分组表实验中 $c = 1{,}000$, block 为 512 或 1,024 线程, $C_{\mathrm{buf}} = 2{,}048$。完整表为 96–384 KiB，两组表为 12–48 KiB。NVIDIA 对 compute capability 12.0 给出的每 block 上限为 99 KiB[24]。按该上限，即使 $M = 96$，完整表加查询级缓冲区也需要 114,696 字节，超过容量；最大的两组表加分组缓冲区仅需 65,560 字节。因此，按源码放置规则，这些配置的完整表走全局内存、两组表走共享内存。这是依据启动规则与公开上限作出的推导；本次检查的记录未保留设备属性打印。

四组和八组都只存每子空间 16 项，却增加查表和合并。位提取、查表依赖、搬运、同步和驻留共同决定实测权衡。不同扫描组可能各自搬运同一查询的表。

这里的 16 + 16 表就是八比特符号码的 nibble LUT。PQ Fast Scan、Quicker ADC 及 IVF-RaBitQ 中均有相关小表执行技术 [1, 2, 33]。这一代数联系并不意味着 JHQ 两级分数与 RaBitQ 距离估计器相同。本文贡献在于 JHQ 主码 / 残差结构上的 GPU 执行，包括扫描、

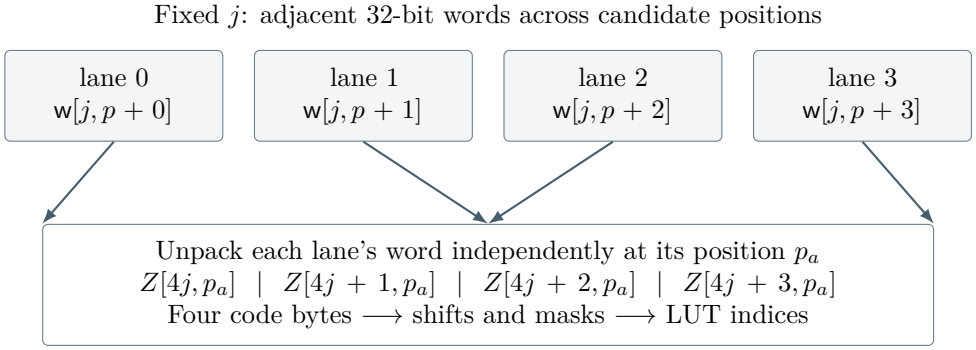



**图 5.2.** 打包后的候选并行访问，图中仅画出部分 warp lane。固定字组 $j$ 时，连续候选位置访问相邻四字节 word。每个 word 包含同一候选的四个子空间编码，解包保持编码值，随后再进行 LUT 拆分。此图说明地址关系，不表示实测内存事务数量。

选择与精排，以及实测支持的表分组选择。第7.3节中，两组在 24 个单元中领先 22 个，其余差距不足 1%。

## 5.3 打包编码与候选并行的内存访问

相邻扫描 lane 处理相邻候选，子空间优先存储使其编码访问相邻；LUT 分解则独立减少每查询的查找表。

### 5.3.1 地址映射与四字节打包

令 $o_\ell$ 为倒排列表 $\ell$ 的起点，$r$ 为列表内行号，$p = o_\ell + r$ 为物理位置。若 $Z[m, p]$ 表示主码子空间 $m$ 的八比特编码，逐字节扫描副本将其存于字节偏移 $mN + p$，而候选优先布局使用 $pM + m$。当各 lane 访问同一列表内连续位置时，前者使固定 $m$ 的访问相邻。跨列表边界时，位置可能跳跃，因此不保证每个 warp 都对应一次连续事务。

打包表示将同一候选的四个子空间编码组成一个无符号字：

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \mathrm{offset}_{\mathrm{bytes}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{5.8}$$

通过 $(\mathsf{w}[j, p] \gg (8t))\ \&\ 255$ 精确恢复四个编码。每个编码再提供式 (5.5) 中的高、低半字节索引。打包四个子空间，与拆分一个子空间的八比特标签不同，也不等于打包四个候选的编码。

建库路径明确将 $[M, N]$ 字节数组重新打包为 $[M/4, N]$ word 数组。仅把原字节数组重新解释为 `uint32_t`，会合并相邻候选的字节，而不是需要的子空间。实现要求 $M$ 能被四整除，并拒绝不完整的最后一个 word。重排在建库时读写一次 $NM$ 编码字节，旧存储和新存储短暂共存；它位于搜索计时边界之外。

### 5.3.2 打包访问省下什么、保持什么

一次 word load 取得四个编码，随后在寄存器中移位与掩码解包。32 个连续位置的逻辑窗口为 128 字节，实际事务取决于对齐、活跃 lane 及内存层次。载荷仍为每候选 $M$ 字节。两种被测布局已按子空间组织，因此开关对照测试的是打包 / 加载 / 解包的综合效果，没有单独分离转置、指令或 cache 的贡献。

第7.3节比较打包路径与逐字节路径的吞吐量、召回率及波动。

## 5.4 通过多 block 扫描实现精确全局主码 Top-c

查询级扫描仅为每条查询暴露一个 block。分组扫描将候选划分给多个 block 后合并保留结果，处理 GPU 选择问题 [18]。精确性针对已计算主码分数，路由与表示仍具有近似性。

### 5.4.1 候选划分与确定性顺序

对固定查询，按探针顺序拼接选中的倒排列表，得到 $n_c$ 个候选出现位置。假设每个被访问数据库行只出现一次，搜索期间索引保持不变。对 $g_{\mathrm{scan}} \in \{1, 2, 4, 8\}$，block $(q, j)$ 负责区间

$$U_j = \left[ \left\lfloor \frac{j n_c}{g_{\mathrm{scan}}} \right\rfloor, \left\lfloor \frac{(j+1) n_c}{g_{\mathrm{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\mathrm{scan}} - 1. \tag{5.9}$$

这里的区间表示相应候选，而不是它们的数据库 ID。设备端依据路由得到的候选数计算区间端点。各组候选数最多相差一个；一个较长的列表可以跨越组边界。这种划分平衡的是候选数量，而平均分配列表数量不一定能达到同样效果。每个线程通过探针前缀偏移找到自己的第一个列表，在扫描区间时向前移动列表游标。此阶段不读取残差码。

选择使用的 key 为

$$K_q(x) = \big(D_{\mathrm{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

其中 $p(x)$ 是固定倒排存储中的物理行位置。它在该查询的候选中唯一，不依赖原子追加的先后顺序，也不是最终结果处理并列时使用的数据库 ID。当分数不含 NaN 时，这一定义为不同候选建立全序。局部比较和合并决策都使用相同顺序。要使选择不受划分方式影响，还要求各条比较路径为同一候选计算出相同的分数；排序规则本身不能消除浮点评分差异。

### 5.4.2 精确局部保留与全局覆盖

用 $\mathrm{Top}_c(V)$ 表示将 $V$ 按 $K_q$ 排序后最前面的 $\min(c, |V|)$ 个候选，其中 $c = \max\{k, \lceil \alpha k \rceil\}$。每个独立组保留完整的局部 top-$c$，而不是分得一个 $c/g_{\mathrm{scan}}$ 配额。这一区别不可省略：当 $c = 4$、组数为二时，四个全局优胜候选可能全在同一组。如果每组只保留两个，即使总容量为四，仍会丢掉全局优胜候选。

**引理 5.1** (局部缓冲区保留)**.** *设 $V_t$ 为一个 block 截至 chunk $t$ 已评分的候选，$B_t$ 为该时刻完成压缩或过滤后的有效缓冲区。假设完整 key 固定、追加容量充足、压缩精确保留 top-c。令 $\beta_t$ 为截至该时刻读到的全部完整共享界的最小值，包含初始顶哨兵，则*

$$\mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

*关闭共享界时，界始终为哨兵，缓冲区包含整个已扫描前缀的 top-c。*

*证明.* 空前缀满足不变量。新候选若未通过局部准入阈值，则已有 $c$ 个保留 key 排在它之前；这些前驱在之后的前缀中仍存在。共享界拒绝的候选均在 $\beta_t$ 之后。旧候选若已不属于前缀 top-$c$，前缀增长也不会改善其名次。因此，每个符合界的新前缀优胜候选，要么已经保留，要么获准追加。精确压缩不会丢弃它：在整个前缀中排在它之前的成员少于 $c$，缓冲区内也如此。过滤仅删除读到的界之后的 key。对追加、压缩及过滤归纳即得结论。若读取可能返回不同的旧发布值，必须使用历史读取的最小值表述不变量。 □

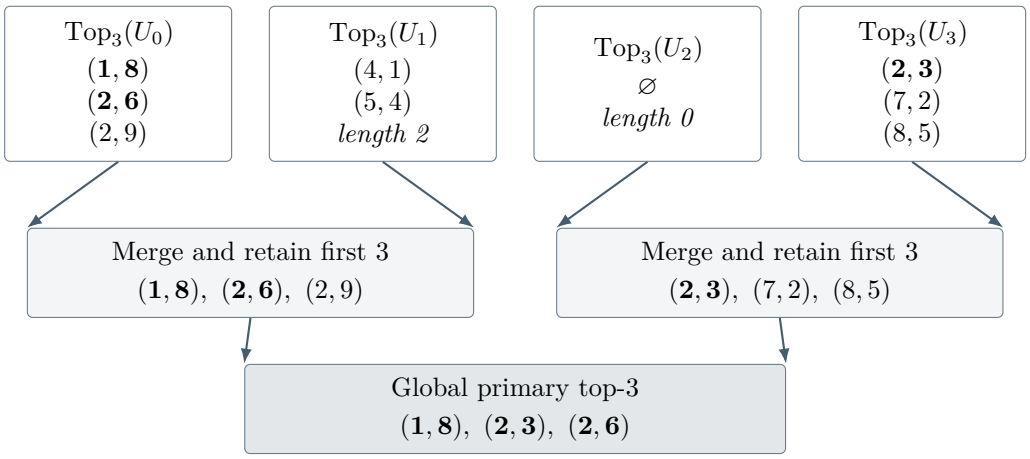



**图 5.3.** 局部覆盖与截断式两两合并，示例中 $c=3$、$g_{\mathrm{scan}}=4$。条目是 $(D_{\mathrm{P}},p)$ key，属于示意而非测量数据。粗体条目是最终全局优胜候选。短组仅贡献有效前缀，空组不贡献条目。每次合并使用同一 key 顺序，并保留至多 $c$ 项。

**命题 5.1** (覆盖与截断合并)**.** *在固定 key 和共同全序下，设 $U$ 为已访问候选全集，$V$ 为任一保留子集。若 $\mathrm{Top}_c(U)\subseteq V\subseteq U$，则 $\mathrm{Top}_c(V)=\mathrm{Top}_c(U)$。有序列表两两合并且每轮截取前 $c$ 项保持这一结论。特别地，对互不相交的分组，有*

$$\mathrm{Top}_c\left(\bigcup_j U_j\right)=\mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

*候选全集不足 $c$ 时，结论同样成立。*

*证明.* 每个全局优胜候选都在 $V$ 内，其余 $V$ 中的成员均排在全局优胜候选之后，因此从 $V$ 选择可恢复全局结果。对独立分组这一特例，设 $x$ 同时属于全局 top-$c$ 和组 $U_j$。全局排在 $x$ 之前的候选少于 $c$ 个，因此组内也少于 $c$ 个。于是 $x\in\mathrm{Top}_c(U_j)$，局部结果并集包含全部全局优胜候选。整个全集不足 $c$ 时，所有成员均保留。

对于合并，两个有序列表的前 $c$ 项就是二者并集的 top-$c$。被截去的候选在该并集中已经有至少 $c$ 个前驱，不可能再进入任何更大并集的 top-$c$。对合并树的各轮归纳即可得到结论。 □

CUDA 实现在共享内存中维护容量为 $C_{\mathrm{buf}}$ 的 key 对缓冲区。扫描 block 含 $T_{\mathrm{blk}}$ 个线程，$C_{\mathrm{buf}}$ 为二的幂且满足 $C_{\mathrm{buf}}\geq c+T_{\mathrm{blk}}$。压缩操作通过 bitonic 网络排序，保留至多 $c$ 个有效条目；当有效条目达到 $c$ 个时，将最后一个保留 key 记为局部阈值。在此之前，阈值是排在所有有效 key 之后的哨兵。两次压缩之间，仅追加排在阈值之前的候选。即使后续候选继续到来，阈值对应的 $c$ 个见证候选仍使它成为已扫描前缀中最优 $c$ 项的安全上界。

每个 chunk 最多追加 $T_{\mathrm{blk}}$ 项；可能越界前先压缩至至多 $c$ 项，$C_{\mathrm{buf}}\geq c+T_{\mathrm{blk}}$ 保证下一块容量。这满足引理 5.1的容量前提，无需逐线程配额。共享时，命题 5.2保证已读阈值不小于全局第 $c$ 个 key，再由缓冲区引理与覆盖命题保证过滤及合并保留全局优胜候选。

另一种方案是先将全部 $n_c$ 个分数 / 位置对写出，再调用独立选择器。这保持相同目标，但增加一个 $O(n_c)$ 中间数组及其写入。这里将打包主码的评分直接送入有界局部缓冲区，以重复局部排序换取 $O(g_{\mathrm{scan}}c)$ 规模的中间输出。基于寄存器的选择也是已有的 GPU 设计 [18]；本轮实验以归档参考路径评估选定的扫描变体。

### 5.4.3 合并实现、存储与边界情况

当 $g_{\text{scan}} > 1$ 时，每条查询由一个 256 线程的 block 执行 $\log_2 g_{\text{scan}}$ 轮两两合并。有效长度为 $a, b$ 的一对列表输出 $\min(c, a+b)$ 个 key。Merge Path 将输出划分为连续的线程片段：先二分查找每个片段对应的输入偏移，再由该线程顺序合并自己的片段 [14]。所有比较都遵循式 (5.10)，各轮之间通过 block 屏障同步。图 5.3 展示了截断不变量。除划分查找与同步工作外，合并树全部内部节点合计最多输出 $c(g_{\text{scan}} - 1)$ 个 key 对。

两个全局分数 / 位置工作区的容量分别为 $[b_{\text{cap}}, g_{\text{scan}}, c]$ 和 $[b_{\text{cap}}, g_{\text{scan}}/2, c]$，在各轮间交替使用。分数和位置各占四字节，因此二者的总载荷为

$$S_{\text{pairs}} = 8 b_{\text{cap}} c \left( g_{\text{scan}} + \frac{g_{\text{scan}}}{2} \right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

各组计数还需要 $4 b_{\text{cap}} g_{\text{scan}}$ 字节；配置共享界存储时另需 $8 b_{\text{cap}}$ 字节。这些空间位于最终保留数组及其他搜索工作区之外。分配容量在 graph capture 之前增长，之后保留复用，因此以后选择较小的 $g_{\text{scan}}$ 不一定释放预留显存。合并只在共享内存中保存列表长度，而不存放全部 $g_{\text{scan}} c$ 个 key。

局部有效前缀由计数界定，而不是仅依靠哨兵分数。压缩时，排序缓冲区填充 $(+\infty, \text{INT_MAX})$；有效位置必须小于这个保留位置。空组计数为零。最终保留数组将未使用槽位填为 $(+\infty, -1)$，残差精排跳过负位置。这两种哨兵用途不同：负位置填充项不能作为有效 key 参与合并。当 $g_{\text{scan}} = 1$ 时，直接写最终保留数组，无需合并。

较早的查询级路径由实现标签 $g_{\text{scan}} = 0$ 选择，表示每条查询一个 block，而不是零个 block。其默认主码选择按分数精确，但仅比较距离的截断规则可能在并列处保留与完整 key 路径不同的成员。有损的 `K_LOCAL=4` 替代路径需要在允许该路径的构建中通过 `JHQ_EXACT_TOPCK=0` 开启；paper-faithful 构建强制使用精确选择。它不是被测默认路径。对仅比较距离的路径，缓冲区论证使用全预序：最终压缩后，保留所有严格小于前缀第 $c$ 小距离的候选，并在该距离处保留足够成员以填满 $\min(c, |V_t|)$ 个槽位。被拒绝的距离已有 $c$ 个距离不更大的保留见证；精确距离排序在每轮压缩保持这一名次性质。并列成员身份不要求唯一。第7.4.1节 的严格 CPU 参考已检查相同计算分数 / 位置目标：55,296 个分组查询实例全部符合完整键顺序；13,824 个单 block 实例全部满足距离全预序，其中 56 个选择了不同截断并列成员。这以有限样例的实现证据补充有条件的缓冲区 / 合并论证。参考在 CPU 上排序 GPU 生成的分数，并未在 CPU 上重新计算原空间距离。

## 5.5 共享完整 Key 阈值与执行调度

保留完整局部 top-$c$ 建立了正确性基础，但为了得到 $c$ 个全局输出，可能生成 $g_{\text{scan}} c$ 个中间候选。共享界组件减少这部分多余的选择工作，调度器再决定暴露多少扫描并行度。二者均在给定精排预算下工作；选择预算是第6章讨论的独立问题。

### 5.5.1 带明确见证的完整 Key 上界

当 $|U| \geq c$ 时，用 $K_{(c)}$ 表示全局第 $c$ 小的 key。若某组持有 $c$ 个互不相同且已完整评分的候选，则这些见证中的最大 key $\tau_j$ 满足 $K_{(c)} \preceq \tau_j$：全局集合中已经存在 $c$ 个不晚于 $\tau_j$ 的候选。共享界初始化为最大哨兵，并按以下方式更新：

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

不足 $c$ 个有效候选时不得发布界。如果整个候选集合不足 $c$，任何组都无法发布这样的界，所有有效候选保持可接纳。

**命题 5.2** (由见证界安全排除候选)**.** *假设 key 固定，每次发布都有 $c$ 个互不相同的候选作为见证，每次读取返回初始哨兵或某个完整的已发布 key。仅在 $K_q(x) \succ \tau$ 时排除候选，不会删除任何全局 top-$c$ 成员，与发布时序无关。*

*证明.* 每个已发布界都不小于 $K_{(c)}$，这些界的最小值也不小于 $K_{(c)}$。全局优胜候选满足 $K_q(x) \preceq K_{(c)} \preceq \tau$，因此不会被排除。较旧的界仍是上界，只会允许更多工作。此前的安全排除也不破坏后续发布的有效性：任意保留的 $c$ 个互不相同候选仍提供 $c$ 个见证，无论它们是否是该组最好的 $c$ 项。 □

共享可以删除已获证的全局非优胜局部候选，保留并集仍满足命题 5.1。与共享界相等的候选仍可接纳，实际条件为

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

其中 $\lambda_j$ 是局部压缩阈值，$\tau_{\text{seen}}$ 是该组最近一次观察到的共享界。

### 5.5.2 发布、过滤与实现边界

对于由平方和生成的非负 FP32 分数，代码将距离位模式和非负位置打包为无符号 64-bit key：

$$\kappa(D, p) = 2^{32}\,\text{uintbits}_{32}(D) + p. \tag{5.16}$$

此时无符号整数比较与字典序一致。这依赖正零编码，并排除 NaN 和负分数；它不是通用的有符号浮点编码。正无穷排在有限分数之后，全一位模式的初始字又排在有效编码之后。无效位置不得用于发布界。

每个 block 的一个线程在每个 chunk 读取该查询的共享界。压缩后，如果有效条目足够，就通过 64-bit `atomicMin` 发布局部第 $c$ 小 key。扫描结束时，再进行一次发布，并通过二分查找只保留不晚于观察到的共享界的有序前缀。各 block 不进行跨 block 屏障同步，也不等待其他组。较晚的更新可能使另一组多输出一些条目，精确合并会安全删除它们。每次搜索前在同一 stream 中重置共享界，扫描 kernel 完成后才执行合并。

归档二进制提取出的 PTX 包含标量`ld.volatile.global.u64` 与 `atom.global.min.u64`。PTX 对作用域兼容、尺寸相同的强访问给出 single-copy atomicity[26]，这些对齐 64 位操作在 PTX 层提供完整 key 读取前提。用 CUDA 13.0 的 `ptxas` 按 `sm_120` 重编同份 PTX，得到单条 64 位强系统读取及 GPU 作用域 atomic-min 归约，归档见第A.4节。toolkit 反汇编支持这一契约，但未捕获驱动 JIT 实际 SASS；论证不单凭 CUDA C++ `volatile`[25]。

每个候选在过滤前完成全部主码评分（图 5.4）。共享界节省追加、排序、中间写出和合并工作，但增加轮询及原子更新，不提供部分距离提前终止。第7.4节的净权衡也包括轻微变慢。

### 5.5.3 在不改变预算的情况下选择并行度

对 batch 大小 $b$，分组扫描启动 $bg_{\text{scan}}$ 个 block，而不是 $b$ 个。这可以暴露原本不足的并行度，但也重复 LUT 搬运和局部选择，并增加合并步骤，因此固定组数的设计不可能在所有

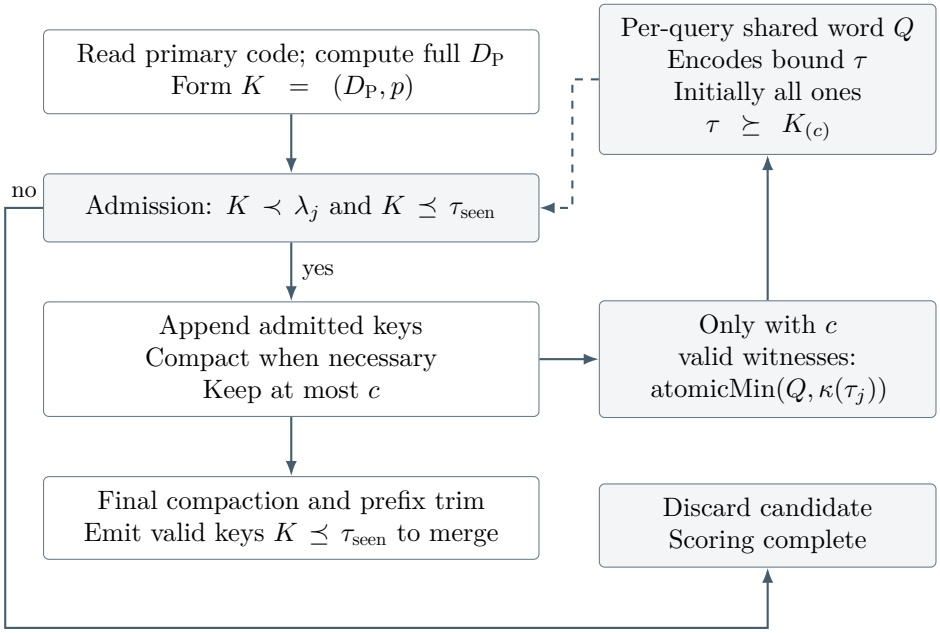



**图 5.4.** 单个扫描组的共享完整 key 过滤。候选的完整主码分数在接纳之前计算。有效的 $c$ 候选见证集合可以收紧查询级共享界；陈旧但完整的界只会削弱过滤。最终前缀裁剪减少合并的有效输入，但不改变已分配工作区的容量。完整 key 读取这一前提的实现边界在正文中另行说明。

情形下都更好。较小静态配额可以减少中间存储，但要保持覆盖，需要额外证书或回退；pilot 观察到的优胜候选集中现象促使系统选择带见证的阈值方案（第7.4节）。

实际调度器结合并行填充的近似指标和工作负载偏斜统计。令 $T_{\mathrm{blk}}$ 为扫描 block 的线程数，$H$ 为 SM 数乘以每个 SM 的最大驻留线程数，$v = \mathrm{sd}(n_c)/\,\mathrm{mean}(n_c)$ 为候选数的变异系数。规则为

$$g_{\mathrm{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\mathrm{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\mathrm{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\mathrm{chosen}} = \max(g_{\mathrm{fill}}, g_{\mathrm{skew}}). \tag{5.18}$$

偏斜规则可能覆盖填充限制。实验程序以 512 个数据库向量作为伪查询，在固定路由配置下校准 $v$，不使用 ground-truth 标签。这与第 6 章的采样输出一致性校准不同。校准之后，调度只依赖 batch 大小和已保存统计，不增加每次搜索的主机读回。两个常数在四个 pilot 配置上拟合，之后再到其他配置上评估。一次性伪查询校准不计入稳态搜索时间。补证分别计得主机收集 0.81–7.04 ms、首次校准调用 6.00–27.77 ms、重复中位 2.72–18.13 ms。此启动成本与精排预算校准分开核算（第A.4节）。

当 $g_{\mathrm{chosen}} = 1$ 时，实现选择更快的查询级路径，标签为 $g_{\mathrm{scan}} = 0$。它不改变索引、探针数或精排预算，但继承该路径仅按距离截断时的并列行为。因此，在声称所有调度选择产生相同保留集合之前，必须检查共同全序前提。第7.6节报告独立的集成输出检查，以及调度器的实测收益和 regret。

### 5.5.4 实际执行路径

对 170 个 SM 的 RTX 5090，公开的每 SM 48 warp 上限给出 $0.6H = 156{,}672$ 线程 [24]。batch 为 1,000、$T_{\mathrm{blk}} = 512$ 时，仅两组的填充代理量已达 1,024,000。因此填充规则选择一组，此时只有足够大的偏斜才强制四组。

主吞吐实验实际使用两种实现，并非全部执行分组扫描。E1 的 38 份运行记录中，九份 $v \geq 0.3$，均为 Vogue，范围约为 0.338–0.622。这些记录共有 132 个运行 / 预算单元：27 个 Vogue 单元调度为四组，105 个 arXiv / OpenAI 单元使用查询级路径。因此，非 Vogue 的大 batch QPS 并未执行分组保留、共享界过滤或合并，但仍包含 LUT、打包访问、lane 精排和探针组件。

Vogue 的调度路径与查询级路径比较中，每 1,000 条查询有 3–15 条仅主码并列成员不同，非并列差异为零，所有记录单元的最终输出均逐位一致。7–11 的较窄范围仅适用于 $n_{\mathrm{probe}} = 128$。

预算实验进一步明确了校准与评测的区别：12 个持出配置中，十个在 batch 500（Stella 为 499）使用查询级扫描，两个 Vogue 配置使用四组。校准样本量 $S = 128$ 时，八个配置使用两组，两个使用四组，两个仍为查询级路径，即 8/12 配置的校准与评测执行路径不同。分组选择主要服务较小 batch 和候选数偏斜的负载，并不存在普遍适用的 batch 256 分界。离线重计数与输入哈希记录在 `data/ch5/review-audit.json`。

该规则是启发式策略，不是 occupancy 计算，也不是延迟 oracle：它没有建模寄存器压力、共享内存驻留限制、与预算相关的压缩成本和合并成本。冻结的伪查询统计也可能无法代表未来查询。记录中的最大 regret 出现在 batch 256，此时某个固定组数方案约快 1.217×。该设计组合候选并行扫描、带见证过滤和工作负载相关执行，并在固定精排预算下测量收益和 regret。后续残差精排与最终 top-$k$ 按第5.6节消费这些有效候选。

## 5.6 Lane 协作的残差评分与最终 Top-k

残差访问仅限于主码保留候选，将坐标工作从 $O(n_c d)$ 缩减到 $O(cd)$；lane 协作在固定预算下加速这一阶段。

### 5.6.1 重建评分与查询相关常数

令 $\widehat{p}(x)$ 为主码重建，$\widehat{r}(x)$ 为变换坐标中的残差解码。编码器存储 $\gamma(x) = 2\langle \widehat{p}(x), \widehat{r}(x) \rangle$。实现中的精排排序分数为

$$D_{\mathrm{H}}(q, x) = D_{\mathrm{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

展开范数即得第二个等式。返回分数是重建平方距离加查询相关常数，保持排序但仍有重建误差 [16]。

残差坐标使用子空间内共享的标量码本项。标量路径遍历每个候选的各维，累加与 $q'$ 的平方差。查询坐标及在容量允许时的残差码本会搬入共享内存，最后再加上已存的修正项和主码分数。

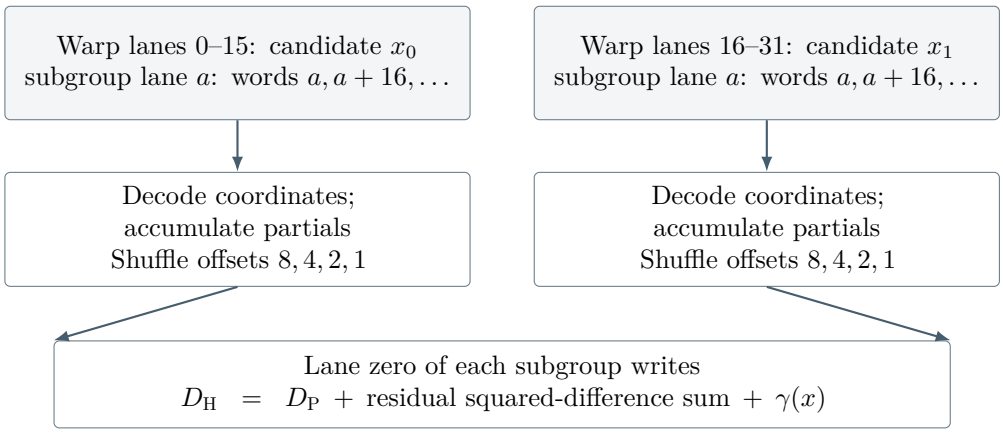



**图 5.5.** 十六 lane 残差精排。一个 warp 处理两个候选；候选内的 lane $a$ 读取残差 word $a, a+16, \ldots$。残差编码为八比特时，每个 32-bit word 解码四个坐标。子组归约合并部分和，再加上保留的主码分数和修正项。即使子组内 word 访问连续，候选物理位置仍可能分散。

### 5.6.2 Lane 到 word 的映射与部分和归约

对 $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$，独立的精排 block 为每个候选分配 $L$ 个相邻 lane。一个 32-bit 残差 word 包含 $d_{\mathrm{word}} = 32/B_R$ 个坐标编码，其中 $B_R$ 为残差位宽。被测八比特路径中 $d_{\mathrm{word}} = 4$，四比特路径中 $d_{\mathrm{word}} = 8$。lane $a$ 读取 word 索引 $a, a+L, a+2L, \ldots$，解码坐标并累加部分和。butterfly shuffle 归约合并 $L$ 个部分和，lane zero 写出精排分数。

启动条件要求残差行按 word 对齐，且 $D_s$ 能被 $d_{\mathrm{word}}$ 整除，从而一个 word 不跨越两个子空间的码本边界。所有 lane 都执行 shuffle 序列，包括分配到无效或超出范围槽位的 lane；它们在输出标为无穷前贡献零。这避免让 warp 参与情况取决于某个候选是否存在。$L = 16$ 时，一个 warp 处理两个候选，而不是每个 lane 各处理一个候选。

实验将两种改变分开。L0 是原来的标量逐字节加载路径，L1 使用 word load，但仍每候选一个 lane，保持其累加顺序。L8/L16/L32 进一步将坐标工作分配给多个 lane，缩短每个 lane 的串行遍历，并在 $c$ 较小时提供有用工作；代价是增加 shuffle，并减少同时处理的独立候选数量。因此 $L$ 并非越大越好。保留候选位置不规则、码本处于共享还是全局内存，也仍影响成本。

### 5.6.3 最终选择与数值边界

当至少有 $k$ 个有限分数的有效候选且 ID 唯一时，最终 kernel 按 $(D_{\mathrm{H}}, \text{database ID})$ 精确选择 top-$k$。每查询一个 block，读入 $c$ 个分数后反复归约最小值、输出 ID 与分数并移除该分数。每轮扫描全部 $c$ 个槽位，没有逐线程配额。退化池中，已选分数变为无穷而位置仍有效：有限条目耗尽后，最小有效 ID 可能以无穷重复输出。全无效池返回无效 ID 和无穷。本文评测满足有限条目契约。

最终 ID 并列规则与主码保留使用的物理位置规则不同。精确的最终选择器不能找回路由或主码筛选丢掉的候选，也不能消除量化误差。多 lane 归约在选择前改变浮点结合顺序，因此即使实数公式不变，近似并列候选也可能发生排序变化。

第7.3节分别报告仅加载与协作对照、数值差异、内核时间及端到端收益。

## 5.7 作为独立原语的 IVF 探针选择

探针选择发生在主码扫描之前，选择的是倒排列表，而不是数据库向量保留候选。对 IVF 中心 $\mu_i$，选择器对以下计算所得 FP32 路由分数排序：

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

省略共同的查询范数不改变中心排序。与主码平方距离表不同，这些分数可以为负。所需输出是按分数排序的前 $n_{\text{sel}} = \min(n_{\text{probe}}, n_{\text{list}})$ 个中心 ID，并列时较小列表 ID 优先，以及它们列表长度的前缀偏移。

点积继承 cuBLAS 算术模式：被测 v69 的 OpenAI 旋转与质心 GEMM 使用 TF32 输入和 FP32 累积。选择器的精确性针对这些已计算分数；之后的 FP32 `fma` 与确定 key 顺序不会恢复 GEMM 已引入的舍入（第7.2.3节）。

### 5.7.1 Radix 选择与确定性排序

较早的 bitonic 选择器对逐渐填充的候选缓冲区反复执行 bitonic 排序。radix 替代方案仍为每条查询使用一个 block，但通过四轮从最高有效字节开始的 radix pass 确定截断值。每轮只统计匹配此前已选前缀的 key，再找到包含剩余顺序统计量的 bin。四轮之后，得到 cutoff key 及仍需选择的同 cutoff 条目数。

浮点映射先将正负零均规范化为正零，再对 32-bit 表示 $u$ 作变换：

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

对非 NaN 浮点数，它保持数值顺序，包括负分数和无穷；规范化使相等的零具有同一 key。这与式 (5.16) 的非负分数编码不同。NaN 不属于选择器的排序契约。每轮使用相同、明确舍入的 `fma` 表达式重新计算分数，因此轮次改变不会改变 key。

直方图有 256 个 bin 和 32 份 lane 副本，仅计数器就占 32 KiB。副本将同一 warp 内对热门 bin 的原子更新分开，不同 warp 之间仍可能竞争。这是以共享存储和归约工作换取更分散的直方图更新，不是消除全部竞争。

收集阶段先取得所有小于 cutoff 的 key。如果等于 cutoff 的 key 过多，就按 ID 顺序重新扫描，通过 ballot 和前缀计数选取恰好所需数量、且列表 ID 最小的成员。因此最终并列子集不由原子到达顺序决定。随后，对 $n_{\text{sel}}$ 个已选条目分别统计有多少其他已选 key 在 $(s_i, i)$ 顺序中排在它之前，再写入相应名次。这无需排序全部中心，就恢复了确定性的有序输出。

最后，block 前缀扫描生成所选列表长度的偏移和总候选数。当 $n_{\text{probe}} > n_{\text{list}}$ 时，实现通过重复最后一个已选 ID 填充 ID 数组，但将多余条目的长度置为零，不会再次扫描同一列表。这里假设中心集合非空，且请求探针数为正。

### 5.7.2 成本与独立正确性契约

直方图和收集各轮会重复访问中心分数行；名次统计需要 $O(n_{\text{sel}}^2)$ 次比较。因此该算法不是对任意探针数都具有统一线性成本的替代品。较大的 $n_{\text{sel}}$ 同时增加排序工作与已选条目的共享存储，而中心集合较小时也仍需预留直方图空间。Radix selection 是已有选择技术；这里的贡献是它在该查询流水线中的确定性集成及实测执行权衡。

原语测试覆盖大量并列、全相等、混合正负零、大部分为无穷、极大量级，以及请求探针数超过列表数；结果与真实查询检查见第7.4节。

可选 FP16 路由改变评分精度，作为独立近似在第7.4节评价，并与跨进程流水线一致性检查分别报告。

# 第 6 章工作负载级预算校准

策略通过无标签输出与大预算参考的一致性选择工作负载级精排倍率，并跨批复用。质量与校准成本约束这一复用。

## 6.1 固定精排预算为何可能浪费

JHQ 先用主码筛选路由候选，再为较小的保留集合读取残差 [16]。对查询 $q$，令 $U(q)$ 为选定 IVF 列表中的候选。精排乘数决定

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha), |U(q)|\}} U(q). \tag{6.1}$$

这里 $C_\alpha$ 指有效保留候选。实现预留 $c(\alpha)$ 个槽位，候选不足时可用哨兵填充。分析假定至少有 $k$ 个有效候选。增大 $\alpha$ 扩展已访问列表内的精排容量；增大 $n_{\mathrm{probe}}$ 改变访问的候选全集。两者均不能自动恢复另一阶段丢失的信息。

较大的固定 $\alpha$ 可能反复精排不改变最终 top-$k$ 的候选；较小值则可能在残差评分前丢弃有用近邻。取舍依赖查询群体、主码重建、列表覆盖及精排分数顺序，因此一个全局默认值未必适合所有数据集—探针数组合。评测网格在 $k = 10$ 时覆盖 $c = 20$ 至 2,000。切换预算复用同一训练索引和常驻残差。降低乘数减少访问的残差行，不释放常驻残差表示。候选缓冲区在容量不足时扩展，之后保留该容量，因此较小预算不代表分配显存按比例下降。

解释性的延迟分解为

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

固定批次和路由时，$T_{\mathrm{base}}$ 包含传输、旋转、路由与主码评分；选择、残差评分和最终选择依赖候选容量（图 6.1）。缓冲容量、occupancy 和资源阈值可阶梯变化，因此该工作量模型不预测单调运行时间。小预算平台的测量见第7.5.5节；策略最小化可接纳预算，而不是带噪声的计时名次。

每个校准试验仍先执行旋转、探针选择和主码评分，再精排。复用索引避免重建或重训练，但不跨预算试验缓存全部查询中间结果。因此，校准成本要计算完整搜索，不能只计两个残差评分工作量之差。

策略为索引、路由配置、$k$ 和查询工作负载选择一个预算，既不训练逐查询预测器，也不为每条到达查询在线决策。扫描分组仍是第5.5节的独立机制；解释预算校准时仍须考虑它实际选择的执行路径。

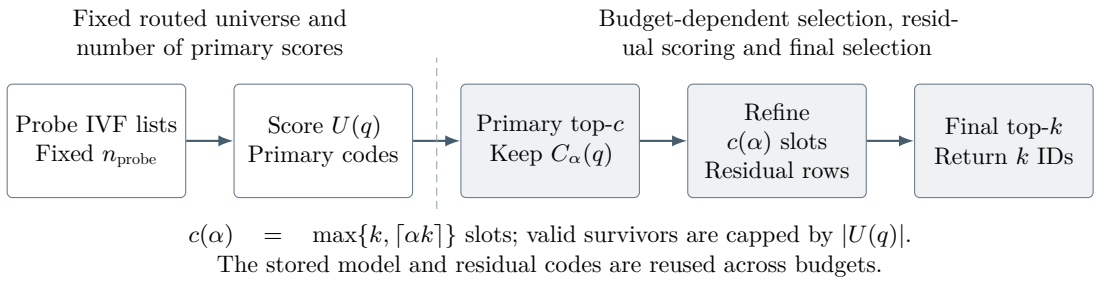



**图 6.1.** 精排预算改变主码评分之后的工作。固定查询、索引与探针数时，路由候选全集不变。着色阶段依赖保留容量；这是逻辑工作量分解，不是实测阶段耗时。扫描内核还执行随预算变化的接纳和压缩。候选少于 $c$ 时需要填充。

**在自适应 ANN 方法中的定位。** 该乘数与 FAISS 的 `IndexRefine::k_factor`、cuVS 精排比例承担相同的候选池规模角色 [7, 10]。FAISS auto-tuning 用 GT 评价准则探索速度—精度工作点 [9]。Li 等从搜索特征学习逐查询终止条件 [19]；DARTH 在执行期间预测 recall，在请求目标处早停 [6]。本方法通过参考输出覆盖率，为无标签工作负载校准一个可复用预算，未训练 recall 预测器。最终评分仍基于压缩重建，而 cuVS 的精排示例重排原始来源向量。这些区别分别定位工作量参数、其选择机制与评分表示。

## 6.2 无需 GT 的采样输出一致性

令 $\mathcal{R}_\alpha(q)$ 为固定索引在预算 $\alpha$ 下返回的 $k$ 个有效且唯一数据库 ID 的集合，定义参考 $\mathcal{R}_{\text{ref}}(q) = \mathcal{R}_{\alpha_{\text{ref}}}(q)$。降序网格为

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\text{ref}} = A[0] = 200. \tag{6.3}$$

对 $S$ 个校准查询 $\mathcal{Q}_S$，定义

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\text{ref}}(q)|\right), \qquad D_S(\alpha) \le \epsilon. \tag{6.4}$$

评测中的 $\epsilon = 1$ 允许整个样本共缺失一个参考 ID。它忽略结果顺序和距离差异；替换一个 ID 计一次，尽管对称差包含两个 ID。有限网格目标为

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \le \epsilon\}. \tag{6.5}$$

参考预算提供非空回退。目标最小化被接纳的乘数；第6.3节说明二分精确求解的条件。

在 $k = 10$ 时，归一化容忍度 $\epsilon/(Sk)$ 对 $S = 32, 64, 128$ 分别为 0.003125、0.0015625、0.00078125，$S = 500$ 时为 0.0002。因此，增大 $S$ 同时改变覆盖和有效严格度。校准与留出评测使用互斥查询；子样本复用同一有限校准池（第7.5节）。

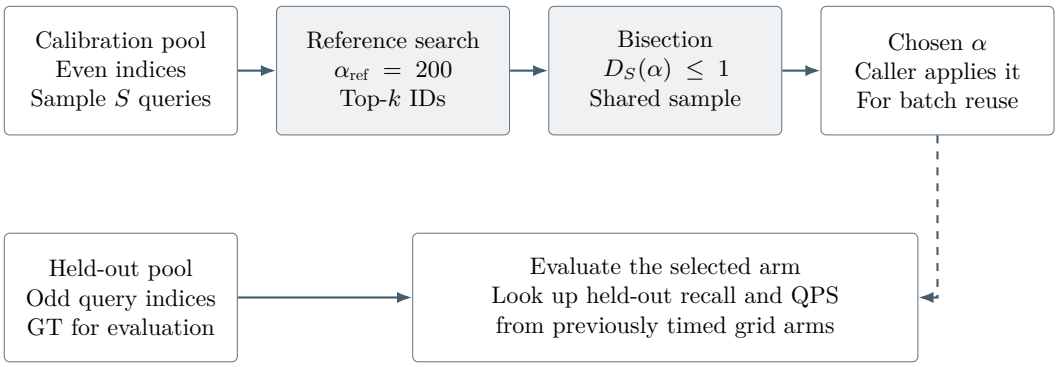



**图 6.2.** 工作负载级校准及其评测。预算决策使用参考 ID，GT 评价留出质量；所选预算结果从已计时的固定预算中查取。实现边界见第A.3节。

### 6.2.1 一致性控制什么

在命题 6.1的条件下，不一致数就是主码筛选排除的参考答案数：

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

对有效定长输出集合，令 $r_{\mathcal{Q}_S}$ 为同一批查询的 GT recall。每个缺失参考 ID 至多损失一个 GT 命中，因此

$$r_{\mathcal{Q}_S}(\alpha) \geq r_{\mathcal{Q}_S}(\mathrm{ref}) - \frac{D_S(\alpha)}{Sk}. \tag{6.7}$$

该界仅在样本内、相对参考成立。一致性控制相对参考的额外损失；参考自身质量仍需 GT 评价。第7.8.2节用重建 pilot 量化这一区别。

### 6.2.2 抽样分辨率与有序检验

令 $H_S$ 为至少缺失一个参考 ID 的查询数。由于 $H_S \leq D_S$，接受意味着 $H_S \leq 1$。对预先指定预算及 i.i.d. 查询，观察到一次事件时的单侧 95% 精确二项上限满足 [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

按查询计，$S = 128$ 和 500 的上限分别为 0.03652、0.009452。将全部 $Sk$ 个结果位看作同参数独立 Bernoulli 是更强假设；相应上限为 0.003701、0.0009484，近似 $4.74/(Sk)$。仅有查询 i.i.d. 不能将查询事件上限再除以 $k$；两种模型均未为固定池的重复子样本建立置信保证。

当不一致数单调且网格顺序固定时，二分选择被接受的前缀。若逐预算决策是有效水平检验，首个不安全预算的论证可给出无需 Bonferroni 因子的 fixed-sequence 错误控制 [3]。当前一个槽位规则是稳定性阈值；GT 风险保证还需要对目标有效的检验。

**有限池扩展。** 假设每条查询在目标样本大小的执行制度下具有固定不一致数 $X_q(\alpha)$，且不随子集成员或位置变化。令池大小为 $N_{\mathrm{cal}}$，$n_j$ 为 $X_q = j$ 的查询数。均匀无放回抽样下，$D_S \leq 1$ 只可能抽到全零查询，或恰好一个计数为一的查询 [23]：

$$\Pr\{D_S(\alpha) \leq 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\mathrm{cal}}}{S}}. \tag{6.9}$$

越界组合数按零计。该有条件接纳模型还需网格接纳结构和选中预算质量映射，才能预测选择风险。接纳不使用标签，GT 损失越阈使用固定预算的留出质量。第7.5.5节的子集和检查在归档抽取上检验计数不变性假设。

## 6.3 预算搜索、嵌套性与并列安全

### 6.3.1 二分与完整枚举

记 $J = |A|$。实现先计算一次样本参考，再在降序网格中寻找最大的可接纳下标：

1. 置 $l = 0$、$h = J - 1$。下标零为参考回退。
2. 当 $l < h$，测试 $j = \lfloor (l + h + 1)/2 \rfloor$，运行 $A[j]$ 并计算 $D_S(A[j])$。
3. 若 $D_S(A[j]) \leq \epsilon$，置 $l = j$；否则置 $h = j - 1$。
4. 返回 $A[l]$；调用方在处理复用批次前应用该值。

八点网格除参考外至多需要三次完整候选预算搜索。调用语义、主机比较复杂度和计时边界见第A.3节。

完整枚举重新计算参考，在完全相同样本上测试其他七个预算，选择最小被接纳乘数。两种计时都含新参考，统一成本边界。即使接纳集合存在空洞，枚举仍评价整个有限网格；二分要保证找到相同最小值，则需要接纳下标构成前缀。

| Case | $C_1$ | $C_2$ | $C_3$ | Reference overlap |
|---|---|---|---|---|
| Nested | $\{b\}$ | $\{b, c\}$ | $\{a, b, c\}$ | 0, 0, 1 |
| Non-nested | $\{a\}$ | $\{b, c\}$ | $\{a, b, c\}$ | 1, 0, 1 |

**图 6.3.** 构造的 $k = 1$ 例子，精排顺序 $a \prec b \prec c$，参考为 $\{a\}$。嵌套情形保持参考成员；主码分数全并列时，不同并列选择可能产生非嵌套的中间候选池，破坏单调性。这些是解释用集合，不是 GPU 实测输出。

### 6.3.2 单调性的充分条件

**命题 6.1** (嵌套候选保留参考成员)**.** *对单条查询，假定路由固定，且 $\alpha \le \beta \le \alpha_{\text{ref}}$ 时 $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\text{ref}}}$。假定至少有 $k$ 个有效候选、精排分数固定、最终并列使用同一个确定全序。每个结果均为该顺序下精确 top-$k$，则*

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\text{ref}}| = |C_\alpha \cap \mathcal{R}_{\text{ref}}|, \qquad D_S(\alpha) \ge D_S(\beta). \tag{6.10}$$

*后一结论要求条件对每个采样查询均成立。*

*证明.* 参考成员在最大保留集合中至多有 $k-1$ 个前驱。若它存在于较小集合，限制集合不会引入新前驱，因此仍处于该集合精确 top-$k$。与参考的交集，恰等于保留集合中存在的参考成员。嵌套只能增加这些成员；对查询求和缺失数即得不等式。 □

在同一固定完整 key 下进行精确主码 top-$c$，可在 $c$ 增大时提供嵌套前提。再固定精排完整 key，接纳便单调，下标形成前缀，二分找到网格中最小可接纳预算。GT recall 和执行速度均不在定理中：近似精排分数可能随候选池扩大而提升错误近邻的名次，运行时间也依赖硬件。

### 6.3.3 评测分组路径的推论

运行头源码默认精确主码选择。同一次校准抽取内，$S$、路由和扫描线程数固定，`scheduled_groups(S)` 与 $c$ 无关。改变预算影响缓冲容量和接纳工作，分组评分器仍使用相同 FP32 LUT 项与累加顺序。精排固定 $B_r$ 和 16 lanes，保持每候选的残差归约；最终选择使用 $(D_{\text{H}}, \text{ID})$。

**推论 6.2** (分组校准)**.** *对评测中的分组配置，在第四、五章的表示与共享界契约下，各预算的主码保留集合嵌套。因此命题 6.1适用，二分与枚举返回同一最小被接纳网格预算。*

*证明.* 引理 5.1与命题 5.1与5.2给出共同完整键 $(D_{\text{P}}, \text{position})$ 下的精确主码 top-$c$，其前缀随 $c$ 增大而嵌套。固定的逐候选精排与精确最终全序提供命题 6.1的其余前提。 □

推论覆盖 $S = 32, 64, 128$ 的 2,304 次抽取中的 2,176 次；其余 128 次是 OpenAI-3072 在 $S = 128$ 的距离单 block 路径。补证逐次记录完整网格计数，2,304 条曲线全部单调，决策均与枚举一致（第7.5.5节）。

**全池覆盖范围。** 十二次 $S = 500$ 调用另计：两个 Vogue 单元分组，十个使用 $g_{\text{scan}} = 0$。补证新增全池枚举，十二条曲线均单调，所选预算均等于枚举得到的最小接纳预算，因此关闭了有限网格的实测缺口。十个距离路径仍不在 推论 6.2中；要给一般嵌套保证，仍可要求共同截断顺序，或所有网格截断（含参考）均无跨界并列。匹配留出路径有助迁移，但不证明一般条件。

**表 6.1.** 预算对照及其回答的问题。所有结果均用留出 GT 评价，但只有诊断性 oracle 用它选择预算。一致性策略的参考固定为 200。

| 对照 | 选择信息 | 回答的问题 |
|---|---|---|
| 固定 100 | 无校准 | 相对沿用默认值收益多少? |
| 固定 64 及更小网格预算 | 无校准 | 简单固定选择能否消除校准需要? |
| 采样二分 | $S$ 条无标签查询、槽位一致性 | 低成本样本能否选好可复用预算? |
| 同样本枚举 | 相同 $S$、参考和阈值，遍历网格 | 二分是否错过最小被接纳网格预算? |
| 全校准池 | 全部 500 校准查询，同一二分规则 | 更广覆盖、更严归一化容忍度会如何? |
| 留出 oracle | 全网格 GT recall 和实测 QPS | 哪个测量预算在诊断质量约束下最快? |

**基于排名的扩展。** 在同样的嵌套与固定分数条件下，令 $\mathrm{rank}_{\mathrm{P}}(x)$ 为参考答案在共同主码 key 下从 1 开始的排名，则

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\mathrm{ref}}(q)} \mathbf{1}\{\mathrm{rank}_{\mathrm{P}}(x) > c(\alpha)\}. \tag{6.11}$$

导出这些排名，可从一次参考搜索恢复整条网格曲线。当前 harness 仍逐预算搜索；排名提取与传输是同一排序条件下的后续工作。

## 6.4 预算基线及其回答的问题

表 6.1区分预算选择与评价。固定预算共享索引、路由和留出查询；调用与计时细节见第A.3节。

令 $\widehat{r}(\alpha)$、$\widehat{Q}(\alpha)$ 为实测留出 recall 和 QPS，$\widehat{r}^{*} = \max_{\alpha \in A} \widehat{r}(\alpha)$。Oracle 为

$$\alpha_{\mathrm{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^{*} - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

recall 容差 $\delta$ 不同于槽位容忍度 $\epsilon$。Oracle 消耗留出标签与计时，是有限网格诊断，不是可部署的无标签决策规则。

全池校准仍无标签，使用二分，不是按 GT 约束遍历全部预算；较大样本还收紧 $\epsilon/(Sk)$。同样本枚举则固定样本覆盖和阈值，隔离搜索过程。Oracle 在质量约束下最大化速度，而一致性规则最小化被接纳乘数。因此，即使二分精确、校准池全部覆盖，也不一定选中 oracle。

固定 64 检查较小的统一默认值是否已能获得收益；固定 200 提供高预算对照。观测质量和所选预算结果见第7.5节与表 7.8。

## 6.5 留出风险、校准成本与回本

### 6.5.1 选择校准规模

全池工作点得到表 7.7中留出结果的支持；采样变体见图 7.6。第6.2节的独立结果位模型提供规划尺度：$k = 10$ 时，$4.74/(Sk) \approx 10^{-3}$ 提示 $S$ 约为 500，而 128 的分辨率较粗。这一强模型针对参考不一致，不是 GT 损失，也不能将经验建议改写为预先指定的质量保证。

FULL 的路径与共同排序条件见第6.3节。

### 6.5.2 风险与所选预算性能

共做 $m$ 次校准抽样，第 $r$ 次选择 $\widehat{\alpha}_r$，定义留出损失和越阈频率：

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m}\sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

平均损失、越阈频率和尾部大小回答不同问题。表 7.8与图 7.6报告归档摘要；补证中最大的逐抽样损失见第7.5.5节。

所选预算速率为 $\overline{Q} = m^{-1}\sum_r \widehat{Q}(\widehat{\alpha}_r)$，由已计时的固定预算查取。它既不同于中位预算的速率，也不同于校准后查询流的吞吐。质量与速度须一起解读（表 7.9）。

### 6.5.3 复用成本

对 $B_{\text{reuse}}$ 个等大后续批次，模型为

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}} T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}} T_f. \tag{6.14}$$

当 $T_{\text{chosen}} < T_f$，回本批数为

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

严格超过 $B^*_{\text{reuse}}$ 才有正节省；若选中批次不更快，则没有吞吐回本。对 $n$ 查询批，由 QPS 换算毫秒为 $T = 1000n/Q$。选择变化时，$1/\mathbb{E}[Q] \le \mathbb{E}[1/Q]$，因此使用 $1/\overline{Q}$ 偏乐观。实测调用边界见第A.3节；四个单元已用直接校准加查询流总成本验证这一模型，并同时报告质量(第7.5.6节)。表 7.9给出当前实验估计。

**一个复用成本算例。** OpenAI-1536 在 128 探针时，全池校准选择 $\alpha = 8$，请求 80 个保留候选，固定 100 则为 1,000。两者 recall 均为 0.9280，速率分别为 267,690 和 167,451 QPS。500 查询时，由此得到 $T_{\text{chosen}} = 1.868\,\text{ms}$、$T_f = 2.986\,\text{ms}$；校准为 9.5 ms，故 $B^*_{\text{reuse}} = 8.50$，随后九批有正模型节省。这些是归档预算的算术计算，不是直接计时的查询流成本。

若每 $H$ 批重新校准，会增加每批 $T_{\text{cal}}/H$；漂移检测和自动重校准仍是后续工作。

## 6.6 策略失效模式与适用范围

复用要求索引、$k$、路由与查询群体稳定；批次相关调度可能改变并列及算术。有效且唯一的 ID、共同排序或配对枚举支撑决策（第6.3节），留出质量、尾损失和摊销成本决定实用性（第6.5节）。程序不会检测查询漂移。

已观察的工作负载族支持 FULL 的质量覆盖，以及 $S = 128$ 的较低校准成本。调用方须显式设置 $\alpha$。第七章评估固定预算 / oracle 对照及四个直接复用测量。

补证轨迹内容及已完成的严格 CPU 参考见sections A.4与A.5。剩余检查为任意子集逐查询不变性、重新归档候选 dump 以供独立重排、Vogue 跨进程差异及未来查询变化。状态维护在`assessment/ch7-pending-experiments.txt`。

预算主实验及轨迹重跑均使用 v69 算术，OpenAI 包含 TF32。独立 FP32 敏感性测量搜索前沿，没有在 FP32 下重新校准或验证这些预算选择。

# 第 7 章实验评估

实验考察构建与容量（RQ1）、执行（RQ2）、预算质量及复用成本（RQ3），以及搜索竞争区间（RQ4）。

## 7.1 实验协议

### 7.1.1 比较协议与可复现性

数据、计时与比较分组沿用sections 3.2、3.3与3.5。RTX 5090 主比较测量主机到主机稳态吞吐，构建与校准分别计时，各消融保留自身分组基线。

归档 CPU JHQ 结果不提供协议匹配的 CPU/GPU 加速倍数：十月局部性实验在 Vogue 使用 1,024 个列表与 $[8, 4]$ 级比特，而 E1 为 4,096 个列表及不同残差预算。较早 CSV 使用本地重实现，并有已记录的历史 recall 协议问题。

自适应调用次数与辅助设备端计时见第A.2节。

指标与前沿估计沿用sections 3.2与3.4；$g_{\mathrm{LUT}}$ 与 $g_{\mathrm{scan}}$ 保留第5.1节中的不同含义。

图表沿用第3.5节的重新生成清单。

## 7.2 现代 GPU ANN 的召回率—QPS 前沿

### 7.2.1 比较范围与前沿构造

图 7.1将 JHQ-GPU 与第3.4节中的图、压缩 IVF 和暴力搜索基线比较 [17, 28, 33]，使用表 3.2的前四个数据集。各方法共享 GT，每次处理单批 1,000 条查询。T5 在第7.2.5节另测小批次，第7.6节报告内部消融。

归档调度在全部 105 个 arXiv/OpenAI 运行—预算单元使用 $g_{\mathrm{scan}} = 0$；Vogue 的 27 个单元全部使用 $g_{\mathrm{scan}} = 4$，由 0.338–0.622 的候选数 CV 触发。因此，非 Vogue 前沿使用 query-centric 扫描，Vogue 则还使用分组保留、共享界与合并（第5.5节）。

按第3.4节的前沿规则，每个数据集上 JHQ 提供 27–39 个点，各基线组提供 54–264 个点。E1 共 1,590 个有效点，没有失败记录、无效 ID 或行内重复标记。这些是参数扫描点，而非独立索引构建。

调参维度见表 3.3，完整覆盖见第B.1节。曲线在目标之前结束表示该有限网格尚未覆盖该区间。

这些曲线并非同显存比较。九比特 RaBitQ 的名义编码预算 $9d/8$ 与 JHQ 接近，其 4/8-bit 变体则更小。表 4.1与第7.7节给出该性能包络对应的表示成本与实测分配。

### 7.2.2 匹配召回率与不同运行区间的反向结果

表 7.1按第3.4节定义的插值与实测达标估计，报告召回率 0.97 处的比较。

相较于 CAGRA fp32，该目标下的插值比值从 arXiv 的 1.317× 到两个 OpenAI 数据集的 1.747× 不等。相较于 IVF-RaBitQ，对应数值分别为 1.059×、0.996×、1.130× 和 1.157×。arXiv 接近平手，方向取决于估计方法。

E1 同批单线程块测量把 RQ2 与这项 RQ4 结果连接起来。以 $g_{\mathrm{scan}} = 0$ 重建 Vogue 前沿，0.97 处相对 RaBitQ 的比值从 1.059 降到 0.933；0.96 处对应比值为 1.250 与 1.072。两者使用相同运行—预算网格、batch 1,000 及 log-QPS 插值规则。因此，这份网格中的有利方向依

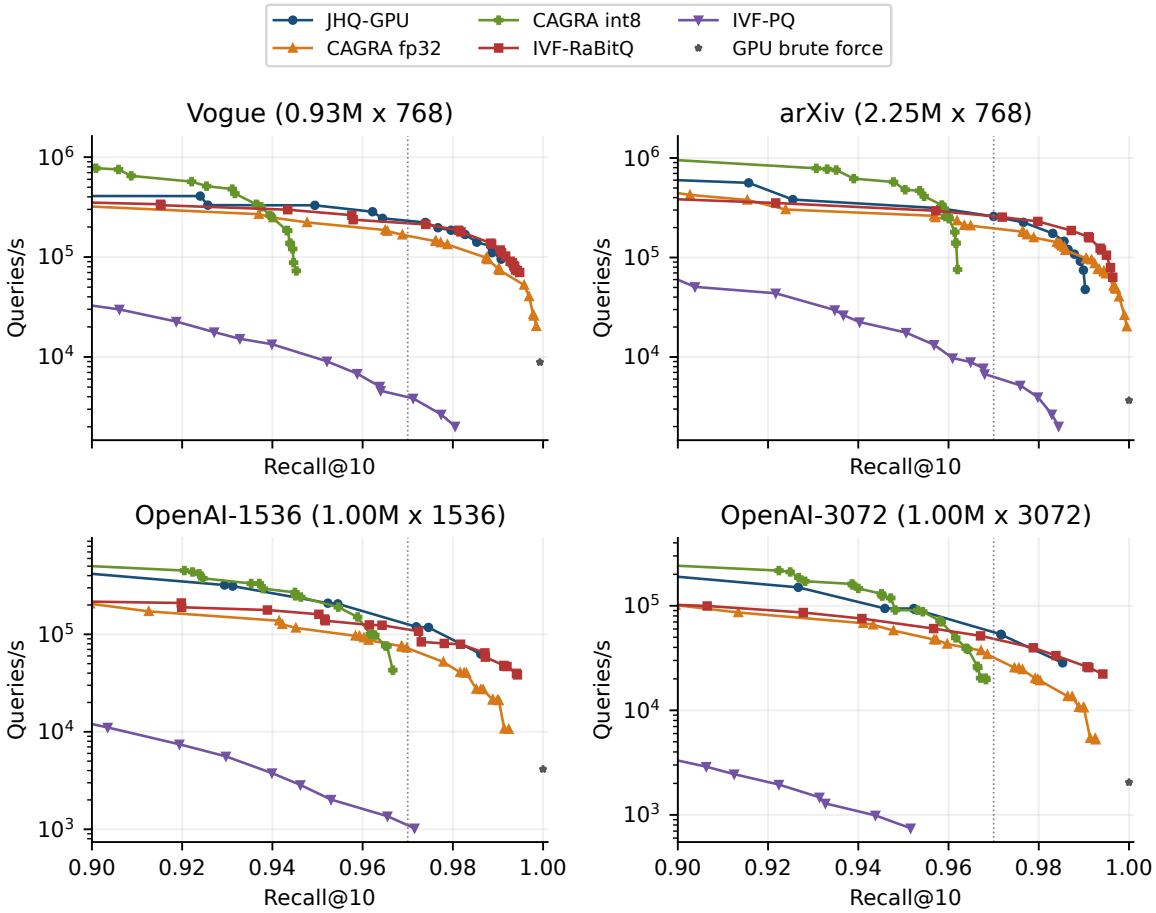



**图 7.1.** 四个 E1 数据集上实测的主机到主机召回率—QPS 前沿。图中从召回率 0.90 开始显示，并在 0.97 处标出竖线；构造前沿时纳入了包括 CAGRA int8 在内的全部基线组。连线连接实测点，吞吐量轴为对数坐标。

**表 7.1.** 召回率 0.97 下的比较。每组比值依次为“插值 / 实测阈值”；大于一表示 JHQ 更快。JHQ kQPS 为插值估计。arXiv 与 RaBitQ 的微小差异随估计方法而改变方向。

| 数据集 | JHQ kQPS | 相对 CAGRA fp32 | 相对 IVF-RaBitQ |
|---|---|---|---|
| Vogue | 230.8 | 1.416 / 1.537 | 1.059 / 1.046 |
| arXiv | 258.6 | 1.317 / 1.422 | 0.996 / 1.015 |
| OpenAI-1536 | 126.7 | 1.747 / 2.294 | 1.130 / 1.111 |
| OpenAI-3072 | 55.9 | 1.747 / 2.088 | 1.157 / 1.346 |

赖调度后的分组路径。每对保持索引、探针、预算和最终输出不变；它测的是集成路径，未单独隔离共享界的贡献。

在有共同覆盖的比较中，目标召回率 0.90–0.985 范围内，JHQ 相对 CAGRA fp32 的插值优势约为 1.10–2.44×。但是，加入 CAGRA int8 后，低召回率区间的解释发生变化：在 0.90–0.92 附近，其低精度图搜索可以快于 JHQ。在更高召回率下，IVF-RaBitQ 常重新领先，包括若干 0.98 及以上目标。在 arXiv 约 0.99 的召回率处，记录网格中的 CAGRA fp32 也快于 JHQ。JHQ 的有利区间同时受到目标召回率与竞争表示的限制。

CAGRA int8 在 Vogue、arXiv、OpenAI-1536 和 OpenAI-3072 上的最高实测召回率分别为 0.9453、0.9620、0.9667 和 0.9683，因此这些网格中没有达到 0.97 的配置。按相同插值规则，目标召回率 0.90–0.92 处，它约比 JHQ 快 1.20–1.98×；在 arXiv 的 0.95 和两个 OpenAI 数据集的 0.94 处也仍然更快。0.97 的结果针对具有测量覆盖的方法，不能推广为低于 0.97 的每个目标均领先。

网格稀疏时，两种估计可能有明显差异。例如，OpenAI-1536 在召回率 0.97 处相对 CAGRA fp32 的比值，插值为 1.747×，而按满足阈值的实测点计算为 2.294×。实测估计支付基线下一个达标点的成本，两种估计之间的差距揭示网格敏感性。

### 7.2.3 OpenAI 数据集上的精度敏感性

补证源码检查确认：E1 实现虽然输入和存储为 FP32，OpenAI 旋转与质心 GEMM 实际采用 TF32 乘法、FP32 累积，源码注释具有误导性（第4.2节）。主 E1 前沿保留该历史实现的测量，D 实验另以 FP32 变体修正 cuBLAS 开关，重跑 37 / 39 个 OpenAI 配置，batch 为 1,000，每配置五轮，在各探针设置间交替变体执行顺序。两者复用同一 TF32 训练模型，改变的是 add / 查询算术，包括数据库编码及路由，而不是独立 FP32 训练模型的对比。表 7.2汇总配对变化。

**表 7.2.** FP32 敏感性。QPS 比值为相同配置下 FP32/TF32 的几何平均，召回差方向相同。最后两列在 recall 0.97 处比较 FP32 前沿与归档 E1 的 IVF-RaBitQ 及 CAGRA fp32；D 实验没有重新测量基线。

| 数据集 | 配对数 | QPS 比值 | ΔRecall | 对 RaBitQ | 对 CAGRA |
|---|---|---|---|---|---|
| OpenAI-1536 | 37 | 0.953 | +0.0003–+0.0005 | 1.086 | 1.679 |
| OpenAI-3072 | 39 | 0.951 | +0.0000–+0.0003 | 1.079 | 1.629 |

相同配置下，FP32 吞吐约低 5%，recall 高 0–0.0005。recall 0.97 处，吞吐比同会话 TF32 前沿约低 3%，仍为归档 RaBitQ 的 1.086× / 1.079×，以及归档 CAGRA fp32 的 1.679× / 1.629×。新会话 v69 测量相对归档 E1 RaBitQ 为 1.118 / 1.114，历史比为 1.130 / 1.157。D 中只有 v69–v72 干预为同期；这些基线比仍是跨会话数值比较。图 7.2展示两条 JHQ 前沿及归档对手。后续 T1 已在同一会话重测基线锚点；第B.4.1节单独报告其受网格影响的结果。

### 7.2.4 结果解释与比较限制

OpenAI 对照来自 Qdrant DBpedia 的 `text-embedding-3-large` 语料族，构成按行对应的嵌入宽度对照 [31, 32]。补证比较全部 1,000 查询及每 97 行一个的 10,299 个 base 行：3,072 维截断并归一化后，与 1,536 维对应向量余弦至少 0.9980，中位 1.000000；GT top-10 平均重合 9.046/10。这强力支持所查导出行顺序一致，但没有导出原始实体 ID 映射。维度翻倍后，历史 0.97 处相对 CAGRA fp32 均为 1.747，RaBitQ 比值从 1.130 变为 1.157；FP32 敏感性也保留相近优势。因此这一配对语料 / 嵌入对照内，维度本身不能解释观测优势。

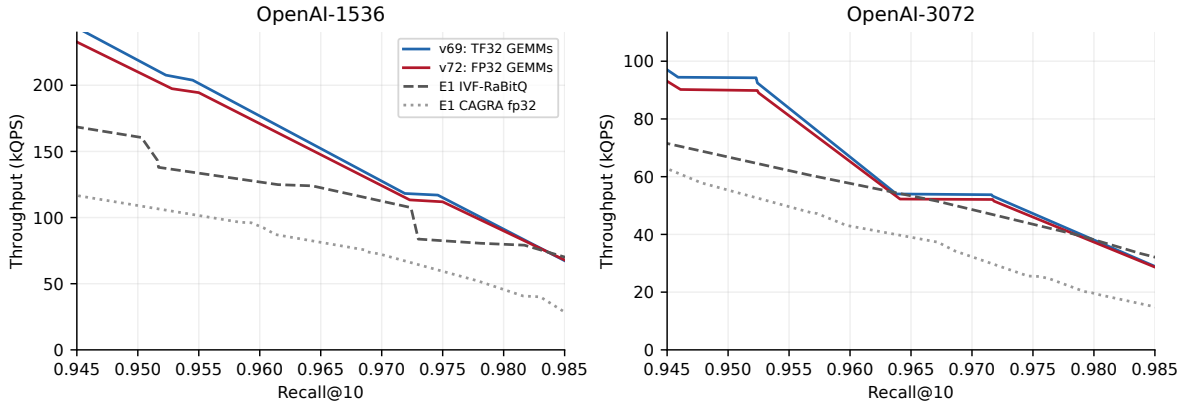



**图 7.2.** D 实验精度前沿。实线为同会话 JHQ 变体，虚线 / 点线为历史 E1 基线；均采用本章前沿及插值规则。

领先幅度反而与精排需求同向变化：E1 设置的 128 探针预算锚点，OpenAI 为 4–8、Vogue 为 64、arXiv 为 100（后来的全池规则在 arXiv 选择 200）；0.97 处相对 RaBitQ 的比值依次为 1.130–1.157、1.059、0.996。这四个数据集的观测关联符合选择性残差访问的设计；锚点既不是各同召回率前沿点的参数，也没有因果隔离主码质量。后文消融直接检查执行组件。

第3.3节中的接口、内存与编译差异仍影响小幅差距的解释；第7.8节进一步检查经验敏感性。

### 7.2.5 小批次改变竞争区间

T5 在四个 E1 数据集、batch 8/32 下比较调度 JHQ、CAGRA fp32/int8 和 IVF-RaBitQ。五轮同步主机到主机调用分别使用 1,000/992 条查询。1,808 个点包含 JHQ 单 block 对照；本实验不含 IVF-PQ 和暴力搜索。图 7.3与表 7.3比较所得到的工作区间。

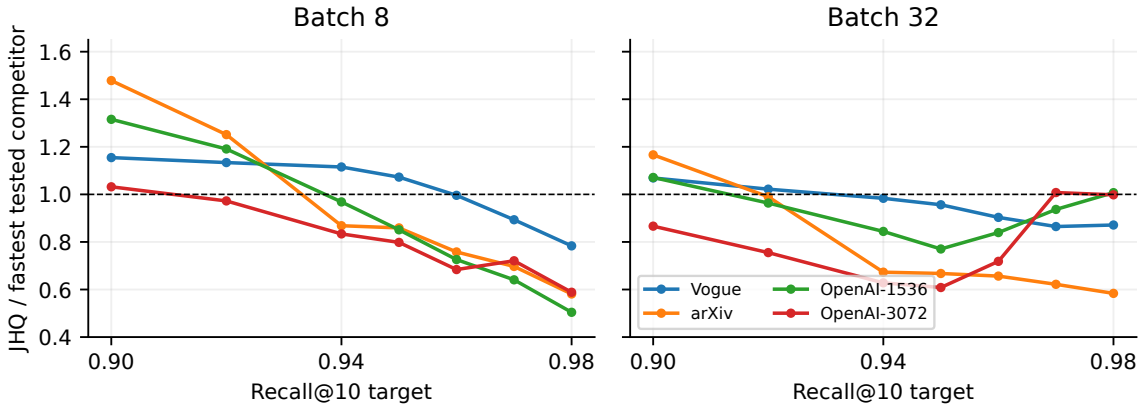



**图 7.3.** T5 包含达标实测端点的插值估计下，JHQ / 最快对手 QPS。大于一表示 JHQ 更快。严格实测阈值见表 7.3。

recall 0.97、batch 8 下，两种估计均显示 CAGRA fp32 在四个数据集领先，JHQ 插值比为 0.641–0.893。batch 32 的插值比为 0.622–1.008；OpenAI-3072 在插值下接近平手，但实测阈值有利于 JHQ。recall 0.95 时，Vogue 的 batch-8 领先同样只出现在插值估计。因此，边界排名受网格分辨率影响。

**表 7.3.** T5 QPS 比值，依次为 “包含达标端点的插值 / 严格实测阈值”。每种估计独立选择最快对手；更高 recall 的达标端点保留，无外推。

| 数据集 | $b$ | $R = 0.90$ | $R = 0.95$ | $R = 0.97$ |
|---|---|---|---|---|
| Vogue | 8 | 1.155 / 1.091 | 1.073 / 0.980 | 0.893 / 0.854 |
| Vogue | 32 | 1.069 / 0.997 | 0.956 / 0.895 | 0.865 / 0.839 |
| arXiv | 8 | 1.479 / 1.454 | 0.859 / 0.851 | 0.697 / 0.698 |
| arXiv | 32 | 1.166 / 1.148 | 0.667 / 0.661 | 0.622 / 0.582 |
| OpenAI-1536 | 8 | 1.316 / 1.131 | 0.851 / 0.825 | 0.641 / 0.620 |
| OpenAI-1536 | 32 | 1.071 / 0.915 | 0.770 / 0.752 | 0.937 / 0.914 |
| OpenAI-3072 | 8 | 1.032 / 0.953 | 0.798 / 0.798 | 0.721 / 0.702 |
| OpenAI-3072 | 32 | 0.866 / 0.734 | 0.608 / 0.771 | 1.008 / 1.351 |

较低 recall 更有利于 JHQ，但必须保留最快前沿点已经超过目标的竞争者。0.90 处，修正后的插值比为 batch 8 的 1.032–1.479 和 batch 32 的 0.866–1.166；严格实测阈值分别给出三个和一个 JHQ 领先。排除这些达标端点会夸大低 recall 优势。调度仍然有用：0.95 处相对单 block 的插值提速为 1.7–3.3×。内部执行提速因此未必转化为跨系统领先。

按数据集、方法与 batch 汇总的基线空调用中位数为 0.012–0.072 ms，该开销仍包含在搜索计时内。CAGRA 使用 Python 绑定，JHQ / RaBitQ 使用 C++，预热策略也有差别。证据支持有限网格、单客户端的工作区间，协议见第A.5节。E1 在 batch 1,000 的高 recall 领先不能推广到小批次。

## 7.3 笛卡尔查找表、打包加载与 lane 协作精排

RQ2 消融同时测量内核与应用效果。

T4 在 128 探针、$\alpha = 100$ 下比较 Vogue、arXiv 和 OpenAI-3072 的相同候选。整表与 $16 + 16$ 表中，22.6–26.1% 候选评分的 FP32 比特模式不同，最大相对误差 $4.53 \times 10^{-7}$。纳入全部表形式后，差异率为 20.5–29.4%，最大相对误差 $4.64 \times 10^{-7}$。Vogue 有 2–4/256 条查询的末端 ID 不同，arXiv / OpenAI 没有。即使主码保留集合不变，舍入后的主码分数仍可经精排改变最终 ID（第A.5节）。

### 7.3.1 查找表的精确因子化及其吞吐量影响

LUT 消融检验第5.2节的成本模型及 FP32 排名影响。

图 7.4 比较六个数据集上的笛卡尔查找表分组，探针数为 8/32/128/512、固定 $\alpha = 100$，以及 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$。每个变体有五轮计时；除 Stella 使用 999 条查询外，批大小均为 1,000。在 24 个单元中，两组查找表有 22 个取得最高吞吐量中位数。例外是 arXiv 的 128 探针配置，单组约快 0.23%；以及 OpenAI-3072 的八探针配置，四组约快 0.84%。这些小差异更适合视为接近平手。新版统计是 22/24，而不是历史 ADC 论文中的 23/24。在明确的 “两组对单组” 配对比较中，两组赢下 23/24 个单元，且各数据集跨四个探针数的几何平均比值均大于一（1.021–1.318）。这支持其在被测工作负载上的选择，无需把共享索引的配置当作独立试验。

两组相对单组的吞吐量比值为 0.998–1.914。四组与八组均存储 16 项，却需要更多查表及加法，说明更小的表不一定提高吞吐；占用率与指令成本尚未单独隔离。任一四变体单元内，报告的最大召回率差为 $10^{-4}$。




**图 7.4.** 固定 $\alpha = 100$ 的笛卡尔查找表分组实验（v71）。组数表示 $g_{LUT}$，不是并行扫描组数。曲线为各数据集相对单组 QPS 比值的中位数，阴影表示范围。

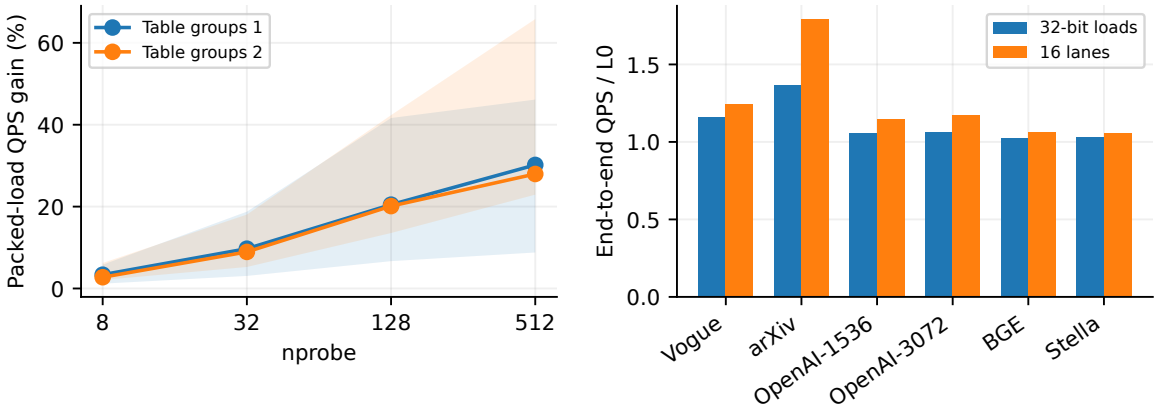



**图 7.5.** 打包访存与精排对照。左：各查找表分组和探针数下，六个数据集打包加载 QPS 增益的中位数，阴影为最小值至最大值。右：128 探针及各数据集普通预算下，仅改变加载和使用 16 个 lane 的实测端到端收益。两个子图来自不同实验组。

### 7.3.2 打包访存与协作式残差评分

历史打包实验固定数据集、探针数和查找表分组，比较以字节和机器字为单位的访存。在六个数据集、四个探针数以及 $g_{LUT} \in \{1, 2\}$ 的 48 对比较中，吞吐量比值为 1.012–1.658，最大召回率差为 $10^{-4}$。八探针时收益通常较小，512 探针时较大。两种实现都已经采用子空间优先布局，因此，这个实验支持打包访存的效果，而不是“仅靠物理转置就取得了提升”的说法。每个变体只报告一个汇总吞吐率，因而这一较早证据提供的不确定性信息弱于五轮计时的 v71 扫描。

图 7.5与表 7.4 进一步区分了残差精排中更宽加载与 lane 协作的作用。L0 是原始标量路径；L1 改变加载方式，但不将单个候选分给多个 lane；L16 则由 16 个 lane 协作处理一个候选。同一进程在五轮计时中轮换变体顺序，每轮每个变体执行三次搜索。在十二个普通配置上，L16 的端到端吞吐量为 L0 的 1.015–1.789×。另外加入的 arXiv $\alpha = 200$ 配置明确属于高预算对照，达到 2.383×；不能把它并入普通所选预算的收益范围。

在被测用例中，仅改变加载的对照与 L0 逐位一致，但已使 arXiv 吞吐量提高至 1.368×。lane 协作进一步缩短了精排时间。在 arXiv 的 $\alpha = 100$ 配置上，精排内核由 4,277.5 降至

**表 7.4.** 128 探针下的精排结果。除最后一列为剖析得到的精排内核加速比外，其余比值均为相对 L0 的配对端到端收益。不能将内核加速比与应用整体加速比混用。

| 数据集 | $\alpha$ | L1/L0 吞吐比 | L16/L0 吞吐比 | 内核加速比 |
|---|---|---|---|---|
| arXiv | 100 | 1.368 | 1.789 | 5.97 |
| BGE | 32 | 1.026 | 1.062 | 1.97 |
| OpenAI-1536 | 8 | 1.055 | 1.145 | 3.34 |
| OpenAI-3072 | 4 | 1.063 | 1.171 | 8.81 |
| Stella | 16 | 1.030 | 1.053 | 2.66 |
| Vogue | 64 | 1.161 | 1.240 | 3.72 |

716.8 $\mu$s，即 5.97× 的内核加速，但整体吞吐量只提高至 1.789×。OpenAI-3072 的精排内核加速为 8.81×，整体吞吐量则为 1.171×。剩余路由、筛选、选择与传输成本限制应用收益。

定量 Amdahl 检验使用基线精排内核时间占主机到主机基线延迟的比例 $f$，以及内核加速比 $s$；若其余耗时不变，整体估计为 $1/(1-f+f/s)$。arXiv 和 OpenAI-3072 的比例约为 0.529 和 0.168，对应估计 1.787× 和 1.175×，接近实测 1.789× 和 1.171×。它结合了单独的内核剖析与墙钟测量，只是机制解释的估计。

协作归约改变了浮点加法顺序。被测多 lane 变体的距离差异很小，最大报告相对误差约为 $1.6\times10^{-6}$，但排序边界附近偶有结果变化。Sanitizer 在较小查询子集上检查记录中的精排与 top-$k$ 内核。这些检查和数值容差界定证据范围，多 lane 评分并非普遍逐位一致。

## 7.4 全局 top-c、共享界与探针选择的正确性及消融

第4.6节核查存储表示，本节评价查询选择及其成本。

### 7.4.1 计算分数选择的严格 CPU 参考

补证 T4 使用 GPU 生成的 FP32 分数，在 CPU 上按 $(D_{\mathrm{P}}, \mathrm{position})$ 对每条查询的全部已访问候选排序。诊断导出器遵循打包扫描布局和累加顺序；所有返回有效主码项的导出分数与选择器分数逐位一致。源码对照确认，诊断版本相对拆表实验实现只改变导出部分。Vogue、arXiv、OpenAI-3072 采用 1/8/128 探针、$\alpha\in\{4,100,200\}$、$g_{\mathrm{scan}}\in\{0,1,2,4,8\}$ 及共享界开 / 关，共 270 个配置，每个配置 256 条查询，即 69,120 个查询实例（表 7.5）。

**表 7.5.** T4 CPU 参考检查。集合及顺序一致使用完整主码键。有效数量检查 $\min(c,n_c)$ 个 survivor；末端一致性按 $(D_{\mathrm{H}}, \mathrm{ID})$ 排序已保留的计算精排分数，包含 FP32 比特比较。计数为查询实例，同一查询在配置间重复使用。

| 路径 | 查询数 | 集合 / 顺序精确 | 纯并列差异 | 有效数量正确 | 末端精确 |
|---|---|---|---|---|---|
| $g_{\mathrm{scan}}\geq1$ | 55,296 | 55,296 / 55,296 | 0 | 55,296 | 55,296 |
| $g_{\mathrm{scan}}=0$ | 13,824 | 13,768 / 8,758 | 56 | 13,824 | 13,824 |

55,296 个分组实例全部与完整键参考的顺序一致。单 block 的 13,824 个实例全部满足距离全预序，56 个集合差异全部位于截断并列。19,120 个候选不足实例均返回正确数量的有效 survivor；全部 69,120 个末端选择均与保留分数参考一致。这直接检查 引理 5.1与命题 5.1与5.2 的阶段契约。此前 64 个扫描对照及 48 个共享界对照仍提供独立的实现间配对证据；归档 sanitizer 覆盖较小子集。导出与归档边界见第A.5节。

### 7.4.2 共享界保持选择目标不变

命题 5.2给出选择保证；配对实验测量共享该界所节省的工作。

**表 7.6.** 已算分数的精确探针选择改动的 v69/v65 端到端吞吐量比。每个工作负载—批大小比较中，每个二进制程序共计时十轮，采用交替运行。大于一表示 v69 更快。

| 数据集 | 批大小 8 | 32 | 256 | 1,000 |
|---|---|---|---|---|
| arXiv | 1.074 | 1.063 | 1.035 | 1.034 |
| Vogue | 1.109 | 1.103 | 1.052 | 1.041 |
| OpenAI-3072 | 1.167 | 1.117 | 1.055 | 1.050 |

四个工作负载与四种 batch 的 48 个共享界 / 独立界比较，主码和末端输出均一致。五轮配对吞吐比为 0.994–1.562×。arXiv 的 $\alpha = 200$、batch 1,000、八组配对剖析中，merge 约由 2,629 降至 471 $\mu$s，scan 由 7,968 降至 5,513 $\mu$s。共享能减少合适工作负载的中间工作，其他配置也有小幅回退。

未采用的静态配额设计面临优胜候选集中：pilot 中第一候选组包含全局 survivor 的 46–81%。等额配额需要正确性回退，校准的不等额配额仍让主导组处于关键路径。这些观测在该 pilot 范围内促成了共享界设计。

### 7.4.3 精确探针选择与流水线一致性的边界

当筛选变快后，IVF 探针选择在小批次时间中的占比会提高。基数探针原语（v69）改变了选取最小质心评分的实现，保留此前算分制度；OpenAI 包含 TF32 GEMM（第7.2.3节）。其合成检查覆盖 1,792 个用例，失败数为零。在真实 arXiv、Vogue 和 OpenAI-3072 工作负载上，检查发现全部 1,000 条查询的探针标识符相同，主码 top-$c$ 位置与距离也相同。

表 7.6 显示端到端吞吐量提高至 1.034–1.167×。批大小为 8 时，剖析得到的探针阶段在 arXiv 上从 56.5 降至 22.6 $\mu$s，在 Vogue 上从 52.7 降至 14.7 $\mu$s，在 OpenAI-3072 上从 77.8 降至 16.7 $\mu$s。未修改阶段限制大批次收益，各分组比值与前述消融采用不同基线。

Vogue 的一组、batch 8 测试中，两种二进制的探针 / 主码 dump 在全部 1,000 条查询上逐字节一致，但最终 recall 为 0.9642 / 0.9644。同一基线也从一致性运行的 0.9642 变成两次计时中的 0.9644。因此，不改探针也会出现差异；在 dump 控制情形下，差异位于主码选择之后，根因仍未解决。各已查进程内，调度与单 block 输出逐位一致；dump 覆盖见附录 A。

最后，可选的 v70 FP16 实验属于另一项近似。在 arXiv 的 936/1,000 条、Vogue 的 979/1,000 条、OpenAI-3072 的 998/1,000 条查询上，其探针集合与 v69 参考完全相同，平均集合重叠率很高。但高重叠率和很小的召回率变化，并不能使该原语变成精确操作。这些结果不纳入主要 E1 前沿，也不纳入精确探针选择结论。

## 7.5 留出查询上的预算质量、吞吐量与回本

RQ3 将第6章的策略与固定 $\alpha = 100$ 及更强的统一固定对照 $\alpha = 64$ 比较。

### 7.5.1 无标签选择与使用标签的诊断性 oracle

六个数据集均测试 128 与 512 探针，采用第6.4节中的固定、采样、FULL、枚举与 oracle 对照。不相交查询池和每个样本大小 64 次抽样沿用第3.2节。

所有规则变体使用式 (6.4)，取 $k = 10$、$\epsilon = 1$。

FULL 使用全部校准查询，诊断 oracle 使用留出标签（表 6.1）。

**表 7.7.** 全池校准（$S = 500$）。损失相对最佳实测留出 recall；速率为所选固定预算的 kQPS。“校准”每项仅一次全池调用，不是重复查询流计时。留出为 500 条，Stella 为 499。

| 数据集 | 探针数 | $\alpha_{\text{full}}$ | 损失 | $Q_{\text{full}}$ | 相对 100 | 相对 oracle | 校准 ms |
|---|---|---|---|---|---|---|---|
| Vogue | 128 | 100 | 0.0000 | 202.9 | 1.000 | 0.908 | 10.1 |
| arXiv | 128 | 200 | 0.0000 | 151.5 | 0.728 | 1.000 | 10.0 |
| OpenAI-1536 | 128 | 8 | 0.0000 | 267.7 | 1.599 | 1.000 | 9.5 |
| OpenAI-3072 | 128 | 4 | 0.0000 | 145.0 | 1.691 | 1.000 | 17.1 |
| BGE | 128 | 64 | 0.0006 | 124.9 | 1.075 | 0.932 | 17.1 |
| Stella | 128 | 16 | 0.0006 | 110.4 | 1.153 | 1.000 | 20.6 |
| Vogue | 512 | 100 | 0.0002 | 100.1 | 1.000 | 0.969 | 20.9 |
| arXiv | 512 | 200 | 0.0000 | 69.0 | 0.831 | 1.000 | 25.1 |
| OpenAI-1536 | 512 | 8 | 0.0000 | 114.9 | 1.326 | 1.000 | 21.8 |
| OpenAI-3072 | 512 | 4 | 0.0002 | 52.1 | 1.219 | 0.977 | 40.5 |
| BGE | 512 | 64 | 0.0006 | 50.8 | 1.033 | 1.000 | 42.3 |
| Stella | 512 | 16 | 0.0006 | 36.0 | 1.041 | 1.000 | 60.3 |

**表 7.8.** $S = 128$ 时的留出质量。规则的 $\alpha$ 是 64 次抽样的中位数，oracle 的 $\alpha$ 使用留出标签。$\overline{R}_{\text{rule}}$ 是重构的平均召回率。“超标”统计相对最佳被测留出召回率损失 $> 10^{-3}$ 的抽样次数；$L_{95}$ 是日志采用较低次序统计量得到的 95 分位损失估计。$R_{100}$ 是固定预算召回率。

| 数据集 | 探针数 | $\alpha$：规则/oracle | $R_{100}$ | $\overline{R}_{\text{rule}}$ | 超标 | $L_{95}$ |
|---|---|---|---|---|---|---|
| Vogue | 128 | 64 / 64 | 0.9680 | 0.9677 | 5/64 | 0.0022 |
| arXiv | 128 | 100 / 200 | 0.9748 | 0.9751 | 40/64 | 0.0024 |
| OpenAI-1536 | 128 | 8 / 8 | 0.9280 | 0.9279 | 4/64 | 0.0000 |
| OpenAI-3072 | 128 | 4 / 4 | 0.9252 | 0.9252 | 0/64 | 0.0000 |
| BGE | 128 | 32 / 32 | 0.9198 | 0.9190 | 1/64 | 0.0008 |
| Stella | 128 | 16 / 16 | 0.9892 | 0.9884 | 6/64 | 0.0030 |
| Vogue | 512 | 64 / 64 | 0.9916 | 0.9915 | 2/64 | 0.0002 |
| arXiv | 512 | 200 / 200 | 0.9878 | 0.9885 | 32/64 | 0.0016 |
| OpenAI-1536 | 512 | 8 / 8 | 0.9740 | 0.9738 | 4/64 | 0.0000 |
| OpenAI-3072 | 512 | 4 / 8 | 0.9704 | 0.9702 | 0/64 | 0.0002 |
| BGE | 512 | 64 / 64 | 0.9670 | 0.9661 | 24/64 | 0.0014 |
| Stella | 512 | 16 / 16 | 0.9954 | 0.9945 | 7/64 | 0.0030 |

### 7.5.2 全池校准作为质量优先工作点

全池校准每个单元使用全部 500 条校准查询一次。选中预算在 12/12 单元均满足 $10^{-3}$ 留出损失目标，最大观测损失为 0.0006；速率为 GT oracle 的 0.908–1.000，校准为 9.5–60.3 ms。相对固定 100 的几何平均速率比为 1.111×，仅计该对照也达标的十项则为 1.193×。完整比较见表 7.7。相对固定 64，全池选择六项更快、两项相等、四项更慢，其主要优势是十二项质量目标均观测达标。

这些单元支持 FULL 的质量覆盖与 $S = 128$ 的较低校准成本；统计范围见第6.2节。

### 7.5.3 低成本采样及其质量风险

表 7.8 给出了 $S = 128$ 的结果。十二个单元中，所选预算的中位数有十个与 oracle 预算一致；不一致的两个是 arXiv 的 128 探针配置和 OpenAI-3072 的 512 探针配置。所选预算跨越 4–200，显示出明显的工作负载依赖。预算中位数与平均 recall 未必对应同一配置；后者由打印的最佳 recall 减平均损失重构，四位小数仅为近似。

一个重要反例是 arXiv。在 128 探针下，即使平均损失仅为 0.0009，仍有 40/64 次抽样的召回率损失超过 $10^{-3}$；512 探针下有 32/64 次超标。BGE 的 512 探针配置有 24/64 次超

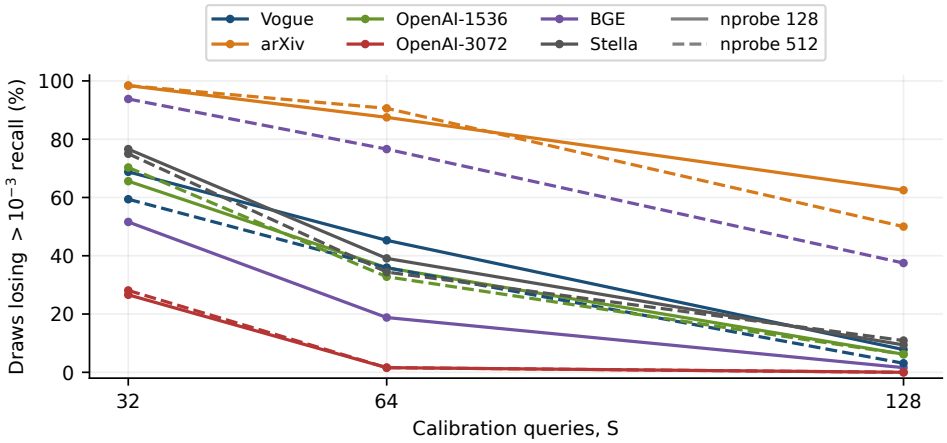



**图 7.6.** 不同样本大小下的留出风险。纵轴为相对最佳被测预算，留出召回率损失超过 $10^{-3}$ 的校准抽样百分比。由于式 (6.4) 始终保持阈值一，增加 $S$ 也收紧了允许的槽位不一致比例。

标，95 分位损失为 0.0014。Stella 超标次数较少，但两个探针数下的 95 分位损失均为 0.0030。OpenAI-3072 在这一有限池中未观察到超标。平均损失、越阈频率和尾部幅度区分这些结果。

按第A.3节中的较低次序统计量约定（从零起算第 59 项），四个非零尾部损失可与 $L_{95} = 0$ 并存，OpenAI-1536 即如此。表中的越阈计数保留这一区别。

图 7.6 显示了样本大小的影响。更大的样本通常使选择更保守，但同时收紧了归一化允许量 $1/(10S)$。在全部 2,304 次抽样（$12 \times 3 \times 64$）中，同样本枚举与二分搜索一致，而打印日志显示枚举耗时为其 1.75–2.01 倍。

推论 6.2覆盖 2,304 次抽取中的 2,176 次。剩下 128 次为 $S = 128$ 的 OpenAI-3072 距离截断并列路径，经验上与枚举一致。八个单元校准到评测的换路涉及预算迁移，区别于本项核查。

八个固定选项中，$\alpha = 64$ 和 100 各有 10/12 单元达标，200 为 12/12；各自达标单元相对诊断 oracle 的最大 QPS 缺口为 26.6%、40.9%、57.1%。完整八档比较保留在仓库补充材料。Vogue 的两个探针数下，FULL 选 100 而 oracle 选 64，说明参考一致性与质量约束下最快选择具有不同目标。此处 $\alpha = 200$ 达到最佳实测 recall，12/12 达标来自其覆盖对照的定义。短计时中的 OpenAI 小预算反转经交替重复被解释为计时变化，并形成低预算平台（第7.5.5节）；最小接纳预算未必使吞吐最大。

### 7.5.4 估计吞吐量与校准经济性

按第A.3节规定的所选配置平均 QPS 估计，$S = 128$ 在十个单元中优于固定 $\alpha = 100$，比值为 1.028–1.691×。arXiv 比值为 0.908 和 0.917：平均召回提高而吞吐降低，因而没有吞吐量回本。

更强的 $\alpha = 64$ 对照缩小了整体收益：所选配置的平均 QPS 在 9/12 个单元中胜出，几何平均比值为 1.054×，范围为 0.821–1.367×。Vogue 几乎不变；BGE 的 512 探针配置只提高 0.6%；OpenAI 的收益范围为 1.052–1.367×。固定 64 和固定 100 都在 10/12 个单元达标，失败的都是两个 arXiv 单元。校准的价值是适应不同精排需求，而不是始终击败已经合理的固定预算。

**表 7.9.** 校准经济性，$S = 128$。速率单位为 kQPS，规则速率为所选配置平均 QPS。比值分别相对固定 100 与固定 64。“回本批数” 相对固定 100，使用 500 条查询，Stella 为 499。“不回本” 表示没有正吞吐节省；质量列独立标记固定 100 是否满足留出损失目标。

| 数据集 | 探针数 | $Q_{100}$ | $\overline{Q}_{\mathrm{rule}}$ | 相对 100 | 相对 64 | 校准 ms | 回本批数 | 固定 100 质量 |
|---|---|---|---|---|---|---|---|---|
| Vogue | 128 | 202.9 | 224.4 | 1.106 | 1.004 | 4.0 | 17.0 | 达标 |
| arXiv | 128 | 207.9 | 188.8 | 0.908 | 0.821 | 4.2 | 不回本 | 未达标 |
| OpenAI-1536 | 128 | 167.5 | 268.5 | 1.603 | 1.367 | 3.9 | 3.5 | 达标 |
| OpenAI-3072 | 128 | 85.7 | 145.0 | 1.691 | 1.230 | 5.7 | 2.4 | 达标 |
| BGE | 128 | 116.2 | 131.2 | 1.130 | 1.051 | 6.5 | 13.2 | 达标 |
| Stella | 128 | 95.7 | 110.3 | 1.153 | 1.087 | 7.5 | 10.8 | 达标 |
| Vogue | 512 | 100.1 | 102.9 | 1.028 | 0.996 | 7.5 | 55.4 | 达标 |
| arXiv | 512 | 83.1 | 76.2 | 0.917 | 0.870 | 9.7 | 不回本 | 未达标 |
| OpenAI-1536 | 512 | 86.6 | 115.0 | 1.327 | 1.268 | 8.8 | 6.2 | 达标 |
| OpenAI-3072 | 512 | 42.8 | 52.1 | 1.219 | 1.052 | 13.1 | 6.2 | 达标 |
| BGE | 512 | 49.2 | 51.1 | 1.039 | 1.006 | 15.8 | 41.2 | 达标 |
| Stella | 512 | 34.6 | 36.0 | 1.041 | 1.028 | 21.3 | 37.1 | 达标 |

arXiv 的超标是相对最佳被测留出召回率，不是相对默认配置。固定 100 在 128 和 512 探针下本来分别损失 0.0012 和 0.0016，而抽样平均损失在两个单元均为 0.0009。因此，即使部分抽样超过相对 oracle 的目标，平均质量仍比默认配置更好。

按第A.3节的计时边界，校准成本为 3.9–21.3 ms。对固定 $\alpha = 100$ 应用式 (6.15)，取毫秒单位的 $T_{\mathrm{rule}} \approx 1000n/\overline{Q}_{\mathrm{rule}}$，得到表 7.9中的估计：有利单元相对固定 100 约需 2.4–55.4 个留出池大小的后续批次。相对固定 64，九个有利单元约需 5.7–404.3 批，Vogue 的 512 探针配置没有吞吐量回本。两个 arXiv 对照也未满足质量目标，因此其计时比较是诊断，不是同质量比较。按对照速率换算，$S = 128$ 的最大正回本估计相对固定 100 为 0.54 s、相对固定 64 为 2.72 s，约 27.7K 和 202.2K 查询。全池正收益项按自身校准成本计算，对应最多 1.52/2.25 s、64.9K/78.7K 查询。这些模型时域支持长稳定任务复用。

这些估计沿用第6.5节分析的平均速率倒数与稳定复用假设；下文直接执行核查它们在归档查询流上的净成本含义。

较新的 lane 精排流水线比历史标量实现保留更少的可消除精排工作，历史对比见仓库补充材料。各次运行共享预算协议与网格，但跨实验比较没有隔离精排作为变化的唯一原因。

### 7.5.5 完整决策轨迹与有限池风险

补证程序保留原网格、训练状态、查询划分与固定 seed 抽样，新增完整日志。十二单元的固定预算 recall 在归档精度下全部复现，全池选择、采样中位选择与越阈计数也全部复现。计时让两个近乎并列单元的 GT oracle 最快臂变化，说明这一诊断也依赖所测速度。

2,304 条采样不一致曲线均单调，选择全部与枚举一致；十二条全池曲线也通过两项检查。进一步，将全池逐查询计数按各次实际样本 ID 求和，18,432 个网格计数全部复现。这比只核对最终预算更强，支持在这些观测中使用式 (6.9)，包括换扫描路径的情形；它验证已记录子集的汇总计数，不证明任意子集的逐查询不变性。十二单元的逐抽样最大损失完整矩阵保留在仓库补充材料。

$S = 128$ 时，arXiv/512 最大损失 0.0050、BGE/512 为 0.0046；$S = 32$ 最大达 0.0294。因此较小平均损失可以伴随少数明显更差的选择。十二个 FULL 单元的最大损失仍为 0.0006。

arXiv/128 在 $S = 32, 64, 128$ 的有限池越阈预测为 0.965、0.875、0.620，实测为 0.984、0.875、0.625（图 7.7）。接纳只用参考输出；将选择概率转为 GT 损失越阈率还要固定预算

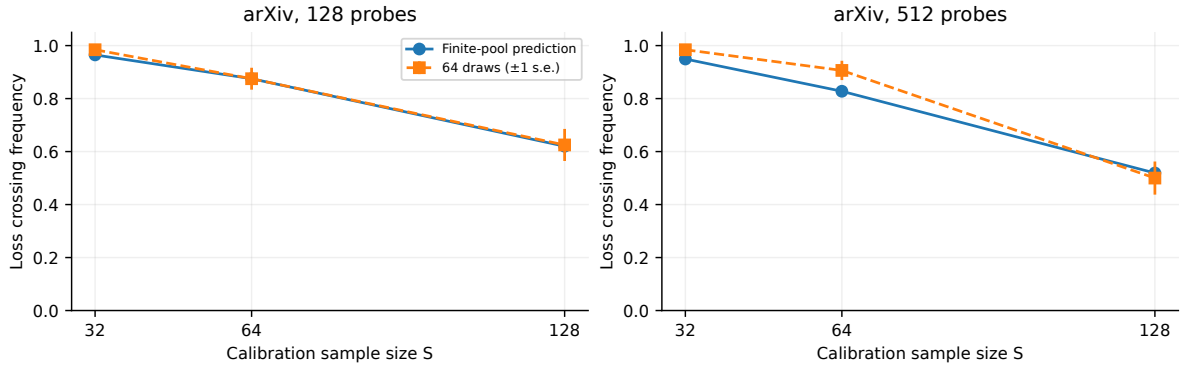



**图 7.7.** arXiv 损失越阈的有限池预测与观测。误差条是在固定池条件下 64 次独立抽取子集的一倍估计标准误，不是未来工作负载的不确定性。

的留出质量映射，因此这是离线风险解释，不是无标签 GT 保证。36 项比较中最大绝对差为 BGE/512 在 $S = 128$ 的 0.144（预测 0.519、实测 0.375）。有限池计算隔离抽样风险，没有用留出 recall 差拟合总体模型。

十轮交替重测也消除了表面的小预算速度反转。OpenAI-3072/128 在 $\alpha = 4/2$ 时为 142.3 / 142.7 kQPS，原短计时为 145.0 / 128.7；BGE/128 与 Vogue/512 同样趋于平台。这支持将归档反转解释为计时变化，但不代表所有预算和资源阈值上的运行时间均单调。

### 7.5.6 直接校准加复用测量

四个单元直接执行固定 64、固定 100、采样 $S = 128$ 与 FULL 策略，五轮间轮换策略顺序。每个校准轮先收集样本、校准一次、显式设置所选乘数，再连续计时 500 个留出批次。首批包含可能的 graph 重新捕获，重复使用同一留出池，没有采样新流量。索引构建、此前预热及另行计时的调度校准不在该边界内。表 7.10报告累计测量。

**表 7.10.** 500 个复用批次的直接总毫秒数，五轮平均；采样与 FULL 包含预算校准，每批 500 查询。质量与计时结果一起讨论。

| 数据集/探测数 | 固定 100 | 固定 64 | 规则 $S = 128$ | 全池 |
|---|---|---|---|---|
| arXiv/128 | 1224 | 1102 | 1291 | 1680 |
| OpenAI-1536/128 | 1491 | 1264 | 852 | 860 |
| OpenAI-3072/128 | 2757 | 2160 | 1769 | 1783 |
| BGE/512 | 5185 | 5018 | 4921 | 5062 |

OpenAI-3072/128 的采样策略每轮均选 4，相对固定 100 将总时间从 2,757 降到 1,769 ms，约节省 36%，测得 recall 同为 0.9252。FULL 选同一预算，含较高校准成本共 1,783 ms。OpenAI-1536/128 的 FULL 为 860 / 固定 100 的 1,491 ms，recall 同为 0.9280；采样策略平均 852 ms、recall 0.9276。

从直接计时的校准和稳态批次拟合，采样策略相对固定 100 / 64 的回本分别为：OpenAI-1536 的 3.6 / 5.5 批、OpenAI-3072 的 3.5 / 8.9 批。小数值是稳态速率拟合，不是直接观测到的小数批次交点；图 7.8绘制含首批设置的实测累计总时间。相对固定 100 / 64，平均累计时间首次正节省在 OpenAI-1536 为 4 / 6 批，OpenAI-3072 为 4 / 9 批。BGE/512 采样收益较小，相对固定 100 为 4,921 / 5,185 ms，平均 recall 为 0.9658 / 0.9670；FULL 恢复至 0.9664，耗时 5,062 ms。arXiv 的 FULL 为提高质量选择 200，慢于固定 100 / 64，而

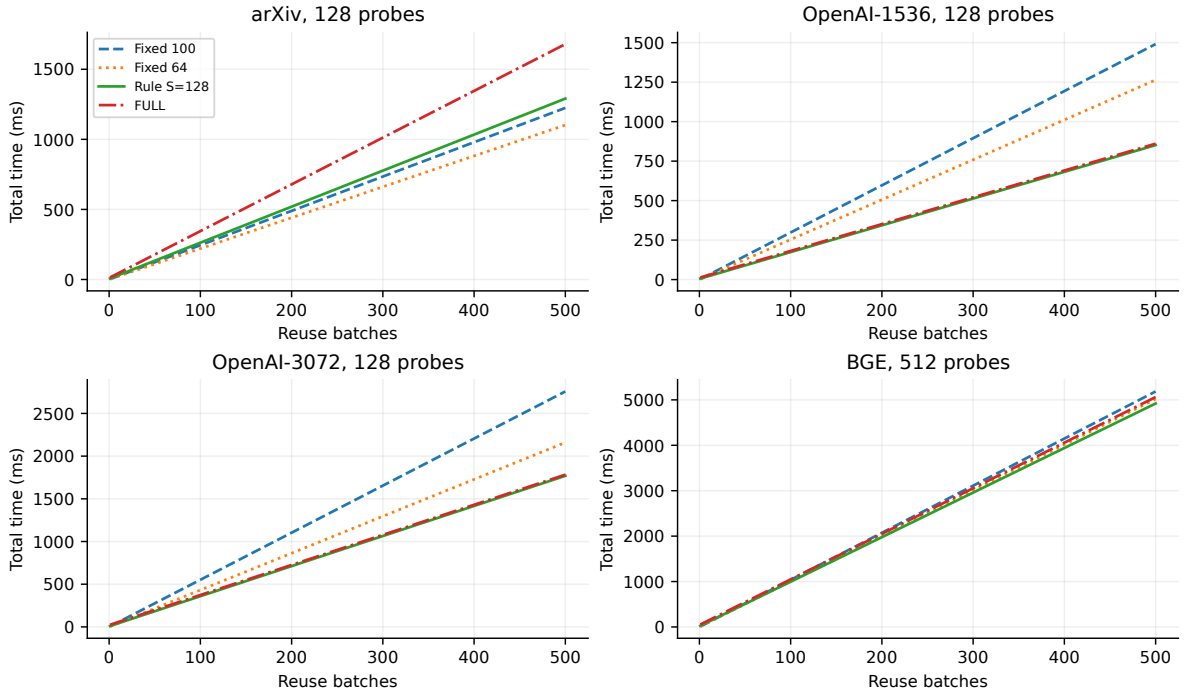



**图 7.8.** 含预算校准及首批设置的直接平均累计时间，各曲线重复同一固定留出池。不同策略 recall 可不同，因此不能仅按时间作质量匹配排名；配置对应表 7.10。

两个固定控制均未达到质量目标。因此必须联合质量和时间判断校准价值。测量支持复用经济性，不要求旧模型回本值在每个单元精确吻合。

## 7.6 批大小、扫描组调度与工作负载偏斜

本节回答 RQ2 的性能区间问题：何时额外扫描组提供有价值的并行度，何时额外工作成为主导。

### 7.6.1 并行度与重复选择工作的权衡

扫描实验揭示了两种执行区间。每条查询使用一个线程块时，批大小 8 只在具有 170 个 SM 的设备上提供八个线程块。划分候选能增加可用线程块并行度。在 v62 实验中，批大小 8 使用八组后，端到端吞吐量为原来的 2.06–2.76×。剖析给出了阶段解释：arXiv 扫描约从 849 降至 216 $\mu$s，Vogue 从 1,059 降至 246 $\mu$s，OpenAI-3072 从 728 降至 155 $\mu$s。额外合并分别约需 16.7、9.1 和 3.7 $\mu$s。即使支付合并成本，净节省仍很大。

批大小为 1,000 时，独立查询已足以提供较多线程块。增加分组会重复搬入查找表、接纳局部候选并排序，还产生额外合并流量。对于 arXiv 的 $\alpha = 200$ 配置，不共享界的 v62 路径可能降至单线程块基线的约 0.46×。全批次剖析显示，$\alpha = 100$ 的扫描从 2,922 增至 6,079 $\mu$s，$\alpha = 200$ 从 3,530 增至 7,652 $\mu$s，后者还增加了 2,624 $\mu$s 合并时间。这一反向结果对设计至关重要：并行拆分能够解决并发不足，但当并发已经充足时，它引入的额外工作可能成为主导。共享界减少了这些额外工作，却没有消除执行区间的变化。

### 7.6.2 在更多配置上评估预先固定的规则

调度器沿用第5.5节的填充 / 偏斜规则及伪查询校准。规则常数在四个配置上确定，之后再测额外配置。

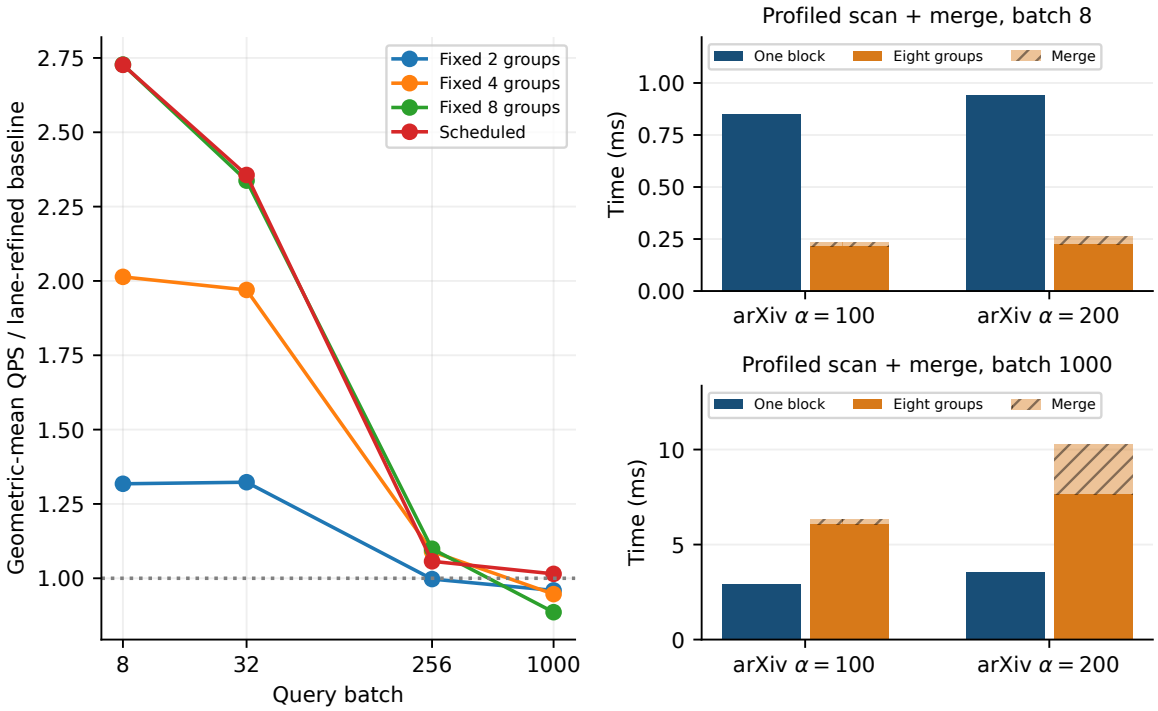



**图 7.9.** 执行区间。左：共享界固定组数与调度执行，相对 lane 精排单线程块基线的吞吐量比，较小批次有 13 个单元，批大小 1,000 有 11 个。右：arXiv 预算 100 和 200 在批大小 8 与 1,000 下的配对扫描、合并剖析；斜线表示合并时间。右侧来自独立界扫描实验，用于解释一项成本，不代表调度器全部收益。

**表 7.11.** 集成执行相对 v61 L16 的收益。每项先计算单元内五轮配对加速比的中位数，再跨单元取几何平均。“留出配置”排除了设置规则时使用的四个配置，指的是未用于设定规则的配置，不是预算实验中的留出查询划分。

| 批大小 | 单元数 | $g_s = 2$ | $g_s = 4$ | $g_s = 8$ | 调度 | 留出配置 |
|---|---|---|---|---|---|---|
| 8 | 13 | 1.318 | 2.014 | 2.728 | 2.727 | 2.930 |
| 32 | 13 | 1.323 | 1.970 | 2.337 | 2.356 | 2.560 |
| 256 | 13 | 0.997 | 1.090 | 1.099 | 1.057 | 1.041 |
| 1000 | 11 | 0.959 | 0.947 | 0.886 | 1.015 | 1.012 |

表 7.11 从原始计时数组重新计算结果。批大小为 8、32、256 和 1,000 时，调度收益的几何平均分别为 2.727×、2.356×、1.057× 和 1.015×。前三种批大小有十三个配置，含高预算 arXiv 对照；batch 1,000 只有十一个，因为 Stella 仅有 999 条查询。留出配置子集含九个和七个，故连线均值对应部分不同的配置集合。

固定八组在批大小 8 时的均值接近调度结果，但在批大小 1,000 时降至 0.886×。调度大体保留了小批次收益，并避免上述大批次损失。批大小 256 时，在一个 BGE 配置上，“最佳固定吞吐量 / 调度吞吐量”的最差比值为 1.217。候选数 CV 无法描述列表遍历、选择或合并的全部成本。

Vogue 同时说明偏斜特征的价值与局限。E1 候选数 CV 随探针数在 0.338–0.622 变化，使批大小 1,000 仍采用四组。集成路径相对单线程块的比值为 0.895–1.175；在 $(n_{\text{probe}}, \alpha) = (16, 16)$、$(16, 64)$、$(32, 64)$ 时变慢，比值依次为 0.955、0.895、0.955。候选数离散程度没有编码每条查询的绝对工作量，因此规则可能给这些低探针配置增加开销，同时又支撑第7.2节中 0.97 的跨系统领先。两项失败都暴露出 CV 特征未覆盖的分解成本。

**表 7.12.** JHQ 冷构建与单独测量的常驻显存。训练 / 添加为三次补证正常构建的分项中位数，合计为配对和的中位数。显存来自 v71 两组、128 探针、$\alpha = 100$ 的运行，包含上下文与工作区。“残差占比”表示模型计算的残差码占实测已用总量的比例，而不是每次查询读取的比例。

| 数据集 | 训练 s | 添加 s | 合计 s | 已用 GiB | 原始 GiB | 残差占比 |
|---|---|---|---|---|---|---|
| Vogue | 0.30 | 0.26 | 0.56 | 1.47 | 2.67 | 45.4% |
| arXiv | 0.48 | 0.51 | 0.99 | 2.58 | 6.45 | 62.5% |
| OpenAI-1536 | 0.61 | 0.38 | 1.00 | 2.39 | 5.72 | 59.7% |
| OpenAI-3072 | 1.16 | 0.67 | 1.82 | 4.14 | 11.43 | 69.0% |
| BGE | 3.56 | 3.65 | 7.20 | 11.93 | 38.50 | 80.7% |
| Stella | 3.55 | 6.99 | 10.55 | 20.44 | 67.81 | 82.9% |

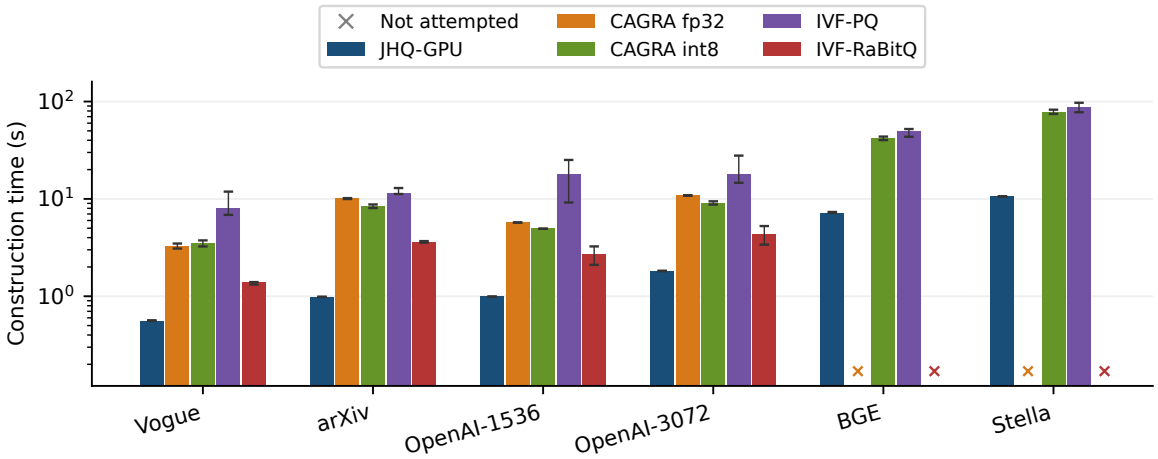



**图 7.10.** 包括 CAGRA int8 的已记录构建成本。JHQ 为热输入页下三次补证正常冷构建的中位数和范围；其他柱形为成功测量的索引配置之间的中位数，误差线表示最小值至最大值。Int8 包含数据集量化；RaBitQ 不包含所需的序列化往返。大数据集缺失的柱形表示未尝试的配置，不代表实测构建失败。纵轴为对数坐标。

最后，正确性检查独立于计时。在 250 个变体/配置比较、234,320 次逐查询比较中，集成实验未发现非并列主码选择差异，也未发现最终标识符集合差异。在并列情况下，最终顺序可能不同，因而不能将整个实验描述为无条件逐位一致。吞吐量结果针对顺序同步提交，不足以建立开放到达过程下的尾延迟表现。

## 7.7 冷构建、常驻显存与已验证规模

通过冷构建、常驻显存与已完成的大数据集搜索评估 RQ1；E1 显存比较也为 RQ4 提供依据。

### 7.7.1 冷构建与生命周期边界

与缓存搜索扫描不同，表 7.12采用每数据集三次正常空缓存训练 / 添加重复。六数据集完整记录保留在仓库补充材料。Vogue 训练 298–303 ms，正常合计约 0.56 s，归档 1.33 s（含 1.056 s 训练）为一次偏慢观测；Stella 为 10.54–10.65 s，接近历史 10.40 s。

构建与启动边界沿用sections A.2与3.3；图 7.10比较这些重复与基线配置。

在 Stella 上，CAGRA int8 成功的隔离构建按图出度不同，包含数据集量化后约为 74.55 和 82.45 s；IVF-PQ 三个配置的实测构建中位数为 86.59 s。在报告边界内，JHQ 重复的 10.54–10.65 s 明显更短；后续质量与常驻成本分别评估。

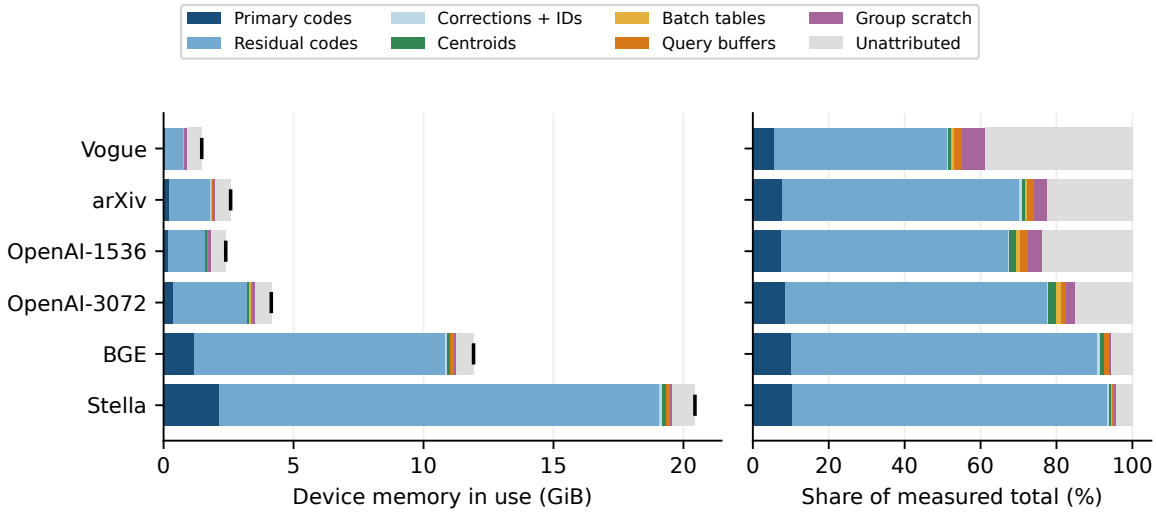



**图 7.11.** 两组查找表、128 探针、$\alpha = 100$ 下的常驻显存分解。组件大小由模型计算，标记为实测已用总量，斜线区域为未归因部分。总量包含上下文与工作区，前文构建测量另报采样峰值，这里的常驻标记不是峰值。

每数据集固定 128 探针工作点的五次独立构建中，最大 recall 范围为 arXiv 的 0.0012；正常计时范围只取三次无阶段插桩构建。较宽启动边界的差别明显：Stella 热输入页 18.2–18.6 s，本会话首次读取 129.9 s。阶段诊断同步各阶段并改为一个流水缓冲区，因此其计时不混入正常构建范围。20 ms 设备采样观察到 BGE 在 add 开头 26.08 GiB、Stella 在 repack 时 22.58 GiB。BGE 三个分配乘积缓冲区共 12 GiB，属于早期流水生命周期，不能直接加到后期 23.05 GiB 的 gather 载荷（式 (4.13)）。因此常驻显存不足以代表构建容量。

### 7.7.2 常驻分配与压缩发挥作用的位置

图 7.11将第4.4节的组件模型与实测常驻总量对应，保留未归因余量；前文构建测量另报采样峰值。

在大数据集上，残差码是主要占用：BGE 的已用总量为 11.93 GiB，其中约 80.7% 为残差码；Stella 为 20.44 GiB，其中约 82.9% 为残差码。仅对应的原始 FP32 向量数组就分别需要 38.50 和 67.81 GiB。因此，JHQ 通过保留压缩层次表示，使这些已测试的数据能够在单卡常驻搜索。降低 $\alpha$ 会减少每条查询访问、评分的残差码，但不会释放已经存储的残差数组。同样，笛卡尔 LUT 因子化减少的是查询相关查找表存储，不是数据库残差码的精度。

工作区同样重要。在这些 $\alpha = 100$ 测量中，报告的列表分组临时区约为 93.8 MiB，即使实际执行选择单线程块路径，也为批容量 1,024 和最多八组预留。因此，实际组数变小，不意味着分配按比例缩小。另一方面，这些常驻样本没有捕获临时构建峰值。基线测试程序也在测量索引显存之后才初始化部分搜索内存池，因此，不能把构建后的索引增量当作各方法完整进程峰值显存预算。

E1 测量提供了表 7.13中的独立跨系统视图。从 JHQ 进程总量减去基线测得的 507.2 MiB 上下文后，约为 CAGRA fp32 索引分配的 0.29–0.39。JHQ 数字包含工作区，CAGRA 数字则是构建时索引增量。这些是参数扫描范围，不是显存匹配的同召回率运行点，也不是相同峰值预算的比较。同表的 CAGRA int8 分配与 JHQ 处于相近量级，相对 fp32 的压缩是多个设计共有的特征；组件模型见表 4.1。

**表 7.13.** E1 分配范围。JHQ 为进程总量减去 507.2 MiB 上下文估计，包含查询工作区；两个 CAGRA 列均为构建时测量的索引分配。范围覆盖参数扫描。

| 数据集 | JHQ 净进程 GiB | CAGRA fp32 索引 GiB | CAGRA int8 索引 GiB |
|---|---|---|---|
| Vogue | 0.90–1.01 | 2.89–2.91 | 0.89–0.91 |
| arXiv | 2.01–2.19 | 6.84–6.98 | 2.01–2.15 |
| OpenAI-1536 | 1.79–2.33 | 5.95–5.97 | 1.67–1.68 |
| OpenAI-3072 | 3.54–3.82 | 11.67–11.68 | 3.10–3.11 |

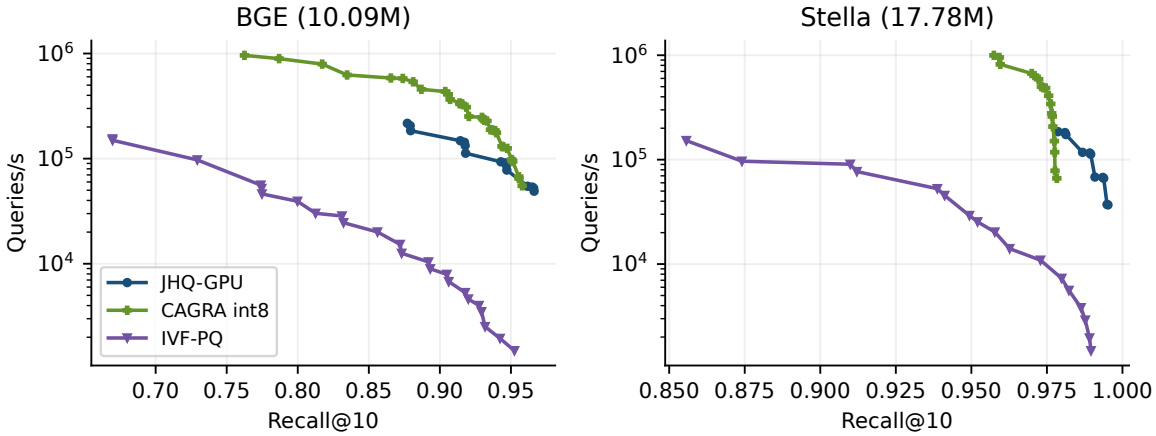



**图 7.12.** 大数据集上的主机到主机前沿，包含 CAGRA int8 成功的隔离重跑。BGE 使用 1,000 条查询，Stella 使用 999 条。曲线端点是已测试网格的终点，不是方法的内在召回率上限。

### 7.7.3 大数据集可运行性与竞争区间

BGE 含 10,091,524 个向量，Stella 含 17,776,615 个向量，均为 1,024 维、32,768 个 IVF 列表。Stella 查询采样自语料库，因此结果描述的是该基准，而不是独立收集的外部查询分布。图 7.12 包含成功的 CAGRA int8 和 IVF-PQ 测量。对于计算存储需求超过可用设备容量的全精度 CAGRA、RaBitQ 配置和 FP32 暴力搜索，大规模比较没有尝试运行。这些缺项是预先进行的可运行性排除，不是观察到的显存耗尽错误或实测召回率失败。

成功的隔离重跑显示，CAGRA int8 能在被测配置下装入 Stella。在 BGE 的共同采样目标 0.88/0.90/0.92/0.94/0.95 上，int8 约比 JHQ 快 1.35–2.96×；在 int8 的最高实测召回率 0.9579 处，JHQ 插值 QPS 是 int8 的 1.086×。因此，优势在高召回率末端附近反转。BGE/Stella 上，int8 的最高被测召回率为 0.9579/0.9783，JHQ 为 0.9663/0.9952，PQ 为 0.9525/0.9896。

Stella 上一个具体高端比较为：JHQ 在召回率 0.978679 时达到 185.4 kQPS，int8 在 0.978278 时为 66.3 kQPS，速率比为 2.80×，两者召回率略有不同。这是一对明确的实测点，不是统一的匹配召回率优势：JHQ 很多更高召回率区间没有 int8 覆盖，而且其最高召回率点也以吞吐量换取质量。附录 A记录修正运行的排除规则与测试程序诊断。

结果证明的是已测试规模和有价值的高召回率区间，而不是任意规模的可扩展性。最大已完成搜索是在单 GPU 上处理 17.78 百万个向量；本章没有多 GPU 实验，没有保持数据分布不变的扩展曲线，只有采样构建峰值，没有逐分配精确的容量证明。Stella 搜索使用实际的 999 条查询池。

## 7.8 数值稳健性、异常与召回率归因

稳健性检查界定 RQ2 的精确性声明、RQ3 的风险评估与 RQ4 中较小的跨系统差距。

### 7.8.1 质量分辨率与重复执行

当 $k = 10$、查询数为 1,000 时，多找回一个邻居会使总体 Recall@10 改变 $10^{-4}$；500 条留出查询时，分辨率为 $2 \times 10^{-4}$。Stella 的 499 查询划分略粗。这种离散分辨率不是不确定性区间，但有助于理解前面的小差异。第四位小数的变化可能只对应一两个结果槽位，而小的平均损失仍可能掩盖若干答案变化较大的查询。

重复调用与重复索引构建回答不同问题。E1 保留了 369 个基线重复调用标识符标记，包括并列顺序变化；这些点仍在前沿中。401 的计数口径及跨协议检查在第A.2节中单独说明。

五轮范围描述固定构建内部的计时变化。另用前沿斜率量化水平 recall 位移的敏感性：在 0.97 邻近的连续 log-QPS 线段上，每 $10^{-3}$ recall 对应约 1.01–3.17% 的速率变化，此处尚未应用接近目标即取实测点的规则。补证五次构建在固定工作点的 recall 范围为：Vogue 0.0008、arXiv 0.0012、OpenAI-1536 0.0004、OpenAI-3072 0.0002。仅作水平敏感性尺度乘以上述局部斜率，对应约数个百分点，而非更大的 OpenAI 相对 CAGRA fp32 差距。未重测完整训练前沿或基线构建，因此它支持局部稳定性判断，不给匹配比值的普遍上界。第7.2.3节则在明确缓存模型边界下直接证明去掉 TF32 后 OpenAI 优势仍在；arXiv 与 LUT 接近平手的结论仍保持较窄解释。

在各数据集 RaBitQ 配置间，“五轮最大值减最小值 / QPS 中位数”的中位数分别为：Vogue 3.79%、arXiv 3.34%、OpenAI-1536 2.87%、OpenAI-3072 5.25%。这些描述性波动与 Vogue 小幅领先的量级接近，而不是匹配比值的置信区间。原始调用轨迹也包含长调用，在原本较快的配置中偶有 30–60 ms 观测。敏感性检验对 1,000 条查询的一次调用，按 $10^6/\,\mathrm{median(call\ latency\ in\ ms)}$ 重算 QPS，以减小长调用的影响。召回率 0.97 处，得到的 JHQ/RaBitQ 比值为 1.063、0.995、1.118 和 1.100。三个有利方向与 arXiv 接近平手的格局保持，因此结果不只是长调用造成的。

### 7.8.2 精确性必须限定到具体阶段

表 5.1的阶段契约区分计算分数选择与原始空间 kNN；lane 舍入、截断并列与进程变化限定不同阶段。

归档重建 pilot 测量了相对全探针重建评分的剩余差距。七个工作负载—预算变体各用 128 条查询，全部 probe 候选的 FP64 评分相对所选预算的 GPU 输出，recall 增益为 0–0.002344。在 arXiv 的 128 探针配置，$\alpha = 200$ 与全 IVF 重建答案集合在全部 128 条查询上一致；100 时 recall 为 0.9734375，全 IVF 为 0.9750，即在联合路由 / 重建缺口 0.0250 之外，再有 0.0015625 的预算缺口（第6.2节）。这些是 pilot 特定的总体差异，七个变体包含同一数据集—探针工作负载的两个预算。FP64 重建评分也不同于 GPU 运算，因而不能在每个变体中单独隔离纯预算效应。进一步分开路由与表示，需要逐阶段 GT 覆盖检查。

# 第 8 章讨论与未来工作

## 8.1 三项跨层认识

### 8.1.1 有价值的表示暴露有价值的复用单位

第5.2节的坐标组 LUT 将计算所得距离复用于多个候选；但分组结果与 Mailman 表明，仅节省存储不足以决定吞吐（图 7.4）。共享内存放置与 word 访问须联合考虑（sections 5.2与5.3）；两组是该 GPU 上的经验默认。码本改变可能破坏 Cartesian 兼容性，因此需将重建误差与评分器可达的 recall–成本前沿共同评价。

### 8.1.2 有效并行受重复状态与选择成本限制

图 7.9的批次趋势反映查询并发充足后，重复暂存、选择与合并的成本。Vogue 的路径相关前沿与 BGE 的 regret 暴露 CV 未描述的绝对工作量及分解成本（sections 7.2与7.6）。这些集成比较仍与单独共享界消融分开。更丰富的调度器可以结合候选数、保留容量与局部缓冲成本，同时保留查询级路径；其校准成本与留出工作负载性能仍需评测。

### 8.1.3 执行变快会改变避免精排的价值

考虑延迟模型：非精排成本固定为 $T_0 > 0$，高 / 低预算精排时间满足 $F_h \geq F_\ell > 0$，共同精排时间倍率为 $0 < \eta \leq 1$。低预算加速比及其导数为

$$S(\eta, T_0) = \frac{T_0 + \eta F_h}{T_0 + \eta F_\ell}, \tag{8.1}$$
$$\frac{\partial S}{\partial \eta} = \frac{T_0(F_h - F_\ell)}{(T_0 + \eta F_\ell)^2} \geq 0, \qquad \frac{\partial S}{\partial T_0} = \frac{\eta(F_\ell - F_h)}{(T_0 + \eta F_\ell)^2} \leq 0.$$

当 $F_h > F_\ell$ 时，精排加快降低削减预算的收益，固定成本降低则提高该收益。模型固定了非精排工作；预算相关扫描维护和资源阈值仍需实测。因果内核比较应固定候选集合及预算决策（sections 7.5与7.5.6）。

FULL 结果与采样尾部说明，该成本决策还需质量约束及足够复用（表 7.7与图 7.6）。

## 8.2 两项深入的负结果案例

### 8.2.1 Mailman 式共享求和：算术复用与散射成本

primary 距离的符号码形式提示：共享位模式的候选可以复用求和。Mailman 提供有限字母矩阵–向量乘法的一般思路 [20]。历史 pilot 固定 Vogue、arXiv、OpenAI-3072 候选流，在 128 probe、batch 8/1,000 上测试 GPU 实现。基线复刻 packed 两表评分循环；替代方案包括原子桶累加、私有直方图及 4/6/8 组宽，数学分数和输入编码不变。

六个单元中最快替代方案只有基线评分吞吐的 0.187–0.581 （图 8.1）。阶段 profile 把大多数插桩周期定位到桶累加。list 内偏置模式造成重复桶目标；私有直方图避免原子操作，却增加状态与共享内存读改写。缩小组宽、绕过主要桶可以改善部分变体，但都未追上基线。这支持停止这些实现与输入上的路线，不说明所有有限字母复用都无效：胜出的两表基线已经沿另一方向复用坐标和。

pilot 只写出 primary 分数，没有返回最终近邻。FP32/FP64 检查揭示 cutoff 附近数值差异，不等于完整流水线等价。以实测评分比 $s$ 和估计可替换延迟占比 $f$，乐观替换模型为

$1/[(1-f)+f/s]$；由于未集成，其预测减速只是模型。辅助无损字典路线也在结构统计阶段停止，有限字节 / 工作节省没有支持 GPU 实现。两条路线均未向主前沿贡献系统实测点。

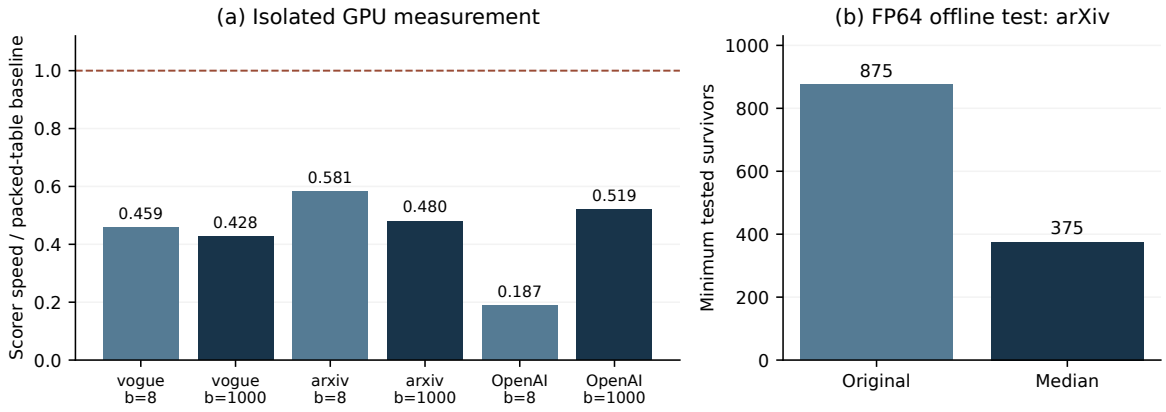



**图 8.1.** 两种不同证据边界。左：RTX 5090、128 probe、Vogue、arXiv、OpenAI-3072 和两种 batch 下，最快 Mailman 评分器 / packed 表评分器速度，全部小于 1。右：arXiv 的 400 查询 FP64 离线测试划分中，满足原 JHQ recall 目标的最小 survivor 数。候选减少不是 GPU 提速；两图均由论文 audit 脚本从归档记录重新生成。

### 8.2.2 面向排序的编码：局部收益不等于通用替代

排序导向研究在相同名义码长下，改变 1-bit 阈值或打分，尝试以更少 survivor 保留有用近邻。主比较在导出候选上用 FP64 回放，划分为 400 校准、200 验证、400 最终测试。方案包括似然权重、中位数阈值、学习排序损失、线性 MMSE 解码与 list 相关阈值。较低的重建 / 内积 MSE 可能使极少数最近候选的顺序变差；改变 primary 电平还会改变残差参照点，因此不同目标暴露不同失败模式。

arXiv 测试集中，中位数阈值把达到原目标所需的最小被测 survivor 数从 875 降至 375。按同一严格标准，成功未推广到另外两个工作负载。共享旋转下的高斯直觉也不能消除真实语料均值偏置，因为旋转保留均值向量的范数。居中编码可以修复部分失配，但必须同时评价 primary 排序与重建分数上限。因此停止的研究支持工作负载相关改进，而非普遍排序最优的新量化器。

后续仅 median 方案的 GPU 跟进是另一层证据：把改变后的阈值、电平及重新训练的残差导入相同搜索流水线，分别扫预算，再按实测 recall 阈值比较。在 arXiv 原 $\alpha = 100$ 目标，batch 8 和 1,000 的比值约为 1.40 和 1.21。Vogue median 前沿最高 0.9630，未达到原工作点 0.9644；OpenAI-3072 基本持平。这修正了“从未做 GPU 测试”的笼统表述，同时保留不采用通用替代的决定。跟进使用独立索引状态、查询池和三轮协议，不能把其吞吐插入 E1。

## 8.3 内部与外部有效性威胁

表 8.1汇总主要解释的威胁。共享输入、计时与来源控制在第 3 章定义，第7.8节在归档边界内评价重复执行、固定工作点重建及精度敏感性。

**表 8.1.** 可能反转或缩小结论的威胁。

| 受影响主张 | 替代解释或反转 | 已有控制与剩余验证 |
|---|---|---|
| 执行收益 | 分数、并列或索引改变产生表面提速 | 配对状态与输出检查；T4 严格 CPU 排序及表形式数值诊断；覆盖有限，候选 dump 已不保留 |
| 预算质量 | 有限池抽样或漂移掩盖有害查询 | 留出 GT、完整轨迹、子集和核对及 FULL 枚举；未来查询及任意子集不变性未测 |
| 竞争差距 | 网格密度、慢调用或 API 开销改变排名 | 两种 recall 估计、保留慢调用、int8 及 TF32/FP32 敏感性；T5 已显示小 batch 反转，跨设备比较仍未测 |
| 容量 / 构建 | 瞬时分配超过常驻样本 | 阶段字节、重复空缓存构建及 20 ms 设备峰值；更短暂分配与完整重训前沿未测 |

外部有效性限于一张 GPU、平方 L2 top-10 和六份准备后的嵌入数据。批次相关工作区间见第7.2.5节。按位置移除查询不认证没有近重复嵌入，这些导出也不是带相关性标签的检索基准。数据库更新、并发流量、部署延迟目标和变化查询总体仍未测试，常驻索引结果也不提供十亿规模或多设备容量证据。这些是把观测区间迁移到部署前仍需完成的检查。

## 8.4 可检验的后续研究方向

### 8.4.1 带风险控制和复用成本的输出一致性策略

后续策略可在有效的选择后置信界下，最小化校准与预计复用时间，并认证逐查询的参考一致性损失。输入为匹配执行路径的逐 query 不一致曲线和固定参考，变量包括样本量及 survivor 预算。绝对 GT 目标还需要独立标签验证，或参考与 GT 之间的额外假设。平稳性、逐查询抽样与共同分数 / 排序契约必须明确，漂移触发器应使旧证书失效并重新认证。

最低实验应在留出池比较固定样本规则、秩曲线复用和成本策略，记录总耗时与尾部一致性损失。旧四单元离线回放的序贯控制器已因额外搜索 launch 吞掉样本节省而失败，单纯重复序贯测试不足以形成改进。新的问题是：复用中间信息与质量证书能否联合改善总成本。违反风险限制，或比相同质量的固定样本对照没有省时，即失败。

### 8.4.2 考虑容量的残差放置

大型索引中常驻残差占主导。可研究 primary 常驻、残差 block 在显式显存上限内缓存，miss 从主机内存取回。固定编码、路由、survivor 集合与精排算术，隔离放置与表示质量。最低实验改变显存上限、batch 和查询局部性，测 hit、传输字节、重叠、尾延迟和吞吐。只有在可接受的总时间前沿上扩展容量才算有效；原本可常驻规模上更慢的 cache 不足以支持方案。这是未实现的存储扩展，与已有残差下界 pilot 不同；降低 $\alpha$ 也不实现它，因为完整数组依然分配。

## 8.5 有条件的部署建议

压缩常驻层次能放入设备、且目标 recall 位于已测竞争区间时，JHQ-GPU 值得评估。E1 支持 batch 1,000 的高 recall 吞吐；T5 则显示 recall 0.97、batch 8 下，四个数据集均由 CAGRA fp32 领先。batch 32 的 OpenAI-3072 在插值下接近平手、实测阈值有利于 JHQ，其

余三个有利于 CAGRA。低 recall 优势同样依赖数据集和网格。分组调度能改善 JHQ 内部执行，但不保证跨系统领先（第7.2.5节）。

以已测 FULL 工作点开始校准，再用部署查询验证质量并直接计时复用成本。在实际批次制度下比较总时间与常驻显存，适用边界见第8.3节。

# 第 9 章结论

## 9.1 四个研究问题的答案

**RQ1：构建与容量。** GPU 训练、batch 编码、IVF 放置及 packed 数组形成可搜索常驻层次。最大被测数据库为 17.78 百万向量、常驻 20.44 GiB，补证 train+add 为 10.54–10.65 s (第7.7节)。Stella 采样构建峰值 22.58 GiB，BGE 在 add 入口达到 26.08 GiB。独立重复构建和启动区间更完整地说明了已测容量边界。

**RQ2：正确高效的执行。** 继承 Cartesian 编码允许实数精确拆表，完整键保留 / 合并及共享界论证说明分组选择条件。两组表在 22/24 单元领先；相对 lane 精排参考，集成调度在 batch 8 的吞吐几何平均为 2.727×，batch 1,000 只有 1.015× （sections 7.3与7.6)。有效并行因 batch 和工作分布而变；T4 严格 CPU 参考与全部 55,296 个分组实例完整键顺序一致，56 个单 block 集合差异全部是截断并列（第7.4.1节)。

**RQ3：精排预算与质量。** 无标签参考一致性无需学习预测器即可选择工作负载 survivor 工作量。FULL 在 12/12 单元满足所测相对 recall 损失目标，最大损失 0.0006；$S = 128$ 一个单元则有 40/64 次越阈（第7.5节)。参考分析解释有条件的预算搜索，不给总体 GT 保证。10 个 FULL 单元的一般完整键嵌套仍有条件，但十二个 FULL 有限网格选择均与枚举一致。四个单元的直接校准加复用测量支持合适工作负载的收益，并保留困难工作负载的质量 / 时间权衡。

**RQ4：竞争区间。** 0.97 recall 的大 batch E1 插值比较中，JHQ 在三个数据集第一，arXiv 与 IVF-RaBitQ 接近平手（第7.2节)；Vogue 领先依赖分组路径。OpenAI E1 使用 TF32。同一缓存模型下移除 TF32 后，D 的跨会话比较对归档基线仍有数值优势。独立 T1 同期锚点也保持方向，幅度受网格影响（第B.4.1节)；两种干预均未完整重训 FP32 模型。低 recall 由 CAGRA int8 领先，部分高目标由 IVF-RaBitQ 领先。T5 收窄该区间：recall 0.97、batch 8 下，四个数据集均由 CAGRA fp32 领先；batch 32 的边界依赖 recall 估计（第7.2.5节)。排名因此依赖 batch、质量和网格，显存另行报告；跨设备排名仍未测。

## 9.2 系统原则与展望

表示、执行与精排工作量必须在同一质量目标下评价。后续应检验复用校准信息能否降低有效风险控制的成本，以及残差放置能否在尾延迟预算内扩展容量（第8.4节)。

# 参考文献


[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Anthropic. Claude: Implementation, experiment and review assistance record, 2026. URL `https://github.com/lrlorry/8800_report/blob/main/docs/ai-use-record.md`. Use documented by retained project and experiment records; record compiled 29 October 2026. Complete original prompts, interaction dates and model versions remain unavailable.

[5] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3 (4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[7] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[8] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[9] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[10] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[11] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[12] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi:

10.1109/CVPR.2013.379. URL https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html.

[14] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL https://proceedings.mlr.press/v119/guo20h.html.

[16] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL https://arxiv.org/abs/1702.08734.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL https://www.cs.yale.edu/publications/techreports/tr1402.pdf.

[21] Rui Luo and Mengxuan Zhang. JHQ-GPU: Representation-driven GPU execution and workload-adaptive hierarchical refinement, 2026. URL https://github.com/lrlorry/JHQ_GPU/blob/70da383056b50f4e9b8e9a8030bc9b92281ba537/paper_adc2026/ADC/mainADC.tex. Project manuscript source at a fixed public revision; its historical measurement versions are distinguished in this thesis.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html.

Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html. PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] OpenAI. ChatGPT: Project review assistance record, 2026. URL https://github.com/lrlorry/8800_report/blob/main/docs/ai-use-record.md. Use documented by retained review excerpts and the project disclosure; record compiled 29 October 2026. Complete original prompts, interaction dates and model versions remain unavailable.

[30] OpenAI. Codex: Selected thesis-preparation interactions, 2026. URL https://github.com/lrlorry/8800_report/blob/main/docs/ai-use-record.md. Selected original user prompts retained from the current conversation; record compiled 29 October 2026. Earlier timestamps and model versions are unavailable; this is not a complete interaction export.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M. Accessed 28 October 2026.

[32] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M. Accessed 28 October 2026.

[33] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[34] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL https://huggingface.co/datasets/macrocosm/arxiv_abstracts. Accessed 28 October 2026; exported embedding preparation is project-specific.

[35] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding. Corpus split; accessed 28 October 2026.

[36] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3. Italian configuration; accessed 28 October 2026.

[37] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025. doi: 10.1109/TBDATA.2025.3581085.

URL https://ssomesh.github.io/docs/papers/tbd25.pdf.

[38] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html.

[39] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf.

# 附录 AReproducibility and Experiment Manifest

This appendix records the contracts needed to interpret the experiments. The expanded protocol, executable names, raw-source paths and result-to-figure mapping are retained in `docs/experiment-protocol.md` in the report repository.

## A.1 Source, Environment and Trained States

The principal environment is RTX 5090, driver 595.71.05, CUDA 13.0 and, for E1, cuVS 26.08.01. JHQ targets compute_80 PTX and uses driver JIT compilation. 表 A.1 separates run-header evidence from versions stated in reports. The analysis checkout in `data/ch7/audit.json` identifies the inspected workspace; a run-head commit identifies source visible to that run. Neither supplies a missing executable or trained-state fingerprint.

Dataset shapes and actual query counts are in 表 3.2. Comparisons reuse the exported vectors and squared-L2 ground truth per dataset. Search sweeps load the recorded `arms6_cache` states; cold-build records start without trained state. Input ledgers hash the consumed analysis records, rather than every vector file, cache or binary. Initial large-baseline attempts that retained an old index are superseded by isolated reruns. Stella's skipped batch-1,000 files contribute no search points; corrected large-search records use its 999 queries.

## A.2 Measurement Contracts

E1 reports the median of five round throughputs, each computed as total queries divided by elapsed host-to-host time. A round normally uses 24 calls; a warm-up threshold reduces this count at 274 points. The Python decision interval includes first-call conversion and validation, so it differs from steady-state latency. Resource reuse, preallocated outputs, pinned input staging and timed int8 query quantisation improve comparability; output-memory types and Python/C++ boundaries still differ. Device-only baseline timing is auxiliary to this common call boundary.

The 369 baseline repeat-call identifier flags include tie-order changes and do not alone exclude points. The earlier count of 401 additionally includes cross-protocol disagreements; the exact union and method counts are in the expanded protocol. JHQ does not populate the same flag. Cold-build train/add timers exclude object construction, host QR and constructor

**表 A.1.** Recorded component provenance. Report and script identities are distinguished from raw run headers; full executable and input paths are in the expanded protocol.

| Component | Recorded source evidence |
|---|---|
| E1 final pipeline | Environment logs include `a31c22d`; extensions retain separate metadata |
| Lane refinement (v61) | Raw header `b37efa4` |
| Grouped scan (v62) | Report `862543a`; run script `e0d0518` |
| Shared bound (v63) | Source and build flags archived; no inferred run head |
| Batch scheduling (v65) | Raw header `d425ab8` |
| Radix probe (v69) | Comparative runs and source archived; binary SHA unrecorded |
| Budget / table sweep | Raw header `d5e1526` |
| Large int8 rerun | Stella degree-64 metadata `ad433a4` |
| Historical budget | Raw header `bf31b41`, v57 family |

library warm-up. Construction diagnostics sample device used memory, including context, every 20 ms; synchronised one-buffer phase runs remain separate from normal pipeline measurements.

## A.3 Budget Harness Contract

The campaign explicitly sets the eight-point grid beginning at 200, overriding a fallback string beginning at 100. Even query indices form the 500-query calibration pool; odd indices form the 500-query held-out pool, or 499 for Stella. Each $S \in \{32, 64, 128\}$ has 64 seeded draws without replacement. The initial warm-up traverses all queries, while its outputs and GT remain outside the agreement decision. Scan-schedule calibration uses separate database pseudoqueries.

Each fixed arm receives one warm-up and three timed calls. Calibration times reference/candidate searches and the $O(Sk^2)$ host agreement comparison with graph capture bypassed; prior $O(Sd)$ sample gathering and separate scan calibration are excluded. The logged p95 is sorted index $\lfloor 0.95(64-1) \rfloor$. The helper returns a grid index and leaves the last tested multiplier installed: callers must apply `set_alpha(A[index])`. Selected-arm rates in the original campaign reuse already timed fixed arms.

Original enumeration checks cover 2,304 subsampled decisions, excluding the twelve FULL calls. Supplementary enumeration also matches all twelve FULL decisions. At FULL size, two Vogue cells use four groups and ten cells use distance-only selection; common cutoff ordering remains the condition for a general binary-search guarantee. Finite-pool prediction models uniform subsets and requires per-query counts invariant to subset membership and position under the relevant execution regime. All 18,432 sampled grid counts equal their recorded full-pool subset sums. Mapping acceptance to GT-loss crossings additionally uses the held-out fixed-arm quality labels. These checks establish the recorded cases, rather than unknown query-population quality.

## A.4 Supplementary Evidence

Records under `JHQ_GPU/experiments/thesis_evidence/` retain campaign identities and timing boundaries. A1 separates first-read, normal warm-input and synchronised phase runs on six datasets; five-build recall is measured at fixed 128 probes. C1 checks four complete database layouts and sampled re-encodings. Its FP64 norm checks use the stored rotation matrix, rather than actual GPU rotation outputs. A3 checks every OpenAI query row and 10,299 strided base rows; original entity-ID mappings are absent.

A2 extracts the recorded executable's sm_80 PTX and reassembles it with CUDA 13.0.88 `ptxas` for sm_120. The observed 64-bit bound read and atomic-min publication use an aligned query-key array with eight-byte stride. The argument uses PTX alignment, scope and proxy semantics; toolkit SASS corroborates it without supplying the driver's loaded JIT code. Binary SHA and operation listings are in the expanded protocol and the archived evidence.

D alternates v69/v72 over the E1 OpenAI grid, reusing TF32-trained states and changing add/query GEMMs. Its baseline denominators remain archived E1. The later contemporaneous T1 anchors are reported separately in 第B.4.1节. Neither experiment constitutes FP32 retraining.

B reproduces the original `srand(12345)` decisions and adds per-query curves, FULL enumeration and direct reuse traces. Direct timing uses a separate seeded sequence, five rotated-policy rounds and 500 repetitions of the held-out batch. It includes gathering, calibration, budget installation and first-batch graph setup; construction, schedule calibration and warm-up remain outside that interval. Schedule calibration startup costs are separately reported.

## A.5 Strict Selection, Small Batches and Artifact Access

T4's v73 source matches v71 after removing its two debug-export blocks. CPU sorting consumes GPU-produced primary and retained refined scores. The exported-score equality check covers returned valid primary entries; the padding counter checks valid-entry counts. Natural short pools are covered, while deliberately empty or adversarial lists remain separate tests. Table-form comparisons align candidate positions at 128 probes, $\alpha = 100$, 256 queries, shared bounds enabled, $g_{\text{scan}} \in \{0, 2\}$ and $g_{\text{LUT}} \in \{1, 2, 4, 8\}$. Temporary candidate dumps were deleted: retained JSONL/stdout and LUT summaries support the reported finite-case counts. The same-model Vogue cross-process discrepancy remains unexplained; within-run T4 selection checks address a different question.

T5 partitions queries into fixed small batches and includes staging, transfers, applicable int8 query quantisation and synchronisation. Each point is the median of five round throughputs; null-call costs remain included. JHQ/CAGRA reuse E1 grids and RaBitQ has its own recorded sweep. Numeric JSON supplies recall thresholds. Interpolated estimates retain qualifying endpoints below the minimum frontier recall; strict measured thresholds remain separate. Python/C++ interfaces and warm-up policies differ, and these measurements do not establish concurrent-service latency.

Source, records, manifests and offline checkers are archived in `evidence/ch5/v71/` and `evidence/review/`. With the required inputs, run `make ch7-data`, then `make pdf` or `make pdf-zh`. GPU reruns additionally require original vectors, trained states, matching builds and environment.

# 附录 B补充结果与正确性检查

本附录记录支持第 3–8 章的参数覆盖、逐阶段验证及次要结果。各测量家族保留各自版本与计时边界，来源映射见附录A。

## B.1 完整参数记录

E1 比较有 1,590 个有效实测配置，覆盖见表 B.1。逐点完整参数 JSON、recall、主机到主机 QPS 及源文件名保存在 `data/appendix/e1-configurations.csv`。`scripts/build_appendix.py` 生成该记录，输入身份见`data/appendix/audit.json`。下列集合表示观测值，不假定其笛卡尔积全部测量过。

**表 B.1.** 包含覆盖扩展的 E1 精确配置数。计数为参数点，不是独立索引构建。

| 方法 | Vogue | arXiv | OpenAI-1536 | OpenAI-3072 | 总计 |
|---|---|---|---|---|---|
| JHQ-GPU v69 | 27 | 29 | 37 | 39 | 132 |
| CAGRA fp32 | 64 | 64 | 64 | 64 | 256 |
| CAGRA int8 | 64 | 64 | 64 | 64 | 256 |
| IVF-RaBitQ | 132 | 132 | 264 | 198 | 726 |
| IVF-PQ | 54 | 54 | 54 | 54 | 216 |
| GPU brute force | 1 | 1 | 1 | 1 | 4 |
| 总计 | 342 | 344 | 484 | 420 | 1590 |

**JHQ-GPU**。 E1 均用 $B = B_r = 8$、$M = d/8$、$k = 10$ 和 batch 1,000；Vogue 为 4,096 个 IVF 列表，其他三个为 8,192。Vogue 观测探针 16、32、64、96、128、192、256、384、512，$\alpha \in \{16, 32, 64, 128\}$，共 27 个实际组合。arXiv 再包含探针 1,024，$\alpha \in \{25, 50, 100, 200\}$，共 29 个组合。两个 OpenAI 均用探针 32、64、128、256、512、1,024；倍率集合分别为 $\{4, 8, 16, 32, 64, 128, 256, 512\}$ 和 $\{2, 4, 8, 16, 32, 64, 128, 256\}$，共 37 / 39 个组合。所报调度路径在 Vogue 用四组，其余用单 block。历史 v69 的 OpenAI 旋转 / 质心 GEMM 为 TF32；v72 FP32 敏感性另成家族（第7.2.3节）。

**CAGRA fp32 与 int8**。 每种表示每数据集 64 点：图度数 32 / 64、中间度数 128、`ivf_pq` 构建，`itopk_size` 为 32、64、96、128、192、256、384、512，搜索宽度 1、2、4、8。int8 数据缩放记录 quantile 0.99，主机搜索边界包含查询量化。

**IVF-PQ**。 八位子码在 768 维使用 192、384、768 字节编码，OpenAI 使用 384、768、1,536 字节。每种长度测试 fp16 / fp32 LUT 及探针 16、32、64、96、128、192、256、384、512，每数据集 54 点。记录的精排比例为一，列表数与 JHQ 相同；编码长度和显存预算不同。

**IVF-RaBitQ 与暴力搜索**。 RaBitQ 包含每维 4、8、9 位、探针 16、32、64、96、128、192、256、384、512、768、1,024。768 维与 OpenAI-1536 含 LUT16、LUT32、QUANT4、QUANT8，OpenAI-3072 含 LUT32、QUANT4、QUANT8；另用 4,096 列表 OpenAI 索引扩展默认 8,192 列表覆盖。精确组合见参数记录。每数据集还含一个 FP32 平方 L2 暴力搜索点。缺少某种模式或更高 recall 点不等于实测失败。

**辅助网格**。 LUT 实验固定 $\alpha = 100$，六数据集探针 8、32、128、512，$g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$，共 24 单元、96 计时臂。预算实验在相同六数据集用探针 128 / 512、$A = (200, 100, 64, 32, 16, 8, 4, 2)$，

每个 $S \in \{32, 64, 128\}$ 抽取 64 次、$\epsilon = 1$；FULL 另用全部 500 校准查询。留出为 500 条，Stella 为 499 条。直接复用为四单元、四策略、五轮，每轮 500 个重复留出批次。边界见sections A.3与A.4。

## B.2 正确性与数值证据

表 B.2区分数值、算法与内存证据。第五章排序论证针对满足所列契约的计算分数；原始向量 recall 另行评测。

**表 B.2.** 验证覆盖及实际核对对象。

| 边界 | 已有证据与范围 |
|---|---|
| 表示 | 四库全行 offset / ID 审计，原始输入采样 FP64 重编码、存储矩阵几何，已识别 TF32 偏差 |
| Cartesian LUT | 实数因子化；96 个计时臂；T4 覆盖四种表形式的逐候选 FP32 误差与排名诊断 |
| 全局选择 | T4：270 配置、69,120 查询实例；55,296 个分组实例与严格 CPU 完整键排序的顺序一致 |
| 共享界 | 48 组配对；提取标量 64 位 PTX 及 toolkit SASS 支持完整 key；未捕获驱动实际 SASS |
| 调度 | 250 个变体 / 配置及 234,320 次查询比较，区分 ID 集合与并列顺序 |
| 探针选择 | 1,792 合成用例零失败；真实查询 ID / offset 检查，针对已计算路由分数 |
| 预算搜索 | 2,304 采样曲线单调、12 个 FULL 枚举一致；18,432 网格计数可由全池子集和复现 |

较小表示审计库在按 ID 关联的一遍 / 两遍构建中主码和残差字节相同，correction 差至多 $8.94 \times 10^{-8}$，列表分配有 1、1、40 行变化；Stella 验证自然两遍路径（第4.6.2节）。T4 已提供计算分数选择的严格 CPU 参考（第A.5节）；同模型 Vogue 跨进程受控诊断仍未执行：归档进程即使 probe / primary dump 一致，最终 recall 仍有 0.9642 / 0.9644。进程内选择检查并未定位这项独立异常。

## B.3 次要结果与证据状态

**不利运行区间。** 调度器最大记录 regret 为 batch 256 的 1.217×；Vogue E1 中调度 / 单 block 吞吐为 0.895–1.175。使 0.97 recall 领先的偏斜规则也会在低探针单元增加开销（第7.6节），这些不利结果与内部调度收益一起保留。T5 跨系统比较中，recall 0.97、batch 8 时四个数据集均由 CAGRA fp32 领先；batch 32 的边界依赖 recall 估计（第7.2.5节）。

**表示替代方案。** Mailman 式 GPU 评分器为独立 kernel 测量，端到端替换只是乐观模型。面向排序的主研究为 FP64 离线回放；另一个 median 编码 GPU 后续实验在记录目标上，arXiv 的 batch 8 为 1.402×、batch 1,000 为 1.208×，Vogue 未达严格目标，OpenAI-3072 接近平手。阈值、级值和残差训练同时改变，不能只归因于阈值（第8.2节）。

**校准与精度。** 有限池预测在查询计数稳定的条件下解释抽样风险，GT 损失映射使用留出固定臂质量。直接复用测量四个重复池的累计成本；精度干预在同一 TF32 训练缓存上改变 add / 查询 GEMM。完整 FP32 重训、未来查询漂移、跨设备排名及在线更新不属于这些实验。

## B.4 构建容量与精度锚点

表 B.3区分输入暂存与独立分配的乘积数组。源码默认三个缓冲区、配置 add 行数 65,536、每个输入 / 旋转浮点数组 128 MiB。这些是分配容量，不是当前子批大小或进程整体峰值。

**表 B.3.** 默认构建缓冲容量。末列为 $4n_b n_{\text{list}} a_{\text{assign}}/2^{30}$；用 $a_{\text{in}}$ 代入会得到错误的一般模型。

| $d$ | $n_{\text{list}}$ | $a_{\text{in}}$ | $a_{\text{assign}}$ | 分配乘积 GiB |
|---|---|---|---|---|
| 768 | 4,096 | 43,690 | 8,192 | 0.375 |
| 1,024 | 32,768 | 32,768 | 32,768 | 12.000 |
| 1,536 | 8,192 | 21,845 | 32,768 | 3.000 |
| 3,072 | 8,192 | 10,922 | 32,768 | 3.000 |

### B.4.1 同期精度锚点

T1 晚于 D，在同一会话重测 recall 0.97 附近的 JHQ 和竞争基线，启用 E1 工作区池。每个 JHQ 配置在基线前后各重复一次，每次五轮计时；每配置 QPS 对两次轮中位数取几何平均。每个 OpenAI 数据集有十二个配对配置，FP32/TF32 几何平均为 0.956 / 0.964。两种变体复用 TF32 训练缓存，改变 add / 查询算术。

**表 B.4.** T1，recall 0.97、batch 1,000。基线列依次为插值 / 严格实测阈值，FP32/TF32 列为插值。这里全部方法均为同期测量，与 D 的跨会话基线比值分开。

| 数据集 | 0.97 处 FP32/TF32 | FP32 / RaBitQ | FP32 / CAGRA |
|---|---|---|---|
| OpenAI-1536 | 0.980 | 1.126 / 1.053 | 1.752 / 1.633 |
| OpenAI-3072 | 0.973 | 1.328 / 1.316 | 1.825 / 2.019 |

两种估计在该有限网格均保留 FP32 数值优势。OpenAI-3072 的 RaBitQ 插值区间为 recall 0.9669–0.9796，揭示较强插值敏感性。这些锚点不替换 E1 完整网格，不建立通用优势幅度，也不是 FP32 重训。原始 JSON 及复算映射见 `data/full-review/audit.json`。

## B.5 材料获取

报告仓库提供参数记录、派生表、图源、分析脚本，以及`evidence/ch5/v71/` 下的 209 文件 v71 源码 / 测量包，可用 `python3 evidence/ch5/v71/check.py` 离线核对。另在`evidence/review/` 保留 T4/T5、T1 和 A2 消费的源码 / 记录子集；检查器核对哈希，并可用这个本地源码根目录重新运行两项离线审计。更多原始记录位于同级 `JHQ_GPU` 项目的清单路径，完整向量、训练状态与所有可执行文件没有打包在本报告仓库。离线复算需要所列输入，GPU 复现还需要相应构建和环境。

固定材料入口（含公开手稿源码及 v71 / T4 / T5 / T1 / A2 证据包）见 `docs/artifact-access.md`。